

Semantics Foundation for Cyber-Physical Systems Using Higher-Order UTP

XIONG XU, SKLCS, Institute of Software, Chinese Academy of Sciences, China and Inria, France

JEAN-PIERRE TALPIN, Inria, France

SHULING WANG, SKLCS, Institute of Software, Chinese Academy of Sciences, China

BOHUA ZHAN, SKLCS, Institute of Software & UCAS, Chinese Academy of Sciences, China

NAIJUN ZHAN*, State Key Lab. of Computer Science and Science & Technology on Integrated Information Science Lab., Institute of Software & UCAS, Chinese Academy of Sciences, China

Model-based design has become the predominant approach to the design of hybrid and cyber-physical systems (CPSs). It advocates the use of mathematically founded models to capture heterogeneous digital and analog behaviours from domain-specific formalisms, allowing all engineering tasks of verification, code synthesis and validation to be performed within a single semantic body. Guaranteeing the consistency among the different views and heterogeneous models of a system at different levels of abstraction however poses significant challenges. To address these issues, Hoare and He's Unifying Theories of Programming (UTP) proposes a calculus to capture domain-specific programming and modelling paradigms into a unified semantic framework. Our goal is to extend UTP to form a semantic foundation for CPS design. *Higher-Order UTP* (HUTP) is a conservative extension to Hoare and He's theory which supports the specification of discrete, real-time and continuous dynamics, concurrency and communication, and higher-order quantification. Within HUTP, we define a calculus of *normal hybrid designs* to model, analyse, compose, refine and verify heterogeneous hybrid system models. In addition, we define respective formal semantics for HCSP (Hybrid Communicating Sequential Processes) and Simulink using HUTP.

CCS Concepts: • **Theory of computation** → **Timed and hybrid models**; *Denotational semantics*.

Additional Key Words and Phrases: UTP, CPS, model-based design, semantic model

1 INTRODUCTION

Hybrid systems, also known under the more architecture-centered denomination of cyber-physical systems (CPSs), exploit networked computing units to monitor and control physical processes via wired and/or radio communications. CPSs are omnipresent, from high-speed train control systems, power grids, automated factories, to ground, sea, air and space transportations. CPSs can be complex, networked, systems of systems, and are often entrusted with mission- and safety-critical tasks. Therefore, the efficient and verified development of safe and reliable CPSs is a priority mandated by many standards, yet a notoriously difficult and challenging field.

*Corresponding author.

Authors' addresses: Xiong Xu, SKLCS, Institute of Software, Chinese Academy of Sciences, Beijing, China and Inria, Rennes, France, xux@ios.ac.cn; Jean-Pierre Talpin, Inria, Rennes, France, jean-pierre.talpin@inria.fr; Shuling Wang, SKLCS, Institute of Software, Chinese Academy of Sciences, Beijing, China, wangsl@ios.ac.cn; Bohua Zhan, SKLCS, Institute of Software & UCAS, Chinese Academy of Sciences, Beijing, China, bzhan@ios.ac.cn; Naijun Zhan, State Key Lab. of Computer Science and Science & Technology on Integrated Information Science Lab., Institute of Software & UCAS, Chinese Academy of Sciences, Beijing, China, znj@ios.ac.cn.

Permission to make digital or hard copies of all or part of this work for personal or classroom use is granted without fee provided that copies are not made or distributed for profit or commercial advantage and that copies bear this notice and the full citation on the first page. Copyrights for components of this work owned by others than ACM must be honored. Abstracting with credit is permitted. To copy otherwise, or republish, to post on servers or to redistribute to lists, requires prior specific permission and/or a fee. Request permissions from permissions@acm.org.

© 2022 Association for Computing Machinery.

1049-331X/2022/1-ART1 \$15.00

<https://doi.org/10.1145/3517192>

To address design complexity under the necessity of verified CPS development, model-based design (MBD) [18] has become a predominant development approach in the embedded systems industry. The classical development process of MBD can be sketched as: (1) to start from an abstract model, ideally with a precise mathematical semantics; (2) to conduct extensive analysis and verification on this abstract model, so as to locate and correct errors as early as possible; (3) subsequently, to refine that abstract, system-level, model to a semantically concrete specification and, eventually, to executable code, by model transformation.

To assist such a development process, design-by-contract (DbC) is an important tool. It allows to define the abstract model of a composite system from the assembly of simple interfaces declaring the behavioural assumptions and guarantees of its constituents. DbC provides a method to conduct verification and development tasks in a modular and compositional manner. Under the DbC paradigm, developers define contracts with two parts: (1) the required behaviour that a constituent guarantees to implement, and (2) the assumptions the constituent makes about its environment. A composite system can thus be characterised by the contracts of its constituents, the composition of which fulfills the expected system contract. Each system constituent can be shown to fulfill its guaranteed behaviour under given assumptions, the violation of which could lead to an unpredictable behaviour of the entire system.

By allowing an arbitrary level of system decomposition, vertically (domain-specific) or horizontally (cross-domain), design complexity becomes tractable and controllable. Errors can be identified and corrected at the very early stages of system design; correctness and reliability can be guaranteed by simulation, verification and refinement; and developers can reuse existing contracts and components, further improving development efficiency. Expectedly, model- and contract-based design practices are much recommended by many standard bodies for safety-critical system development.

Nonetheless, guaranteeing the consistency of a system based on the models of its components poses significant challenges. To address it, ideally, one would need a uniform semantics domain capable of capturing all component models, at different abstraction levels, across heterogeneous domain concerns, into the same analysis, automated reasoning and verification framework.

Motivated by the Grand Unified Theory in physics, in the literature, Hoare and He's unifying theories of programming (UTP) [28] is the first proposal towards unifying different programming paradigms and models under a common relational calculus suitable for design and verification. In UTP, programs are interpreted as guarded designs consisting of a guard, pre- and post-condition [24, 28]. UTP defines operators and a refinement relation over guarded designs to form a lattice. Hence, in UTP, arbitrarily different programming constructs, paradigms, and models of computation can be logically interpreted and compared using the Galois connection induced by their lattice ordering.

Guaranteeing consistency among different models at different levels of abstraction is a grand challenge in control and software engineering. To address it for the design of hybrid systems, this paper follows the UTP paradigm by extending the classical UTP with higher-order quantification on continuous variables (functions over time) and differential relations to form a higher-order UTP (HUTP), that can be used to give uniform semantics of hybrid systems at different levels of abstraction, from higher-level abstract models like Simulink or Hybrid Communicating Sequential Processes (HCSP), down to the low-level models of computation of their implementation by generated source code in SystemC or ANSI-C, allowing the consistency among models of hybrid systems at different levels of abstraction/implementation to be proved using theorem proving techniques. Note that HUTP in this paper differs from the higher-order UTP defined in [28, 61, 62]. The latter focuses on higher-order programming, i.e., programs can be treated as higher-order variables in the paradigm. Higher-order quantifications over functions and predicates are also addressed in related works, such as [62]. However, it is still unclear whether the classical UTP can be extended to hybrid systems like the extension to higher-order programs.

Our work starts from [10], which extends UTP by allowing differential relations and higher-order quantification, establishing a theory of *higher-order UTP* (HUTP). Additionally, [10] defines HUTP syntactically, which still leaves the question open of equipping it with an adequate algebraic structure (e.g., complete lattices) to serve as a semantics foundation for CPS design.

During the same period of [10], to address the context of CPS design with UTP, Woodcock et al. introduce time-dependent trajectory variables into the alphabet and define *hybrid relations* [17], based on which the semantics of Modelica [42] with a mixture of differential algebraic equations and event preemption is given. To unify discrete and hybrid theories, they introduce continuous-time traces to the trace model and define a generalised theory of reactive processes [16] in terms of trace algebra. Subsequently in [15], they define reactive design contracts that provide the basis for modelling and verification of reactive systems in the presence of parallelism and communication, yet limited to discrete controllers. Based on the theory of reactive processes, [13] provides an alternative definition of *hybrid relations* to define a denotational semantics of hybrid programs [48]. However, the above bulk of works does not address some important features, including

- *Super-dense computation*. As argued in [38], *super-dense computation* provides a very appropriate abstract computation model for embedded systems, as computer is much faster than other physical devices and computation time of a computer is therefore negligible, although the temporal order of computations is still there. As [17] considers finite timed traces with right continuity, super-dense time computation violates right continuity at time points, where a sequence of discrete events happen, and therefore cannot be modelled, although [16] argues that the theory of finite timed traces is extensible to deal with infinite ones.
- *Parallelism with communication-based synchronisation* is not addressed yet, although widely used in CPS designs.
- *Declaration of local variables and channels*. Variables and channels are interpreted as functions over time, therefore declaration of local variables and channels have to resort to higher-order quantification. [14] uses symmetric lenses to deal with local variables, but only discrete variables are considered.

For the drawbacks of our previous work [10], we propose a new theory of HUTP and investigate the suitable algebraic structure for HUTP by taking all important features of hybrid systems into consideration, to enable its use as a semantics foundation for CPS design. Inspired by the Duration Calculus [66], we introduce real-time variables, denoted as functions over time. Derivatives of real variables are additionally allowed in predicates, to form differential relations. In order to model synchronous communication between processes, we introduce timed traces to record the communication history of processes. Noteworthy contributions of previous works [15, 16, 28] with related aim regarded traces as finite abstract sequences. By contrast, timed traces in this paper can be infinite. Therefore, non-terminating behaviours such as divergence and deadlock can be described explicitly. These definitions ground the elaboration of the concepts of hybrid processes and (normal) hybrid designs, based on which we define formal semantics for HCSP [22, 67] and Simulink [39] using HUTP, and justify the correctness of the translation from Simulink to HCSP in [69] as an application of HUTP. In addition, by abstracting hybrid processes and (normal) hybrid designs, we propose a skeleton of equipping HUTP with compositional assume-guarantee reasoning by means of contracts.

Our work is similar to [15] in some aspects, as both are extensions of traditional UTP [28]. However, they differ in essence, namely:

- Our HUTP theory separates the concerns of time, state and traces. Especially, we introduce continuous state variables and construct a trace model to address concurrency and communication for hybrid systems. In [15, 16], although continuous states are encapsulated into traces, parallelism based on traces is not made explicit. For example, the interleaving of traces carrying continuous states is confusing, as, in reality, physical evolution never interleaves with discrete models.

- Our approach can deal with infinite behaviour very well, in which therefore deadlock and divergence can be modelled formally. For example, the Zeno's paradox (Example 3.5) can be modelled as a normal non-terminating (and not necessarily unpredictable) behaviour. In contrast, it is unclear how to model the Zeno's paradox and other infinite behaviours using [15], as it seems that only finite traces are involved.
- Since our HUTP theory supports infinite behaviours, auxiliary variables *wait* and *wait'*, denoting reactivity in discrete models [15, 28], are interpreted explicitly with our approach.

In summary, our proposed HUTP is a conservative extension to the classical UTP of [28] and contributes:

- the separation of concerns in hybrid system design into time, state and trace;
- a timed trace model recording execution history and captures communication behaviours;
- real-time variables and their derivatives, which are functions over time, and differential relations over them that are very powerful to express all kinds of continuous dynamics;
- higher-order quantification for specifying locality;
- a calculus of hybrid processes and (normal) hybrid designs to model and analyse hybrid systems with (potentially) infinite behaviours; and
- the HUTP semantics of HCSP and Simulink.

Paper Organisation. The rest of the paper is organised as follows. Section 2 retrospects some preliminary concepts of the Unifying Theories of Programming. Section 3 defines a timed trace model which will play an important role in the HUTP theory. Section 4 introduces hybrid processes which combine time, states and traces to provide a uniform semantics for hybrid systems. To elaborate a sound theory, we extend hybrid processes to (normal) hybrid designs in Section 5. In Section 6, the HUTP is applied to verify the translation from Simulink diagrams to HCSP as an application of HUTP. Section 7 addresses the related work and Section 8 concludes this paper and discusses future works.

2 UNIFYING THEORIES OF PROGRAMMING

UTP [28] is an alphabetised refinement calculus unifying heterogeneous programming paradigms. An alphabetised relation consists of an alphabet $\alpha(P)$, containing its variables x and primes x' , and a relational predicate P referring to this vocabulary. The terms x and x' are called observable variables: x is observable at the start of execution and x' is observable at the end of execution. The behaviour of a program is encoded as a relation between the observable variables x and x' . In particular, assignment, sequential composition, conditional statement, non-deterministic choice, and recursion of imperative programs can be specified as alphabetised relations below, where $\mathbf{x}$ and $\mathbf{x}'$ are sequences or vectors of variables, $\mathbf{x} \setminus \{x\}$ ($\mathbf{x}' \setminus \{x'\}$) denotes excluding x (x') from $\mathbf{x}$ ($\mathbf{x}'$). To start with, the relation calculus comprises all operators of first-order logic.

$$\begin{aligned}
 x := e &\equiv x' = e \wedge \mathbf{x}' \setminus \{x'\} = \mathbf{x} \setminus \{x\} \\
 P \circledast Q &\equiv \exists \mathbf{x}_* \cdot P[\mathbf{x}_*/\mathbf{x}'] \wedge Q[\mathbf{x}_*/\mathbf{x}] \\
 P \triangleleft b \triangleright Q &\equiv (b \wedge P) \vee (\neg b \wedge Q) \\
 P \sqcap Q &\equiv P \vee Q \\
 P \sqcup Q &\equiv P \wedge Q
 \end{aligned}$$

Assignment $x := e$ is defined by observing the update x' of variable x once its value e is evaluated, leaving other variables in the alphabet $\mathbf{x}$ unchanged. Sequence $P \circledast Q$ is modelled by locally binding, through $\mathbf{x}_*$, the final state $\mathbf{x}'$ of P and the initial state $\mathbf{x}$ of Q both are instantiated to it. Note that $\circledast$ requires that $\alpha_{out}(P) = \alpha'_{in}(Q)$, where $\alpha_{out}(P)$ and $\alpha_{in}(Q)$ denote the sets of output and input variables in $\alpha(P)$ and $\alpha(Q)$, respectively, and $\alpha'_{in}(Q)$ is the primed version of $\alpha_{in}(Q)$. The conditional $P \triangleleft b \triangleright Q$ evaluates as P if b is true or as Q otherwise. $P \sqcap Q$ non-deterministically chooses P or Q , whereas $P \sqcup Q$ is a conjunction of P and Q .

Let P and Q be two predicates with the same alphabet, say $\{x, x'\}$. Then, Q is a *refinement* of P , denoted $P \sqsubseteq Q$, if $\forall x, x' \cdot Q \Rightarrow P$. In addition, $P \sqsubseteq Q$ iff $P \sqcap Q = P$ iff $P \sqcup Q = Q$. With respect to the refinement order $\sqsubseteq$, the least (μ) and greatest (ν) fixed points of a program function F can be defined as follows:

$$\begin{aligned}\mu F &\triangleq \bigsqcap \{X \mid F(X) \sqsubseteq X\} \\ \nu F &\triangleq \bigsqcup \{X \mid X \sqsubseteq F(X)\}\end{aligned}$$

Example 2.1. Given an alphabet $\{x, y, x', y'\}$, the semantics of the assignment $x := 1$ is different from $x' = 1$. The former is equivalent to $x' = 1 \wedge y' = y$, while the latter indicates that y and y' can be arbitrary, i.e., $x' = 1 \sqsubseteq x := 1$. They are equivalent if the alphabet contains only x and x' .

A UTP theory is a well-defined subset of alphabetised relations that satisfies certain conditions, called *healthiness conditions*. If a predicate is a fixed point of the healthiness condition, $P = \mathcal{H}(P)$, then it is said to be $\mathcal{H}$ -healthy. In other words, a healthiness condition $\mathcal{H}$ defines an invariant predicate set $\{X \mid \mathcal{H}(X) = X\}$, and is required to be idempotent ($\mathcal{H} \circ \mathcal{H} = \mathcal{H}$), which means that taking the medicine twice leaves you as healthy as taking it once (no overdoses). So, in UTP, the healthy predicates of a theory are the fixed points of idempotent functions. When $\mathcal{H}$ is monotonic with respect to the refinement order $\sqsubseteq$, then according to the Knaster-Tarski theorem [57], the UTP theory satisfying $\mathcal{H}$ forms a complete lattice and, additionally, recursion can be well specified. Multiple healthiness conditions $\mathcal{H}_0, \mathcal{H}_1, \dots, \mathcal{H}_n$ can also be composed as $\mathcal{H} \triangleq \mathcal{H}_0 \circ \mathcal{H}_1 \circ \dots \circ \mathcal{H}_n$, provided that $\mathcal{H}_i$ and $\mathcal{H}_j$ commute, for all $0 \leq i, j \leq n$. Healthiness conditions play an important role in UTP, and distinct healthiness conditions can be defined to specify properties of different programming paradigms. In Section 4, we use healthiness conditions to define the concept of hybrid processes.

Example 2.2. Given an alphabet $\{x, x', \text{term}, \text{term}'\}$, where the Boolean variable term' denote whether the program has terminated. If the preceding program is non-terminated ($\neg \text{term}$), the current program P should do nothing but keep the values of all variables unchanged, expressed by the following healthiness condition:

$$\mathcal{H}(P) \triangleq P \triangleleft \text{term} \triangleright (x' = x \wedge \text{term}' = \text{term})$$

The statement $x := 1$ is not $\mathcal{H}$ -healthy, but we can make it $\mathcal{H}$ -healthy by

$$\mathcal{H}(x := 1) = (x := 1) \triangleleft \text{term} \triangleright (x' = x \wedge \text{term}' = \text{term})$$

It is easy to check that the above program is indeed a fixed point of $X = \mathcal{H}(X)$. When term is **true**, i.e., the preceding program is terminated, the above program executes $x := 1$. Note that this assignment is equivalent to $x' = 1 \wedge \text{term}' = \text{term}$. If non-termination is expected, we can write $\mathcal{H}(x' = 1 \wedge \neg \text{term}')$. It is a left-zero of sequential composition, i.e.,

$$\mathcal{H}(x' = 1 \wedge \neg \text{term}') \circ P = \mathcal{H}(x' = 1 \wedge \neg \text{term}')$$

provided that P is any $\mathcal{H}$ -healthy predicate whose alphabet is $\{x, x', \text{term}, \text{term}'\}$.

Following the convention of UTP, in this paper, we will define a set of healthiness conditions to identify the features of hybrid systems. The operators for classical sequential programs based on first-order relational calculus, such as $\circ$, $\sqcap$ and $\sqcup$, can be easily lifted to the theory for hybrid systems proposed in this paper based on higher-order differential relational calculus, and we will prove that they still hold the desired properties like in classical UTP.

3 TIMED TRACE MODEL

In this section, we construct a timed trace model to capture the communication behaviour of processes. Traces play an important role in our HUTP theory which separates the concerns of time, state and trace. For processes in parallel, we assume that no shared variable is allowed between them and exchange of data between processes is

described solely by communications. The synchronous communication between concurrent processes is achieved using input and output data channels. The input action $ch?x$ receives a value along channel ch and assigns it to the variable x , and the output action $ch!e$ sends the value of the expression e along channel ch . A synchronous communication takes place immediately when both the sending and the receiving actions are enabled. If one party is not ready, the other has to wait.

3.1 Trace Blocks

Each timed trace is a sequence of communication, wait and internal blocks defined below:

- A pair $\langle ch*, d \rangle$ is a communication block, where $ch*$, a channel operation, indicates input ($ch?$), output ($ch!$) or synchronised communication (ch), and d is a value along channel ch . It represents that a communication along $ch*$ takes place instantly. For example, $\langle ch?, d \rangle$ describes the communication receiving the value d along the channel ch at current time.
- A pair $\langle \delta, RS \rangle$ is a wait block, where $0 < \delta \leq +\infty$ is the time period and RS is a set of waiting channel operations, called *ready set*. It means the process evolves during the period with the channel operations in RS ready for communication. The dual of RS is defined by

$$\overline{RS} = \{ch\bar{*} \mid ch* \in RS, * \in \{?, !\}\}$$

where $\bar{?} = !$ and $\bar{!} = ?$, and we assume that $RS \cap \overline{RS} = \emptyset$.

- The symbol τ is an internal block standing for an internal or invisible action such as a timeless discrete computation.

Example 3.1. For the communication of two HCSP processes: $ch?x \parallel ch!2$, since $ch?x$ and $ch!2$ are ready simultaneously at the beginning, the communication takes place instantly according to the semantics of HCSP [64]. The timed traces of $ch?x$ and $ch!2$ are $\langle ch?, 2 \rangle$ and $\langle ch!, 2 \rangle$, respectively.

Example 3.2. Consider the delay communication $ch?x \parallel (\text{wait } 1; ch!2)$. Since the right process waits for 1 time unit before $ch!2$ is ready, the left process has to wait for 1 time unit for synchronisation although it has been ready since the beginning. The communication takes place after 1 time unit. Therefore, the timed trace of the left is $\langle 1, \{ch?\} \rangle \wedge \langle ch?, 2 \rangle$ and the right is $\langle 1, \emptyset \rangle \wedge \langle ch!, 2 \rangle$.

Example 3.3. Consider an ordinary differential equation (ODE) with communication interruption:

$$x := 1; \langle \dot{x} = x \&\text{true} \rangle \triangleright \parallel (sensor!x \rightarrow actuator?x) \parallel \text{wait } 2; sensor?y; actuator!(y + 1)$$

The assignment $x := 1$ is a timeless discrete action denoting initialisation. The left HCSP process means the continuous evolution depicted by the ODE $\dot{x} = x$ is preempted as soon as the communication $sensor!x$ takes place, followed by $actuator?x$. The right process indicates that the ODE of the left process evolves for 2 time units before $sensor!x$ takes place when the value of x reaches $\exp(2)$. Therefore, the timed trace of the left process is

$$\tau \wedge \langle 2, \{sensor!\} \rangle \wedge \langle sensor!, \exp(2) \rangle \wedge \langle actuator?, \exp(2) + 1 \rangle$$

and the timed trace of the right process is

$$\langle 2, \emptyset \rangle \wedge \langle sensor?, \exp(2) \rangle \wedge \langle actuator!, \exp(2) + 1 \rangle$$

3.2 Timed Traces

In the previous works such as [15, 16, 28], traces are finite sequences. In our work, we consider infinite timed traces. A timed trace is a finite or infinite sequence of communication, wait, and internal blocks which records the execution history of a process.

Let Σ be the set of all channels and $\Sigma?$ and $\Sigma!$ the respective sets of receiving and sending events along channels in Σ . The sets of communication and wait blocks are respectively

$$\begin{aligned}\mathbb{C} &\triangleq (\Sigma? \cup \Sigma! \cup \Sigma) \times \mathbb{D} \\ \mathbb{W} &\triangleq \{ \langle \delta, RS \rangle \mid 0 < \delta \leq +\infty, RS \subseteq (\Sigma? \cup \Sigma!), RS \cap \overline{RS} = \emptyset \}\end{aligned}$$

where $\mathbb{D}$ is some domain.

Definition 3.4 (Timed Trace Set). Let $\mathbb{TT} = (\mathbb{C} \cup \mathbb{W} \cup \{\tau\})^* \cup (\mathbb{C} \cup \mathbb{W} \cup \{\tau\})^\omega$ be the timed trace set, ϵ the empty trace and $\wedge$ the concatenation between trace blocks. Then, the algebra $(\mathbb{TT}, \epsilon, \wedge)$ satisfies the following axioms, where $\wedge$ takes precedence over $=$.

Associativity $\forall tt_0, tt_1, tt_2 \in \mathbb{TT} \cdot (tt_0 \wedge tt_1) \wedge tt_2 = tt_0 \wedge (tt_1 \wedge tt_2)$

Unit $\forall tt \in \mathbb{TT} \cdot \epsilon \wedge tt = tt \wedge \epsilon = tt$

Left Zero $\forall tt \in \mathbb{TT} \cdot \langle +\infty, RS \rangle \wedge tt = \langle +\infty, RS \rangle$

Additivity $\langle \delta_0, RS \rangle \wedge \langle \delta_1, RS \rangle = \langle \delta_0 + \delta_1, RS \rangle$

The axiom **Left Zero** reflects a simplifying function $lz : \mathbb{TT} \rightarrow \mathbb{TT}$ that removes the redundant tail from each trace. For example, $lz(\langle +\infty, RS \rangle \wedge \langle 2, RS \rangle) = \langle +\infty, RS \rangle$. The axiom **Additivity** reflects a simplifying function $add : \mathbb{TT} \rightarrow \mathbb{TT}$ that merges all the consecutive wait blocks with the same ready set in each trace. For example, $add(\langle 1, RS \rangle \wedge \langle 2, RS \rangle) = \langle 3, RS \rangle$. Then, the composite function $add \circ lz : \mathbb{TT} \rightarrow \mathbb{TT}$ returns the simplest form of each trace. A trace tt is called *canonical normal form* if $|tt| = |add \circ lz(tt)|$, where $|\cdot|$ returns the length of a trace. Thus, we say tt is canonical if it cannot be simplified. In what follows, all traces are referred to canonical normal form if not otherwise stated.

We say tt_0 is *equivalent* to tt_1 , denoted by $tt_0 = tt_1$, if each finite prefix of tt_0 is also a prefix of tt_1 , and vice versa. We say $tt_0 < tt_1$ if tt_0 is finite and there exists a trace tt such that $tt_0 \wedge tt = tt_1$, and in addition we let $tt_1 - tt_0 = tt$. Therefore, $tt_0 < tt_1$ means tt_0 is a finite prefix of tt_1 , and $tt_1 - tt_0$ denotes the suffix after deleting tt_0 .

Let $\lim(tt)$ be the sum of the time periods of all the wait blocks in tt . If no wait block appears in tt , then we let $\lim(tt) = 0$. A trace tt is *non-terminated* if $\lim(tt) = +\infty \vee |tt| = +\infty$ and *divergent* if $\lim(tt) < +\infty \wedge |tt| = +\infty$. Let $tt(n)$ be the n -th (start from 0) block of tt . If $|tt| < +\infty$, for any $n \geq |tt|$ we let $tt(n) = tt(|tt| - 1)$, i.e., the last block of tt . We say tt is *deadlocked* if

$$\lim(tt) = +\infty \wedge \exists N \in \mathbb{N} \cdot \forall n \geq N \cdot tt(n) = \langle \delta, RS \rangle \wedge RS \neq \emptyset \quad (1)$$

where $\mathbb{N}$ is the set of natural numbers. (1) says that, from a certain time point, the process will be waiting for communication forever. Clearly, a deadlock is non-terminated because $\lim(tt) = +\infty$. This non-termination occurs because some channel operations are never matched. From the view of failures-divergence model [54], the standard denotational semantics for CSP, a ready set is exactly the dual of a refusal set, which contains the events (channel operations) that are not refused. Thus, (1) means that, from a certain time point, the dual channel operations in tt 's ready set are always refused by the environment, which leads to a deadlock. Especially, if $\lim(tt) = +\infty \wedge |tt| < +\infty$, the last block of tt must be a wait block $\langle +\infty, RS \rangle$, in which case tt will be deadlocked if $RS \neq \emptyset$.

Example 3.5 (Zeno's Paradox). Consider a ball falling from an initial height h with initial velocity 0. The ball falls freely with its dynamics given by the ODE $\dot{v} = -g$, where v is the velocity and g is the gravitational acceleration. After $\sqrt{2h/g}$ time units, it hits the ground ($h = 0$) with $v = -\sqrt{2hg}$. Then, there is a discontinuous update in its velocity. This discrete change can be modelled as a mode switch with the update given by $v := -c \cdot v$. This assumes that the collision is inelastic, and the velocity decreases by a factor c , for some appropriate constant $0 < c < 1$. Next, the ball bounces back with velocity $c\sqrt{2hg}$ and reaches the peak ($v = 0$) after $c\sqrt{2h/g}$ time units.

The ball then falls again and this process repeats infinitely. In summary, the timed trace of the bouncing ball is $\langle \sqrt{2h/g}, 0 \rangle \frown \tau \frown \langle 2c\sqrt{2h/g}, 0 \rangle \frown \tau \frown \dots \frown \langle 2c^{n+1}\sqrt{2h/g}, 0 \rangle \frown \tau \frown \dots$ where the internal blocks (τ s) denote the mode switch of the velocity ($v := -c \cdot v$). The limit of this timed trace is $\frac{1+c}{1-c} \cdot \sqrt{2h/g}$ but the length of it is infinite, i.e., a divergence.

3.3 Parallel Composition

For each pair of processes in parallel, we assume that they communicate over a set of channels I . Concretely, they can only exchange messages or data synchronously via the channels in I , and they communicate with the environment via channels outside I . In [54], this parallel is called *generalised* or *interface* parallel, and in [64], it is called *alphabetised* parallel and the channels in I are called *common* channels. Therefore, in this paper, the set I is called a *common channel set* between processes (traces). For simplicity, we also assume:

- (1) Each input or output channel can only be owned by one sequential process, i.e., a channel operation like $ch?$ cannot occur in two different sequential processes in parallel.
- (2) Any pair of symmetric input and output channels cannot be owned by one sequential process, i.e., channel operations like $ch?$ and $ch!$ cannot occur in one sequential process.

Such restrictions are not essential, as the general communication model can always be reduced to this simplified model, please refer to [27] for the details.

Let $tt_0 \parallel_I tt_1 \rightsquigarrow tt$ denote that the parallel composition of two timed traces tt_0 and tt_1 over a common channel set I derives a new timed trace tt . Since infinite timed traces are included, we give the following coinductive rules to define the parallel composition of timed traces, where the binding priority among operators is that $\frown$ prior to $\parallel_I$ prior to $\rightsquigarrow$, and $\uplus$ is a disjoint union.

$$\begin{array}{c}
\frac{}{\epsilon \parallel_I \epsilon \rightsquigarrow \epsilon} [\text{Empty}] \quad \frac{\overline{RS_0} \cap RS_1 = \emptyset \quad tt_0 \parallel_I tt_1 \rightsquigarrow tt}{\langle \delta, RS_0 \rangle \frown tt_0 \parallel_I \langle \delta, RS_1 \rangle \frown tt_1 \rightsquigarrow \langle \delta, RS_0 \uplus RS_1 \rangle \frown tt} [\text{SynWait}] \\
\\
\frac{ch \in I \quad tt_0 \parallel_I tt_1 \rightsquigarrow tt}{\langle ch?, d \rangle \frown tt_0 \parallel_I \langle ch!, d \rangle \frown tt_1 \rightsquigarrow \langle ch, d \rangle \frown tt} [\text{SynIO}] \\
\\
\frac{ch \notin I \quad tt_0 \parallel_I tt_1 \rightsquigarrow tt}{\langle ch*, d \rangle \frown tt_0 \parallel_I tt_1 \rightsquigarrow \langle ch*, d \rangle \frown tt} [\text{NoSynIO}] \quad \frac{tt_0 \parallel_I tt_1 \rightsquigarrow tt}{\tau \frown tt_0 \parallel_I tt_1 \rightsquigarrow \tau \frown tt} [\tau\text{-Act}]
\end{array}$$

Rule [Empty] says that the parallel composition of two empty traces trivially reduces to an empty trace. Rule [SynWait] states that when both sides are waiting for a communication, they can be synchronised for the same length of period if their ready sets are unmatched, i.e., $\overline{RS_0} \cap RS_1 = \emptyset$, because $\overline{RS_0} \cap RS_1 \neq \emptyset$ indicates that the two sides are waiting for a communication with some channel operations matched, that should be taken immediately, and no time synchronization is allowed. Rule [SynIO] synchronises the input and output communications along some channel $ch \in I$. The synchronization will never be applied to channels outside I or internal actions τ as stated by Rules [NoSynIO] and [τ -Act]. In addition, there are symmetric versions of these rules, which means that $\parallel_I$ is commutative.

PROPERTY 1 (ASSOCIATIVITY). *Let I_0, I_1 and I_2 be the respective common channel sets between timed traces tt_0 and tt_1 , tt_1 and tt_2 , and tt_2 and tt_0 , then $(tt_0 \parallel_{I_0} tt_1) \parallel_{I_1 \uplus I_2} tt_2 = tt_0 \parallel_{I_0 \uplus I_2} (tt_1 \parallel_{I_1} tt_2)$.*

Example 3.6. The timed trace $tt_0 = \langle 2, \{ch?\} \rangle \frown \langle ch?, 5 \rangle$ indicates a delay for 2 time units, during which the channel operation $ch?$ is waiting, followed by receiving the value 5 along channel ch . The trace $tt_1 = \langle 2, \emptyset \rangle \frown \langle ch!, 5 \rangle$ indicates a delay for 2 time units, during which no channel operations are waiting, followed by

sending the value 5 along channel ch . The parallel composition of tt_0 and tt_1 over the channel ch is

$$tt_0 \parallel_{\{ch\}} tt_1 \rightsquigarrow \langle 2, \{ch?\} \rangle \frown \langle ch, 5 \rangle$$

by [SynWait], [SynIO] and [Empty]. In fact, recording $ch?$ in the ready set of the composed traces is not necessary, as ch is a channel that only connects tt_0 and tt_1 , making it impossible for the environment to synchronise with it via ch . However, on the other hand, recording $ch?$ has no side effect, and just indicates that the process itself was waiting for the channel operation $ch!$ during the period, no matter whether ch is a common channel between tt_0 and tt_1 or not. This notion is in accordance to the semantics of HCSP (please refer to [64] for the details).

Example 3.7. The timed trace $tt_0 = \langle dh?, 3 \rangle \frown \langle 2, \emptyset \rangle$ indicates receiving the value 3 along channel dh at the initial time, followed by a delay of 2 time units during which no channel operations are waiting. Let $tt_1 = \langle 1, \emptyset \rangle \frown \langle 1, \{ch?\} \rangle$. Then, we have

$$tt_0 \parallel_{\{ch\}} tt_1 \rightsquigarrow \langle dh?, 3 \rangle \frown \langle 1, \emptyset \rangle \frown \langle 1, \{ch?\} \rangle$$

by [NoSynIO], [SynWait], [Empty] and **Additivity** of wait blocks (see Definition 3.4). Note that if tt_0 and tt_1 also communicate via dh , then no rules can be applied for $tt_0 \parallel_{\{ch, dh\}} tt_1$ because $dh?$ cannot be synchronised, i.e., they are uncomposable.

Example 3.8. Let $tt_0 = \langle ch?, 3 \rangle \frown \langle dh?, 4 \rangle$ and $tt_1 = \langle ch!, 3 \rangle \frown \langle dh!, 5 \rangle$, then no rules can be applied for $tt_0 \parallel_{\{ch, dh\}} tt_1$ as the values along channel dh are different although the heads of tt_0 and tt_1 are matched by [SynIO], i.e., tt_0 and tt_1 are uncomposable.

Example 3.9. Let $tt_0 = \langle 1, \{ch?\} \rangle$ and $tt_1 = \langle 1, \{ch!\} \rangle$, then no rules can be applied for $tt_0 \parallel_{\{ch\}} tt_1$, as it is impossible that the symmetric channel operations $ch?$ and $ch!$ are waiting simultaneously, indicating that tt_0 and tt_1 are uncomposable.

Example 3.10. Consider two infinite timed traces

$$\begin{aligned} tt_0 &= \langle ch?, 0 \rangle \frown \langle ch?, 1 \rangle \frown \dots \frown \langle ch?, n \rangle \frown \dots \\ tt_1 &= \tau \frown \langle ch!, 0 \rangle \frown \tau \frown \langle ch!, 1 \rangle \frown \dots \frown \tau \frown \langle ch!, n \rangle \frown \dots \end{aligned}$$

The timed trace tt_0 indicates that the process is receiving values along channel ch endlessly at current time. The timed trace tt_1 indicates the infinite sending events at current time and before each sending event there is an internal action. Their parallel composition over ch is also infinite:

$$tt_0 \parallel_{\{ch\}} tt_1 \rightsquigarrow \tau \frown \langle ch, 0 \rangle \frown \tau \frown \langle ch, 1 \rangle \frown \dots \frown \tau \frown \langle ch, n \rangle \frown \dots$$

by $[\tau\text{-Act}]$ and [SynIO]. It is a divergence as the time never progresses.

Example 3.11. Let $tt_0 = \tau \frown \langle 1, \{ch?\} \rangle \frown \langle 2, \emptyset \rangle$ and $tt_1 = \langle 2, \emptyset \rangle \frown \tau \frown \langle 1, \{ch!\} \rangle$. Then, their parallel composition via channel ch is

$$tt_0 \parallel_{\{ch\}} tt_1 \rightsquigarrow \tau \frown \langle 1, \{ch?\} \rangle \frown \langle 1, \emptyset \rangle \frown \tau \frown \langle 1, \{ch!\} \rangle$$

by $[\tau\text{-Act}]$, [SynWait] and **Additivity** of wait blocks (see Definition 3.4). Notice that in this example, the symmetric channel operations $ch?$ and $ch!$ appear in one timed trace. Similar to Example 3.6, ch is a common channel between tt_0 and tt_1 and therefore it does not connect to the environment. $ch?$ and $ch!$ appearing in one timed trace is the result from parallel composition, which indicates this trace denotes a parallel process, not a sequential one. Actually, we can construct the following parallel HCSP processes:

$$\begin{aligned} P &\hat{=} x := 0; \langle \dot{x} = 1 \& x < 1 \rangle \triangleright \llbracket (ch?x \rightarrow \mathbf{skip}); \mathbf{wait}(2) \rrbracket \\ Q &\hat{=} \mathbf{wait}(2); y := 0; \langle \dot{y} = 1 \& y < 1 \rangle \triangleright \llbracket (ch!y \rightarrow \mathbf{skip}) \rrbracket \end{aligned}$$

The timed trace of $P \parallel Q$ is exactly $\tau \frown \langle 1, \{ch?\} \rangle \frown \langle 1, \emptyset \rangle \frown \tau \frown \langle 1, \{ch!\} \rangle$ as shown above.

4 HYBRID PROCESSES

Following Hehner’s philosophy “programs as predicates” [25], the first-order relational calculus of UTP [28] can be used to model programs specified with possibly different or heterogeneous programming styles. In this section, we introduce continuous dynamics into rational predicates to model hybrid systems, and use the term *hybrid processes* to denote such extended predicates. Hybrid processes serve as the basis of the HUTP theory in this paper.

To use UTP for modelling hybrid systems’ behaviours, one first needs to extend it with an explicit notion of time. This can be done by introducing two observational variables $ti, ti' \in \mathbb{R}_{\geq 0} \cup \{+\infty\}$ to specify the start- and end-time of the observed behaviour. Let $tr \in add \circ lz(\mathbb{T}\mathbb{T})$, where $add \circ lz(\mathbb{T}\mathbb{T})$ denotes the set of canonical normal traces, represent the timed trace before the process is started and $tr' \in add \circ lz(\mathbb{T}\mathbb{T})$ stand for timed trace up to the moment of observation.

In a real-time setting, process state variables are interpreted as functions over time. To specify its real-time value, we use $\underline{s}$ to represent it. In a word, we have three versions for each state variable s :

- the version $s \in \mathbb{D}$ stands for its initial value in the domain $\mathbb{D}$, i.e., the input state variable, where $\mathbb{D}$ could be a Banach space;
- the primed version $s' \in \mathbb{D}$ stands for the final value, i.e., the output state variable; and
- the real time version $\underline{s} : [ti, ti'] \rightarrow \mathbb{D}$ stands for its dynamic trajectory from the start time ti to the end time ti' , and $\dot{\underline{s}} : (ti, ti') \rightarrow \mathbb{D}$ is a partial function denoting the derivative of $\underline{s}$.

We use the boldface symbols $\mathbf{s}$, $\mathbf{s}'$, $\underline{\mathbf{s}}$ and $\dot{\underline{\mathbf{s}}}$ to denote respective vectors of input, output, real-time state variables and the derivatives. The alphabet that our theory depends on is $\{ti, ti', tr, tr', \mathbf{s}, \underline{\mathbf{s}}, \dot{\underline{\mathbf{s}}}, \mathbf{s}'\}$ by default. Therefore, first-order predicate $P(\mathbf{x}, \mathbf{x}')$ introduced in Section 2 can be extended to high-order differential relation $P(ti, ti', tr, tr', \mathbf{s}, \underline{\mathbf{s}}, \dot{\underline{\mathbf{s}}}, \mathbf{s}')$. By introducing some healthiness conditions for high-order differential relations, we can then define hybrid processes.

4.1 Healthiness Conditions

As stated in Section 2, healthiness conditions play an important role in UTP. Therefore, we introduce the following healthiness conditions to identify the features of hybrid processes:

- Time must be irreversible, and trace should be monotonically increasing because processes are not permitted to undo past events, i.e.,

$$\mathcal{H}_0(X) = X \wedge ti \leq ti' \wedge tr \leq tr'$$

This healthiness condition is a variant of **R1** of reactive processes in [28] and [15]. Here, however, **R1** does not deal with time. Similar variants include **HC1** in [23] and **HCT1** in [17], but they do not involve the concept of traces.

- Trace should keep consistent with time:

$$\mathcal{H}_1(X) = X \wedge ti = \lim(tr) \wedge ti' = \lim(tr')$$

- If the preceding process does not terminate, i.e., $ti = +\infty \vee |tr| = +\infty$, the current process should do nothing but keep the time and trace observations unchanged, i.e.,

$$\mathcal{H}_2(X) = (ti = ti' \wedge tr = tr') \triangleleft (ti = +\infty \vee |tr| = +\infty) \triangleright X$$

where $P \triangleleft b \triangleright Q \triangleq (b \wedge P) \vee (\neg b \wedge Q)$. This healthiness condition is similar to **R3** in [28] and **R3_h** in [15], except that we specify the auxiliary variable *wait* explicitly as $ti = +\infty \vee |tr| = +\infty$. Hence, unlike in classical UTP, variables *wait* and *wait'* are not used in our HUTP.

- If the current process does not terminate, i.e., $ti' = +\infty \vee |tr'| = +\infty$, the values of the output state variables are unobservable, i.e.,

$$\mathcal{H}_3(X) = (\exists s' \cdot X) \triangleleft (ti' = +\infty \vee |tr'| = +\infty) \triangleright X$$

As stated above, $ti' = +\infty \vee |tr'| = +\infty$ specify variable *wait'* in the traditional definition of reactive processes of the UTP, where *wait'* being **true** denotes the observation that all primed variables stand for intermediate observations, not final ones. However, in our work, we hide the primed state variables by an existential quantifier when $ti' = +\infty \vee |tr'| = +\infty$, since observing the state values at time infinity makes no sense. The idea of $\mathcal{H}_3$ is similar to **R4** for a healthy process in [9], which expresses the intuition that “program variable state is not visible while waiting for external events”.

- If the process evolves for a period of time, i.e., $ti < ti'$, the real-time value $\underline{s}$ should keep right-continuous and semi-differentiable, i.e.,

$$\mathcal{H}_4(X) = X \wedge RC \wedge SD$$

where

$$\begin{aligned} RC &\hat{=} \forall i \cdot \forall t \in [ti, ti') \cdot \underline{s}_i(t) = \underline{s}_i(t^+) \\ SD &\hat{=} \forall i \cdot \forall t \in (ti, ti') \cdot \exists d_0, d_1 \cdot \dot{\underline{s}}_i(t^-) = d_0 \wedge \dot{\underline{s}}_i(t^+) = d_1 \end{aligned}$$

denote the right-continuity and semi-differentiability, respectively, and $\underline{s}_i$ ($\underline{s}_i$ and $\underline{s}'_i$) is the i -th variable in $\underline{s}$ ($\underline{s}$ and $\underline{s}'$). $\mathcal{H}_4$ rules out some ill behaviours, such as the Dirichlet function (returning 1 if t is a rational number and 0 otherwise) and the Weierstrass function (continuous everywhere but differentiable nowhere).

Definition 4.1 (Hybrid Process). A hybrid process HP is a fixed point of $X = \mathcal{H}_{HP}(X)$, where

$$\mathcal{H}_{HP} \hat{=} \mathcal{H}_0 \circ \mathcal{H}_1 \circ \mathcal{H}_2 \circ \mathcal{H}_3 \circ \mathcal{H}_4$$

and the alphabet of HP is $\alpha(HP) \hat{=} \{ti, ti', tr, tr', \underline{s}, \underline{s}', \dot{\underline{s}}, \dot{\underline{s}}'\}$.

Since *wait* and *wait'*, used to denote reactivity in discrete models [15, 28], are interpreted respectively as $ti = +\infty \vee |tr| = +\infty$ and $ti' = +\infty \vee |tr'| = +\infty$ in $\mathcal{H}_2$ and $\mathcal{H}_3$, the hybrid processes defined by Definition 4.1 are essentially reactive.

THEOREM 4.2 (IDEMPOTENCE AND MONOTONICITY). $\mathcal{H}_{HP}$ is idempotent and monotonic.

4.2 Operations

4.2.1 Meet and Join. Let HP_0 and HP_1 be two hybrid processes. Non-determinism is simply modelled by logical disjunction:

$$HP_0 \sqcap HP_1 \hat{=} HP_0 \vee HP_1$$

It stands for a process that internally chooses to execute either HP_0 or HP_1 , regardless of the environment. The dual of non-determinism choice is conjunction:

$$HP_0 \sqcup HP_1 \hat{=} HP_0 \wedge HP_1$$

The alphabets are expanded by $\sqcap$ and $\sqcup$, i.e., $\alpha(HP_0 \sqcap HP_1) = \alpha(HP_0 \sqcup HP_1) = \alpha(HP_0) \cup \alpha(HP_1)$. In the literature, $\sqcap$ is also called *demonic choice* or *meet*, while $\sqcup$ is called *angelic choice* or *join*. Generally, $\sqcap$ means $\wedge$ and $\sqcup$ means $\vee$, but in UTP [15, 28], they always denote $\vee$ and $\wedge$, respectively. Thus, in this paper, we will follow this notational convention.

PROPERTY 2. If HP_0 and HP_1 are $\mathcal{H}_{HP}$ -healthy, then so are $HP_0 \sqcap HP_1$ and $HP_0 \sqcup HP_1$.

4.2.2 Sequential Composition. The sequential composition in traditional UTP can be adapted to hybrid processes:

$$HP_0 \circ HP_1 \hat{=} \exists ti_*, s_*, tr_* \cdot HP_0[ti_*, s_*, tr_*/ti', s', tr'] \wedge HP_1[ti_*, s_*, tr_*/ti, s, tr]$$

provided that $\alpha_{out}(HP_0) = \alpha'_{in}(HP_1)$, where $\alpha_{out}(HP_0)$ and $\alpha_{in}(HP_1)$ denote the sets of output and input variables (containing time and trace variables) in $\alpha(HP_0)$ and $\alpha(HP_1)$, respectively, and $\alpha'_{in}(HP_1)$ is the primed version by priming all the variables in $\alpha_{in}(HP_1)$. If $\alpha_{out}(HP_0) \neq \alpha'_{in}(HP_1)$, then we can extend the alphabets by $\alpha_{out}^+(HP_0) = \alpha_{in}^+(HP_1) \hat{=} \alpha_{out}(HP_0) \cup \alpha'_{in}(HP_1)$ to ensure the well-definedness of sequential composition.

PROPERTY 3. If HP_0 and HP_1 are $\mathcal{H}_{HP}$ -healthy, then so is $HP_0 \circ HP_1$.

With the definition of sequential composition, we can prove the following property for the healthiness condition $\mathcal{H}_3$ defined before.

PROPERTY 4. The healthiness condition $\mathcal{H}_3$ is equivalent to

$$X = X \circ \text{skip}$$

where $\text{skip} \hat{=} \mathcal{H}_2(ti = ti' \wedge tr = tr' \wedge s = s')$.

4.2.3 Complement. The complement of a hybrid process HP is interpreted by

$$\neg_l HP \hat{=} \mathcal{H}_{HP}(\neg HP)$$

Note that $\neg HP$ is not $\mathcal{H}_{HP}$ -healthy, as it may violate constraints such as $tr \leq tr'$. Therefore, we enforce healthiness by $\mathcal{H}_{HP}$, resulting in $\neg_l HP$. Intuitively, $\neg_l HP$ can be seen as the complement of HP in the context of hybrid processes, i.e., we can prove that $\neg_l HP \sqcap HP = \perp_{HP}$ and $\neg_l HP \sqcup HP = \top_{HP}$. Then we define implication by

$$HP_0 \Rightarrow_l HP_1 \hat{=} \neg_l HP_0 \vee HP_1$$

These logic operators are similar to $\neg_r$ and $\Rightarrow_r$, used in [15].

4.2.4 Quantification. To support declaration, abstraction and instantiation of variables, as in [28], we introduce quantifications over three versions s , s' and $\dot{s}$ of observable state variable s in a hybrid process HP . Existential quantifications of first-order variables s and s' are as usual:

$$\begin{aligned} \exists s \cdot HP &\hat{=} \bigvee \{HP[d/s] \mid d \in \mathbb{D}\} \\ \exists s' \cdot HP &\hat{=} \bigvee \{HP[d/s'] \mid d \in \mathbb{D}\} \end{aligned}$$

The quantification of real-time variable $\dot{s}$ is of higher-order and it will meanwhile quantifies the derivative of $\dot{s}$, as follows:

$$\exists \dot{s} \cdot HP \hat{=} \bigvee \left\{ HP[f, g/\dot{s}, \dot{s}] \mid \begin{array}{l} f : [ti, ti'] \rightarrow \mathbb{D} \quad g : (ti, ti') \rightarrow \mathbb{D} \\ \forall t \in (ti, ti') \cdot \dot{f}(t^-) = g(t^-) \wedge \dot{f}(t^+) = g(t^+) \end{array} \right\}$$

Existential quantification removes the quantified variables from the alphabet of a process, i.e., $\alpha(\exists s \cdot HP) = \alpha(HP) \setminus \{s\}$, $\alpha(\exists s' \cdot HP) = \alpha(HP) \setminus \{s'\}$ and $\alpha(\exists \dot{s} \cdot HP) = \alpha(HP) \setminus \{\dot{s}, \dot{s}'\}$. Dually, universal quantifications of s , s' and $\dot{s}$ can be defined by

$$\begin{aligned} \forall s \cdot HP &\hat{=} \neg_l(\exists s \cdot \neg_l HP) \\ \forall s' \cdot HP &\hat{=} \neg_l(\exists s' \cdot \neg_l HP) \\ \forall \dot{s} \cdot HP &\hat{=} \neg_l(\exists \dot{s} \cdot \neg_l HP) \end{aligned}$$

Since real-time variables are functions over time, such quantifications are of higher-order (functional). It can also be proven that quantifiers $\exists$ and $\forall$ are $\mathcal{H}_{HP}$ -preserving. In addition, higher-order $\exists$ and $\forall$ enjoy the following properties:

PROPERTY 5 (QUANTIFICATION).

$$\begin{aligned}
\exists \tilde{x} \cdot (\text{HP}_0 \circ \text{HP}_1) &= (\exists \tilde{x} \cdot \text{HP}_0) \circ (\exists \tilde{x} \cdot \text{HP}_1) \\
\forall \tilde{x} \cdot (\text{HP}_0 \circ \text{HP}_1) &\sqsupseteq (\forall \tilde{x} \cdot \text{HP}_0) \circ (\forall \tilde{x} \cdot \text{HP}_1) \\
\exists \tilde{x} \cdot (\text{HP}_0 \sqcap \text{HP}_1) &= (\exists \tilde{x} \cdot \text{HP}_0) \sqcap (\exists \tilde{x} \cdot \text{HP}_1) \\
\forall \tilde{x} \cdot (\text{HP}_0 \sqcap \text{HP}_1) &\sqsupseteq (\forall \tilde{x} \cdot \text{HP}_0) \sqcap (\forall \tilde{x} \cdot \text{HP}_1) \\
\exists \tilde{x} \cdot (\text{HP}_0 \sqcup \text{HP}_1) &\sqsubseteq (\exists \tilde{x} \cdot \text{HP}_0) \sqcup (\exists \tilde{x} \cdot \text{HP}_1) \\
\forall \tilde{x} \cdot (\text{HP}_0 \sqcup \text{HP}_1) &= (\forall \tilde{x} \cdot \text{HP}_0) \sqcup (\forall \tilde{x} \cdot \text{HP}_1) \\
\neg_i \exists \tilde{x} \cdot \text{HP} &= \forall \tilde{x} \cdot \neg_i \text{HP} \\
\neg_i \forall \tilde{x} \cdot \text{HP} &= \exists \tilde{x} \cdot \neg_i \text{HP}
\end{aligned}$$

Note that in the above the refinement order $\sqsubseteq$ for predicates introduced in Section 2 can be lifted to hybrid processes naturally. With the aid of higher-order quantifiers, the lexical locality of a variable s in a hybrid process HP can be specified by

$$\text{local } s \cdot \text{HP} \hat{=} \exists s, \tilde{s}, s' \cdot \text{HP}$$

Example 4.3. Consider a hybrid process with independent state variables x and y :

$$\text{HP}_0 \hat{=} \mathcal{H}_{\text{HP}} \left(\begin{aligned} &ti < ti' \wedge tr' - tr = \langle ti' - ti, \emptyset \rangle \wedge \\ &x = \tilde{x}(ti) \wedge x' = \tilde{x}(ti'^-) \wedge y = \tilde{y}(ti) \wedge y' = \tilde{y}(ti'^-) \\ &\wedge \forall t \in (ti, ti') \cdot \dot{\tilde{x}}(t) = 1 \wedge \dot{\tilde{y}}(t) = 2 \end{aligned} \right)$$

with $\alpha(\text{HP}_0) = \{ti, ti', tr, tr', x, \tilde{x}, \dot{\tilde{x}}, x', y, \tilde{y}, \dot{\tilde{y}}, y'\}$. A literal abstraction of variable x from HP_0 is

$$\text{local } x \cdot \text{HP}_0 = \mathcal{H}_{\text{HP}} \left(\begin{aligned} &ti < ti' \wedge tr' - tr = \langle ti' - ti, \emptyset \rangle \wedge \\ &y = \tilde{y}(ti) \wedge y' = \tilde{y}(ti'^-) \wedge \forall t \in (ti, ti') \cdot \dot{\tilde{y}}(t) = 2 \end{aligned} \right)$$

with the alphabet $\{ti, ti', tr, tr', y, \tilde{y}, \dot{\tilde{y}}, y'\}$.

Example 4.4. Consider another interesting example, let

$$\text{HP}_1 \hat{=} \mathcal{H}_{\text{HP}} \left(\begin{aligned} &ti < ti' \wedge tr' - tr = \langle ti' - ti, \emptyset \rangle \wedge \\ &x = \tilde{x}(ti) \wedge x' = \tilde{x}(ti'^-) \wedge y = \tilde{y}(ti) \wedge y' = \tilde{y}(ti'^-) \\ &\wedge \forall t \in (ti, ti') \cdot \dot{\tilde{x}}(t) + \dot{\tilde{y}}(t) = 0 \end{aligned} \right)$$

The abstraction of x from HP_1 is

$$\text{local } x \cdot \text{HP}_1 = \mathcal{H}_{\text{HP}} \left(ti < ti' \wedge y = \tilde{y}(ti) \wedge y' = \tilde{y}(ti'^-) \wedge tr' - tr = \langle ti' - ti, \emptyset \rangle \right)$$

with the alphabet $\{ti, ti', tr, tr', y, \tilde{y}, \dot{\tilde{y}}, y'\}$. Note that when x is localised, the ODE $\dot{\tilde{x}}(t) + \dot{\tilde{y}}(t) = 0$ is also abstracted away.

4.2.5 Parallel Composition. The parallel composition of two hybrid processes is written as

$$\text{HP}_0 \parallel_M \text{HP}_1$$

which uses the parallel-by-merge scheme developed in UTP [15, 28]. We assume that the state variables of concurrent processes are disjoint, i.e., $\alpha(\text{HP}_0) \cap \alpha(\text{HP}_1) = \{ti, ti', tr, tr'\}$. Let the notation HP_X make an X -version of HP by adding the shared outputted variables (ti' and tr') in HP with the X -subscript, i.e.,

$$\text{HP}_X \hat{=} \text{HP} \circ (ti = ti'_X \wedge tr = tr'_X \wedge s = s') = \text{HP}[ti'_X, tr'_X / ti', tr']$$

Let s_0 and s_1 be state variables ($s_0 \cap s_1 = \emptyset$) owned by hybrid processes HP_0 and HP_1 , respectively. Let $s \hat{=} s_0 \uplus s_1$. Then, the parallel composition of HP_0 and HP_1 can be defined as follows:

$$\text{HP}_0 \parallel_M \text{HP}_1 \hat{=} \mathcal{H}_{\text{HP}} ((\text{HP}_{0,X} \wedge \text{HP}_{1,Y} \wedge tr = tr') \circ M)$$

where M is a merge predicate with $\alpha(M) \triangleq \{ti_X, ti_Y, ti', tr_X, tr_Y, tr, tr', s, \underline{s}, \underline{s}', s'\}$. Therefore, the alphabet of the parallel composition is $\alpha(HP_0 \parallel_M HP_1) \triangleq \{ti, ti', tr, tr', s, \underline{s}, \underline{s}', s'\}$. The parallel-by-merge scheme is illustrated pictorially in Fig. 1.

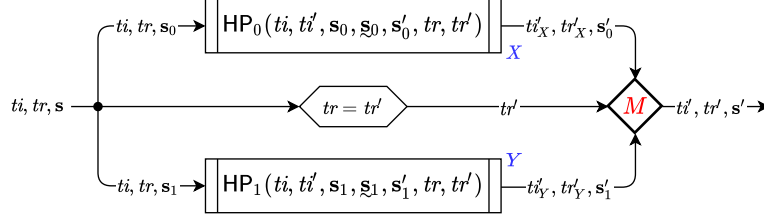


Fig. 1. Parallel-by-merge scheme ($s = s_0 \uplus s_1$ and $s' = s'_0 \uplus s'_1$)

From the above figure, we can see that by renaming the common dashed variables, two processes $HP_{0,X}$ and $HP_{1,Y}$ execute independently and their respective outputs, as well as their common input trace tr , are fed into the merge predicate M . Then, M produces the merged result as the output of the parallel composition. Each merge predicate reflects a corresponding parallel scheme, i.e., the parallel composition is parametric over M . Fig. 1 in this paper looks like Fig. 1 which illustrates the parallel-by-merge dataflow in [15], because both of them are derived from the parallel-by-merge of the classic UTP of [28], but they differ in the following two aspects:

- (1) Continuous state variables $\underline{s}_0$ and $\underline{s}_1$ are involved and state variables s_0 and s_1 are disjoint (no sharing state variables) in our setting;
- (2) We only transfer the input trace tr to the merge predicate, as it would be necessary in parallel composition (see Definition 4.5 for an example).

A merge predicate M is *well-defined* if it satisfies the following three properties:

- (1) $M = M[ti_X, tr_X, s_0, ti_Y, tr_Y, s_1 / ti_Y, tr_Y, s_1, ti_X, tr_X, s_0]$, indicating $\parallel_M$ is commutative.
- (2) The end time of the parallel composition is infinite iff the end time of one process is infinite:

$$M = M \wedge (ti' = +\infty \triangleleft (ti_X = +\infty \vee ti_Y = +\infty) \triangleright ti' < +\infty)$$

- (3) Similarly, the timed trace of the parallel composition is infinite iff the timed trace of one process is infinite:

$$M = M \wedge (|tr'| = +\infty \triangleleft (|tr_X| = +\infty \vee |tr_Y| = +\infty) \triangleright |tr'| < +\infty)$$

The following defines a well-defined merge predicate M_I that specifies parallelism.

Definition 4.5 (M_I). Let s_0 and s_1 be state variables ($s_0 \cap s_1 = \emptyset$) owned by HP_0 and HP_1 , respectively, and I a set of channels via which HP_0 and HP_1 communicate. Then, we define

$$M_I \triangleq \text{syn} \circ \text{mrg}$$

where

$$\begin{aligned} \text{syn} &\triangleq ti'_X = ti_X \wedge ti'_Y = ti_Y \wedge s'_0 = s_0 \wedge s'_1 = s_1 \wedge tr' = tr \wedge \\ &\quad (ti_X < ti_Y \Rightarrow \forall t \in [ti_X, ti_Y] \cdot \underline{s}_0(t) = s_0 \wedge tr'_X = tr_X \wedge \langle ti_Y - ti_X, \emptyset \rangle \wedge tr'_Y = tr_Y) \wedge \\ &\quad (ti_X > ti_Y \Rightarrow \forall t \in [ti_Y, ti_X] \cdot \underline{s}_1(t) = s_1 \wedge tr'_X = tr_X \wedge tr'_Y = tr_Y \wedge \langle ti_X - ti_Y, \emptyset \rangle) \wedge \\ &\quad (ti_X = ti_Y \Rightarrow tr'_X = tr_X \wedge tr'_Y = tr_Y) \\ \text{mrg} &\triangleq ti' = \max\{ti_X, ti_Y\} \wedge s'_0 = s_0 \wedge s'_1 = s_1 \wedge (tr'_X - tr) \parallel_I (tr'_Y - tr) \rightsquigarrow (tr' - tr) \end{aligned}$$

The predicate syn synchronises the termination time ($+\infty$ if non-terminated) of two processes. If $ti_X < ti_Y$, then it delays the termination of the process tagged by X until it reaches ti_Y . During the period of delay, the values of the corresponding state variables keep unchanged, and meanwhile the wait block $\langle ti_Y - ti_X, \emptyset \rangle$ should be added to the end of the trace tr_X . Following syn is mrg which merges times, states and traces, respectively. It uses the maximum time as the termination time ($+\infty$ if non-terminated) of the parallel composition ($ti' = \max\{ti_X, ti_Y\}$).

PROPERTY 6 (WELL-DEFINEDNESS). M_I is well-defined.

4.3 Complete Lattice

PROPERTY 7 (MONOTONICITY). The operators $\%, \sqcup, \sqcap, \triangleleft, \triangleright, \exists$ and $\|_M$ are monotonic and $\neg_l$ is anti-monotone, with respect to the refinement order $\sqsubseteq$.

A (finite or infinite) set $\mathcal{S}$ of hybrid processes equipped with the refinement order $\sqsubseteq$ forms a partially ordered set, denoted by $(\mathcal{S}, \sqsubseteq)$. Let $\bigsqcup \mathcal{S}$ the disjunction of all hybrid processes in $\mathcal{S}$, denoting a system that behaves as any of hybrid processes in $\mathcal{S}$. Obviously, $\text{HP} \sqsubseteq \bigsqcup \mathcal{S}$ iff $\forall X \in \mathcal{S} \cdot \text{HP} \sqsubseteq X$. Conversely, the conjunction of all hybrid processes in $\mathcal{S}$ is denoted by $\bigsqcap \mathcal{S}$, which denotes a system that only has the behaviour shared by all hybrid processes in $\mathcal{S}$. In lattice theory, $\bigsqcup \mathcal{S}$ and $\bigsqcap \mathcal{S}$ are called the infimum and supremum of a set $\mathcal{S}$, respectively.

THEOREM 4.6 (COMPLETE LATTICE). Let $\mathcal{H}_{\text{HP}}$ be the image of $\mathcal{H}_{\text{HP}}$, i.e., it is the set of all hybrid processes, then it forms a complete lattice with top and bottom:

$$\begin{aligned} \top_{\text{HP}} &\hat{=} \bigsqcup \mathcal{H}_{\text{HP}} = \mathcal{H}_{\text{HP}}(\text{false}) \\ \perp_{\text{HP}} &\hat{=} \bigsqcap \mathcal{H}_{\text{HP}} = \mathcal{H}_{\text{HP}}(\text{true}) \end{aligned}$$

Recursion can be specified once a complete lattice is formed. Theoretically speaking, recursion can be denoted by the fixed points of the equation $X = F(X)$, where F constructs the body of the recursion. If $F(X)$ is monotonic, the fixed points of $X = F(X)$ form a complete lattice according to Knaster-Tarski theorem [57]. The least fixed point is denoted by $\mu X \cdot F(X)$ and the greatest fixed point by $\nu X \cdot F(X)$. The definitions of the least and greatest fixed points refer to Section 2. In particular, $\mu X \cdot X = \perp_{\text{HP}}$ and $\nu X \cdot X = \top_{\text{HP}}$.

4.4 Non-termination, Deadlock, Divergence and Miracle

Non-termination has a special property, i.e., being a left zero of sequential composition:

Definition 4.7 (Non-termination). A hybrid process HP is non-terminated iff $\text{HP} \%; \text{HP} = \text{HP}$ for any hybrid process HP .

A hybrid process is *divergent* if its execution time is finite but its trace is infinite; and *deadlocked* if its execution time is infinite with some channel operations waiting forever, specified as follows. It can be demonstrated that both divergence and deadlock are non-terminated by Definition 4.7.

Definition 4.8 (Divergence). A divergence is a fixed point of $X = \mathcal{H}_{\text{HP}} \circ \text{DIV}(X)$ where

$$\text{DIV}(X) = X \wedge ti' < +\infty \wedge |tr'| = +\infty$$

Definition 4.9 (Deadlock). A deadlock is a fixed point of $X = \mathcal{H}_{\text{HP}} \circ \text{DL}(X)$ where

$$\text{DL}(X) = X \wedge ti' = +\infty \wedge \exists N \in \mathbb{N} \cdot \forall n \geq N \cdot tr'(n) = \langle \delta, RS \rangle \wedge RS \neq \emptyset$$

A *miracle* is a miraculous process that only occurs after a non-terminating hybrid process, and thus it is impossible to implement a miracle in reality. Formally,

Definition 4.10 (Miracle). A miracle is a fixed point of $X = \mathcal{H}_{\text{HP}} \circ \text{MIR}(X)$ where

$$\text{MIR}(X) = X \wedge (ti = +\infty \vee |tr| = +\infty)$$

THEOREM 4.11 (MIRACLE). $\top_{\text{HP}}$ is the only miracle of hybrid processes.

Miracle refines any hybrid process HP, i.e., $\text{HP} \sqsubseteq \top_{\text{HP}}$, and it enjoys the following two desired properties, where Property 8 indicates miracle is non-terminated by Definition 4.7.

PROPERTY 8 (LEFT ZERO OF $\circ$). $\top_{\text{HP}} \circ \text{HP} = \top_{\text{HP}}$ for any hybrid process HP.

PROPERTY 9 (ZERO OF $\parallel_M$). $\text{HP} \parallel_M \top_{\text{HP}} = \top_{\text{HP}} \parallel_M \text{HP} = \top_{\text{HP}}$ for any hybrid process HP.

In contrast to the miracle, $\perp_{\text{HP}}$ is the most unpredictable behaviour, because it allows non-deterministically choosing a behaviour among possibly infinitely many processes; in other words, *chaos*. Generally, chaos is expected to be non-terminated (left-zero of sequential composition) and the parallel composition of any process with chaos should result in chaos (zero of parallel composition). However, the chaos $\perp_{\text{HP}}$ does not enjoy such desired properties as the miracle $\top_{\text{HP}}$ does. This problem will be solved in Section 5.

4.5 Abstract Hybrid Processes

A hybrid process models the state and trace of a system over time to accurately render its concrete behavior. For the purpose of verification, however, such a level of detail is certainly excessive, as instead modular abstraction may be desired. To address this issue, we propose the notion of *abstract hybrid process*, where traces are simply abstracted. Formally,

Definition 4.12 (Abstract Hybrid Process). An abstract hybrid process is a fixed point of

$$X = \mathcal{H}_0^A \circ \mathcal{H}_2^A \circ \mathcal{H}_3^A \circ \mathcal{H}_4(X)$$

where

$$\begin{aligned} \mathcal{H}_0^A(X) &\triangleq X \wedge ti \leq ti' \\ \mathcal{H}_2^A(X) &\triangleq (ti = ti') \triangleleft ti = +\infty \triangleright X \\ \mathcal{H}_3^A(X) &\triangleq (\exists s' \cdot X) \triangleleft ti' = +\infty \triangleright X \end{aligned}$$

For brevity, we use $\mathcal{H}_{\text{HP}}^A$ to denote $\mathcal{H}_0^A \circ \mathcal{H}_2^A \circ \mathcal{H}_3^A \circ \mathcal{H}_4$.

THEOREM 4.13 (IDEMPOTENCE AND MONOTONICITY). $\mathcal{H}_{\text{HP}}^A$ is idempotent and monotonic.

Note that $\mathcal{H}_0^A$, $\mathcal{H}_2^A$ and $\mathcal{H}_3^A$ are abstract versions of $\mathcal{H}_0$, $\mathcal{H}_2$ and $\mathcal{H}_3$ in Definition 4.1, respectively. A predicate is an abstract hybrid process iff it is $\mathcal{H}_{\text{HP}}^A$ -healthy. Compared with Definition 4.1 of $\mathcal{H}_{\text{HP}}$, abstract hybrid processes only consider time (ti and ti') and state (s , $\underline{s}$, $\bar{s}$ and s'), ignoring the timed trace (tr and tr'). Therefore, $\mathcal{H}_1$ is no longer necessary here. By Theorem 4.13, abstract hybrid processes can form a complete lattice with top and bottom

$$\begin{aligned} \top_{\text{AHP}} &\triangleq \mathcal{H}_{\text{HP}}^A(\text{false}) \equiv ti = ti' = +\infty \\ \perp_{\text{AHP}} &\triangleq \mathcal{H}_{\text{HP}}^A(\text{true}) \equiv (ti = ti') \triangleleft ti = +\infty \triangleright (ti \leq ti' \wedge RC \wedge SD) \end{aligned}$$

Clearly, $\top_{\text{AHP}} \sqsubseteq \top_{\text{HP}}$ and $\perp_{\text{AHP}} \sqsubseteq \perp_{\text{HP}}$ if we ignore the alphabets as $\top_{\text{AHP}}$ and $\perp_{\text{AHP}}$ do not contain tr or tr' . Abstract hybrid processes also form a Boolean algebra $(\mathbb{AHP}, \sqcap, \sqcup, \neg_i^A, \top_{\text{AHP}}, \perp_{\text{AHP}})$, where $\mathbb{AHP}$ is the image of $\mathcal{H}_{\text{HP}}^A$ and the negation $\neg_i^A$ can be defined by $\neg_i^A X \triangleq \mathcal{H}_{\text{HP}}^A(\neg X)$. Furthermore, an *abstract hybrid condition* is a fixed point of $\neg_i^A X = \neg_i^A X \circ \perp_{\text{AHP}}$. The equation can be denoted by a healthiness condition $\mathcal{H}_{\text{HC}}^A$, i.e., a predicate is an abstract hybrid condition iff it is $\mathcal{H}_{\text{HC}}^A$ -healthy. Abstract hybrid conditions are a subset of abstract hybrid processes.

Additionally, by definition of the refinement relation, for a hybrid process HP with explicit timed traces, if $P \sqsubseteq \text{HP}$ (ignoring the alphabets) then HP does satisfy the property P , which is represented by an abstract hybrid process. Hence, thanks to the lattice structure induced by $\sqsubseteq$, we can define an (α, γ) Galois-connection stipulating the concretisation of a property P to be the meet of all hybrid processes that satisfy it, i.e., $\gamma(P) \triangleq \bigsqcap \{\text{HP} \mid P \sqsubseteq \text{HP}\}$ and, dually, the abstraction of a hybrid process HP to be the join of all properties satisfied by it, i.e., $\alpha(\text{HP}) \triangleq \bigsqcup \{P \mid P \sqsubseteq \text{HP}\}$. In particular, $\gamma(\perp_{\text{AHP}}) = \perp_{\text{HP}}$ and $\alpha(\top_{\text{HP}}) = \top_{\text{AHP}}$.

5 HYBRID DESIGNS

In this section, we propose the concept of *hybrid design*. Hybrid designs are a subset of hybrid processes but enjoy some desired algebraic properties which are not featured by hybrid processes. Based on hybrid designs, we will then propose *normal hybrid designs*, which are the first-class notions in the HUTP theory.

The notion of design in UTP was first introduced by Hoare and He in [28]. A design consists of a pair of predicates $(Pre, Post)$ standing for the pre- and post-condition of a program, augmented with the Boolean variables ok and ok' to express whether the program has started and terminated:

$$Pre \vdash Post \quad \hat{=} \quad (ok \wedge Pre) \Rightarrow (ok' \wedge Post) \quad \equiv \quad \neg ok \vee \neg Pre \vee (ok' \wedge Post)$$

which states that if the program starts in a state satisfying the pre-condition Pre , it will terminate, and on termination the post-condition $Post$ will be true. The pair $(Pre, Post)$ describes a contract between the component and its environment, and therefore supports the decomposition of engineering tasks to resolve system design complexity.

In related works, the meanings of ok and ok' are slightly different. For example, in [15], ok and ok' are used to indicate divergence. Here, however, since divergence can be observed from time and trace ($ti' < +\infty \wedge |tr'| = +\infty$), ok and ok' have different meanings. We extend the notion of design to hybrid systems by interpreting ok and ok' to denote a process's status: $ok' = \mathbf{true}$ indicates that the process executes normally (not necessarily terminating), and hence $ok = \mathbf{false}$ means that the preceding process executed abnormally, causing the behaviour of the current process to be unpredictable. Thus, it is necessary to distinguish normal and abnormal non-termination. Specifically, $\neg ok'$ indicates the abnormal behaviour, which should be *abnormal non-termination*; $ok' \wedge (ti' = +\infty \vee |tr'| = +\infty)$ indicates *normal non-termination*; and $ok' \wedge ti' < +\infty \wedge |tr'| < +\infty$ indicates (normal) termination.

Definition 5.1 (Hybrid Designs). Let HP_A and HP_C be hybrid processes. Then,

$$HP_A \vdash HP_C \quad \hat{=} \quad (ok \wedge HP_A) \Rightarrow_i (ok' \wedge HP_C)$$

is a *hybrid design* with the alphabet $\alpha(HP_A) \cup \alpha(HP_C) \cup \{ok, ok'\}$, where HP_A and HP_C are called *assumption* and *commitment*, respectively.

Hybrid designs are a subset of hybrid processes, as $ok \wedge HP_A$ and $ok' \wedge HP_C$ are $\mathcal{H}_{HP}$ -healthy and $\Rightarrow_i$ is $\mathcal{H}_{HP}$ -preserving, which means $HP_A \vdash HP_C$ is also $\mathcal{H}_{HP}$ -healthy. Intuitively, when a hybrid design starts normally ($ok = \mathbf{true}$) and its assumption HP_A holds, then it executes normally and the commitment HP_C holds. The hybrid design $\perp_{HP} \vdash HP_C$, where $\perp_{HP}$ is a variant of $\mathbf{true}$ (which is a convention in UTP), is abbreviated as $\vdash HP_C$ meaning not making any assumptions.

5.1 Matrix Representation

Given a hybrid design $HP_A \vdash HP_C$, if ok is **false**, i.e., its predecessor is abnormal, then it will behave unpredictably, i.e., chaos $\perp_{HP}$; if ok is **true** but ok' is **false**, it indicates that the assumption HP_A must be violated, i.e., $\neg_i HP_A$; if both ok and ok' are **true**, the commitment HP_C can be guaranteed as soon as the assumption HP_A holds, i.e., $HP_A \Rightarrow_i HP_C$. In order to make this semantics of hybrid designs clearer, we use the matrix encoding proposed in [43] to define hybrid designs as 2×2 matrices by instantiating ok and ok' with all four possible combinations:

$$HP_A \vdash HP_C \quad \hat{=} \quad \begin{pmatrix} \neg ok' & ok' \\ \perp_{HP} & \perp_{HP} \\ \neg_i HP_A & HP_A \Rightarrow_i HP_C \end{pmatrix} \begin{matrix} \neg ok \\ ok \end{matrix} \quad (2)$$

With the aid of the matrix representation, the proofs of the theorems and properties on hybrid designs become more concise and intuitive. The proofs of this section are largely based on this matrix representation and the calculus on it (see the appendix for the details).

5.2 Normal Hybrid Designs

In general, UTP allows the assumption to refer to both primed and unprimed versions of variables. However, as explained in [21], only *normal designs*, whose assumptions are *conditions* and do not refer to primed (state) variables, are significant. Similarly, only *normal hybrid designs*, whose assumptions are fixed points of the equation below, make sense in hybrid system design.

$$\neg_i X = \neg_i X \circ \perp_{HP}$$

Intuitively, the above equation indicates that if an execution violates an assumption, then we expect that any extension of the execution will also violate the assumption. For example, let HP be a hybrid process specifying the expected behaviour, then $\neg_i HP$ represents the unexpected behaviour. It is desirable that all the extensions of $\neg_i HP$ should also be unexpected, i.e., $\neg_i HP \sqsubseteq \neg_i HP \circ \perp_{HP}$. Meanwhile, we have $\neg_i HP \circ \perp_{HP} \sqsubseteq \neg_i HP$ according to the monotonicity of $\circ$ (Property 7). In a word, we expect the property $\neg_i HP = \neg_i HP \circ \perp_{HP}$.

The above equation can be denoted by a healthiness condition

$$\mathcal{H}_{HC}(X) = \neg_i(\neg_i X \circ \perp_{HP})$$

A predicate is called a *hybrid condition* if it is $\mathcal{H}_{HC}$ -healthy in a similar manner as reactive conditions in [15]. Hybrid conditions are a subset of hybrid processes. For example, $\mathcal{H}_{HP}(s > 0)$ is a hybrid condition while $\mathcal{H}_{HP}(s' > 0)$ is not. Especially, $\perp_{HP}$ and $\top_{HP}$ are hybrid conditions because it is easy to check that $\perp_{HP} = \neg_i(\neg_i \perp_{HP} \circ \perp_{HP})$ and $\top_{HP} = \neg_i(\neg_i \top_{HP} \circ \perp_{HP})$.

A hybrid design, say $HC \vdash HP$, is called *normal hybrid design*, if HC is a hybrid condition ($\mathcal{H}_{HC}$ -healthy). In what follows, we only consider normal hybrid designs. Operations on normal hybrid designs can be thus reduced to standard matrix operations. In particular, non-deterministic choice (meet) $\sqcap$ and sequential composition $\circ$ correspond to matrix addition and multiplication, respectively. Moreover, reasoning is completely component-free (with no mention to *ok* and *ok'* or variable substitutions), i.e., *compositional*.

THEOREM 5.2 (OPERATIONS).

- (1) $(HC_0 \vdash HP_0) \sqcap (HC_1 \vdash HP_1) = (HC_0 \wedge HC_1) \vdash (HP_0 \vee HP_1)$
- (2) $(HC_0 \vdash HP_0) \sqcup (HC_1 \vdash HP_1) = (HC_0 \vee HC_1) \vdash ((HC_0 \Rightarrow_i HP_0) \wedge (HC_1 \Rightarrow_i HP_1))$
- (3) $(HC_0 \vdash HP_0) \circ (HC_1 \vdash HP_1) = (HC_0 \wedge \neg_i(HP_0 \circ \neg_i HC_1)) \vdash (HP_0 \circ HP_1)$

Parallel-by-merge, as defined for hybrid processes in Section 4.2.5, can naturally be extended to normal hybrid designs, except that the form of merge predicates should be adapted to deal with auxiliary variables *ok* and *ok'*. Concretely, the merge predicate for normal hybrid designs can be defined by:

$$NHD(M) \triangleq (ok_X \wedge ok_Y) \Rightarrow_i (M \wedge ok')$$

where $\alpha(M) \cap \{ok_X, ok_Y, ok, ok'\} = \emptyset$. It states that only parallel designs execute normally (i.e. $ok_X \wedge ok_Y$) can they merge by *M* as usual. Otherwise, merge becomes unpredictable.

THEOREM 5.3 (PARALLEL COMPOSITION).

$$(HC_0 \vdash HP_0) \parallel_{NHD(M)} (HC_1 \vdash HP_1) = \left(\begin{array}{l} \neg_i(\neg_i HC_0 \parallel_{\perp_{HP}} \neg_i HC_1) \\ \wedge \neg_i(\neg_i HC_0 \parallel_{\perp_{HP}} HP_1) \\ \wedge \neg_i(\neg_i HC_1 \parallel_{\perp_{HP}} HP_0) \end{array} \right) \vdash HP_0 \parallel_M HP_1$$

Intuitively, the assumption of parallel composition, above, stipulates that unexpected interactions cannot happen. For example, $\neg_i HC_0$ denotes the unexpected behaviour of $HC_0 \vdash HP_0$. Hence, its parallel composition with the behaviour of the other side, i.e., $\neg_i HC_0 \parallel_{\perp HP} \neg_i HC_1$ and $\neg_i HC_0 \parallel_{\perp HP} HP_1$, should also be unexpected, as negated by the assumption, where the merge predicate $\perp_{HP}$ denotes the weakest parallelism, i.e., $(P \parallel_{\perp HP} Q) \sqsubseteq (P \parallel_M Q)$ for any merge predicate M .

If state variables play the dominant role in the assumptions and traces are not the concern, such as $\mathcal{H}_{HP}(s > 0)$, we can give an alternative definition to parallel composition as follows, which is simpler than $\parallel_{NHD(M)}$.

Definition 5.4 (Parallel Composition).

$$(HC_0 \vdash HP_0) \parallel_M^{HP} (HC_1 \vdash HP_1) \hat{=} (HC_0 \wedge HC_1) \vdash (HP_0 \parallel_M HP_1)$$

For normal hybrid designs, the assumption for non-chaos is made explicitly. Since parallel execution involves the evaluation of both processes, it is necessary to ensure that both their assumptions are valid to start with. Thus, the assumption for successful execution of the parallel composition is the conjunction of their separate assumptions rather than their disjunction. It states that the parallel composition $HP_0 \parallel_M^{HP} HP_1$ can only be executed when both of the components successfully start with the assumptions HC_0 and HC_1 holding, otherwise, it is a chaos. Note that $\parallel_M^{HP}$ is different from $\parallel_{NHD(M)}$, the former composes assumptions and commitments separately while the latter treats hybrid designs as hybrid processes.

THEOREM 5.5 (CLOSURE). *Normal hybrid designs are closed on $\sqcap$, $\sqcup$, $\circ$, $\parallel_{NHD(M)}$ and $\parallel_M^{HP}$.*

THEOREM 5.6 (REFINEMENT). *$(HC_0 \vdash HP_0) \sqsubseteq (HC_1 \vdash HP_1)$ iff*

$$HC_1 \sqsubseteq HC_0 \quad \text{and} \quad HP_0 \sqsubseteq (HC_0 \wedge HP_1)$$

This theorem indicates that the monotonic hybrid refinement relation can be lifted to a contravariant design refinement relation, and hence the monotonicity of modalities for normal hybrid designs can be proved, specified as the following two properties.

PROPERTY 10 (CONTRA-VARIANCE). *If $HC_0 \supseteq HC_1$ and $HP_0 \sqsubseteq HP_1$ then*

$$(HC_0 \vdash HP_0) \sqsubseteq (HC_1 \vdash HP_1)$$

PROPERTY 11 (MONOTONICITY). *Operators $\sqcup$, $\sqcap$, $\circ$, $\parallel_{NHD(M)}$ and $\parallel_M^{HP}$ for normal hybrid designs are monotonic with respect to $\sqsubseteq$.*

5.3 Complete Lattice

The laws for $\sqcap$ and $\sqcup$ can be respectively generalised to $\prod$ and $\bigsqcup$, which indicates that normal hybrid designs form a complete lattice, as stated by the following theorem.

THEOREM 5.7 (COMPLETE LATTICE). *Normal hybrid designs form a complete lattice with top*

$$\top_{NHD} \hat{=} \perp_{HP} \vdash \top_{HP}$$

and bottom

$$\perp_{NHD} \hat{=} \top_{HP} \vdash \perp_{HP}$$

PROPERTY 12 (CHAOS). $\perp_{NHD} = \top_{HP} \vdash HP$ *for any hybrid process HP .*

PROPERTY 13 (NON-TERMINATION). *For any normal hybrid design NHD ,*

$$\perp_{NHD} \circ NHD = \perp_{NHD} \quad \text{and} \quad \top_{NHD} \circ NHD = \top_{NHD}$$

PROPERTY 14 (PARALLEL COMPOSITION). *For any normal hybrid design NHD,*

$$\text{NHD} \parallel_{\text{NHD}(M)} \top_{\text{NHD}} = \top_{\text{NHD}} \parallel_{\text{NHD}(M)} \text{NHD} = \top_{\text{NHD}}$$

PROPERTY 15 (PARALLEL COMPOSITION). *For any normal hybrid design NHD,*

$$\text{NHD} \parallel_M^{\text{HP}} \perp_{\text{NHD}} = \perp_{\text{NHD}} \parallel_M^{\text{HP}} \text{NHD} = \perp_{\text{NHD}}$$

The above conclusions demonstrate that the chaos $\perp_{\text{NHD}}$ of normal hybrid designs enjoys the desired properties that (1) it is non-terminated (left-zero of sequential composition, see Property 13) and (2) its parallel composition with any normal hybrid design can result in itself (zero of parallel composition $\parallel_M^{\text{HP}}$, see Property 15), which fixes the hole in the HUTP theory mentioned at the end of Section 4 (where the chaos $\perp_{\text{HP}}$ of hybrid processes does not enjoy such desired properties).

6 REFLECTION OF HCSP AND SIMULINK WITH HUTP

In this section, we give the HUTP semantics of HCSP [64] and Simulink [39], representing two representative imperative and data-flow formalisms for hybrid system design in academia and industry, respectively. Furthermore, we prove the consistency between the operational semantics of HCSP and its HUTP semantics. We illustrate by an example the refinement relation between a Simulink diagram and the corresponding HCSP model within the HUTP framework, demonstrating the expressive capabilities of the UTP as a meta-theory for translation validation.

6.1 Syntactic Sugar

For brevity, we introduce the following syntax sugar for HUTP. We define the notations $\llbracket \cdot \rrbracket$ and $\lceil \cdot \rceil$ which are similar to $\llbracket \cdot \rrbracket$ and $\lceil \cdot \rceil$ from Duration Calculus [66]. Let $P(\underline{s}, \dot{\underline{s}})$ denote a predicate relating $\underline{s}$ and its derivative $\dot{\underline{s}}$.

$$\llbracket P(\underline{s}, \dot{\underline{s}}), RS \rrbracket_{\phi(ti, ti')} \hat{=} \mathcal{H}_{\text{HP}} \left(\begin{array}{l} ti < ti' < +\infty \wedge \phi(ti, ti') \wedge \underline{s} = \underline{s}(ti) \wedge \underline{s}' = \underline{s}(ti') \wedge \\ \forall t \in (ti, ti') \cdot P(\underline{s}(t), \dot{\underline{s}}(t)) \wedge tr' - tr = \langle ti' - ti, RS \rangle \end{array} \right)$$

is a finite continuous process over the time interval $[ti, ti']$, and it says that $\underline{s}$ is differentiable with $P(\underline{s}, \dot{\underline{s}})$ holding at every instant t from ti to ti' . During the period, the channel operations in the ready set RS are waiting, recorded by the trace history $tr' - tr$. The predicate $\phi(ti, ti')$ characterises the relation of ti and ti' , such as $ti' - ti = 2$. We omit $\phi(ti, ti')$ if $\phi(ti, ti') = (ti < ti')$. For brevity,

$$\begin{aligned} \llbracket P(\underline{s}, \dot{\underline{s}}), RS \rrbracket_d &\hat{=} \llbracket P(\underline{s}, \dot{\underline{s}}), RS \rrbracket_{ti' - ti = d} \\ \llbracket P(\underline{s}, \dot{\underline{s}}), RS \rrbracket_{\sim d} &\hat{=} \llbracket P(\underline{s}, \dot{\underline{s}}), RS \rrbracket_{(ti' - ti) \sim d} \end{aligned}$$

where $d > 0$ and $\sim \in \{<, \leq, \neq, >, \geq\}$.

A sequence of operations which is assumed to take no time is called *super-dense computation* [38]. Under super-dense computation, the computer processing the sequence of the operations is much faster than the physical devices attached to it, rendering the time to compute the discrete operations negligible. However, the temporal order of computations is still present. Under the assumption of super-dense computation, a discrete process without communication is defined by

$$\lceil P(\underline{s}, \underline{s}') \rceil \hat{=} \mathcal{H}_{\text{HP}}(ti = ti' < +\infty \wedge P(\underline{s}, \underline{s}') \wedge tr' - tr = \tau)$$

It executes instantly at time $ti = ti' < +\infty$, rather than continuously over a time interval. Especially,

$$\lceil \rceil \hat{=} \mathcal{H}_{\text{HP}}(ti = ti' < +\infty \wedge \underline{s} = \underline{s}' \wedge tr = tr')$$

Note that $\lceil \rceil \neq \lceil \underline{s} = \underline{s}' \rceil$ as their traces are different. We can prove the following property for $\lceil \rceil$.

PROPERTY 16 (UNIT). $\lceil \rceil$ is a unit of hybrid processes w.r.t. $\circ$.

COROLLARY 1 (UNIT). $\vdash [\]$ is a unit of normal hybrid designs w.r.t. $\circ$.

Instant communications can be defined by discrete processes with communication:

$$\begin{aligned} [ch?, s_i] &\hat{=} \mathcal{H}_{HP}(ti = ti' < +\infty \wedge \exists d \cdot s_i := d \wedge tr' - tr = \langle ch?, d \rangle) \\ [ch!, e(s)] &\hat{=} \mathcal{H}_{HP}(ti = ti' < +\infty \wedge s' = s \wedge tr' - tr = \langle ch!, e(s) \rangle) \end{aligned}$$

where s_i is the i -th variable in s and $e(s)$ is an expression containing s .

In addition, we also define infinite continuous processes:

$$\llbracket P(\underline{s}, \underline{\dot{s}}), RS \rrbracket_\infty \hat{=} \mathcal{H}_{HP} \left(\begin{aligned} &ti < ti' = +\infty \wedge s = \underline{s}(ti) \wedge \\ &\forall t \in (ti, +\infty) \cdot P(\underline{s}(t), \underline{\dot{s}}(t)) \\ &\wedge tr' - tr = \langle +\infty, RS \rangle \end{aligned} \right)$$

Since we cannot observe the state variables of the point at infinity, the process will not output any values of state variables, i.e., the output state variables s' are hidden.

6.2 From HCSP to HUTP

HCSP is a formal language for describing hybrid systems. It extends Communicating Sequential Processes by introducing differential equations for modelling continuous evolutions and interrupts for modelling arbitrary interactions between continuous evolutions and discrete jumps. The syntax of HCSP is given as follows, adapted from [64].

$$\begin{aligned} P &::= \mathbf{skip} \mid x := e \mid \mathbf{wait} \ d \mid ch?x \mid ch!e \mid X \mid \mathbf{rec} \ X \cdot P \mid P; P \mid P \sqcap P \mid B \rightarrow P \mid \\ &\quad \langle F(\dot{s}, s) = 0 \& B \rangle \mid \langle F(\dot{s}, s) = 0 \& B \rangle \geq_d P \mid \langle F(\dot{s}, s) = 0 \& B \rangle \geq \llbracket i \in I (io_i \rightarrow P_i) \rrbracket \\ S &::= P \mid S \parallel S \end{aligned} \tag{3}$$

The HCSP constructs can be defined respectively by normal hybrid designs (Section 5.2) as follows:

- The **skip** statement terminates immediately having no effect on variables, and it is modelled as the relational identity:

$$\llbracket \mathbf{skip} \rrbracket_{HUTP} \hat{=} \vdash [\]$$

It is a unit of normal hybrid designs w.r.t. $\circ$ by Corollary 1.

- The assignment of the value e to a variable x is modelled as setting x to e and keeping all other variables constant if e can be successfully evaluated [28]. Therefore, the assignment can be interpreted as follows:

$$\llbracket x := e \rrbracket_{HUTP} \hat{=} E(e) \vdash [x := e]$$

where $E(e)$ is $\mathcal{H}_{HC}$ -healthy and specifies the condition by which e can be evaluated. Note that E is a function supplied for all expressions. For example, $\mathcal{H}_{HP}(s \neq 0)$ can be such a condition, where s is a state variable in e .

- The **wait** statement will keep idle for d time units keeping variables unchanged:

$$\llbracket \mathbf{wait} \ d \rrbracket_{HUTP} \hat{=} \vdash \llbracket \dot{s} = \mathbf{0}, \mathbf{0} \rrbracket_d$$

- The sequential composition $P_0; P_1$ behaves as P_0 first, and if it terminates, as P_1 afterwards:

$$\llbracket P_0; P_1 \rrbracket_{HUTP} \hat{=} \llbracket P_0 \rrbracket_{HUTP} \circ \llbracket P_1 \rrbracket_{HUTP}$$

- $P_0 \sqcap P_1$ denotes internal choice, which behaves as either P_0 or P_1 , and the choice is made by the process. It is interpreted as a demonic choice of two operands in a standard way:

$$\llbracket P_0 \sqcap P_1 \rrbracket_{HUTP} \hat{=} \llbracket P_0 \rrbracket_{HUTP} \sqcap \llbracket P_1 \rrbracket_{HUTP}$$

- The alternative $B \rightarrow P$, where B is a Boolean expression, behaves as P if B is true; otherwise it terminates immediately:

$$\llbracket B \rightarrow P \rrbracket_{\text{HUTP}} \hat{=} \llbracket P \rrbracket_{\text{HUTP}} \triangleleft B \triangleright \llbracket \text{skip} \rrbracket_{\text{HUTP}}$$

- A process variable X is interpreted as a predicate variable:

$$\llbracket X \rrbracket_{\text{HUTP}} \hat{=} X$$

- The recursion $\text{rec } X \cdot P$ means that the execution of P can be repeated by replacing each occurrence of X with $\text{rec } X \cdot P$ itself during executing P , i.e., $\text{rec } X \cdot P$ behaves like $P[\text{rec } X \cdot P/X]$. The semantics for recursion can be defined as the least or the greatest fixed point by

$$\begin{aligned} \llbracket \text{rec } X \cdot P \rrbracket_{\text{HUTP}}^{\mu} &\hat{=} \mu X \cdot \llbracket P \rrbracket_{\text{HUTP}} \\ \llbracket \text{rec } X \cdot P \rrbracket_{\text{HUTP}}^{\nu} &\hat{=} \nu X \cdot \llbracket P \rrbracket_{\text{HUTP}} \end{aligned}$$

For example, the semantics of P^* can be defined as

$$\llbracket P^* \rrbracket_{\text{HUTP}} \hat{=} \llbracket \text{rec } X \cdot \text{skip} \sqcap (P; X) \rrbracket_{\text{HUTP}}^{\mu} = \exists n \in \mathbb{N} \cdot \llbracket P^n \rrbracket_{\text{HUTP}}$$

where $\mathbb{N}$ is the set of non-negative integers and $P^0 \hat{=} \text{skip}$.

- A continuous evolution statement $\langle F(\dot{s}, s) = 0 \& B \rangle$ says that the process keeps waiting, and meanwhile keeps continuously evolving following the differential equations F , until the domain constraint B is violated:

$$\llbracket \langle F(\dot{s}, s) = 0 \& B \rangle \rrbracket_{\text{HUTP}} \hat{=} \text{exit} \sqcap (\text{ode}^0 ; \text{exit})$$

where

$$\begin{aligned} \text{ode}^{RS} &\hat{=} \vdash \llbracket F(\dot{s}, \underline{s}) = 0 \wedge B[\underline{s}/s], RS \rrbracket \\ \text{exit} &\hat{=} \vdash \llbracket \neg B(s) \wedge s' = s \rrbracket \end{aligned}$$

Note that ode^{RS} and exit take F and B as parameters. For brevity, the parameters are not shown in the following content. The above states that the process can either terminate at the beginning, i.e., exit , or evolve for a finite period before terminating without waiting communications, i.e., $\text{ode}^0 ; \text{exit}$, depending on whether B holds or not.

- $\langle F(\dot{s}, s) = 0 \& B \rangle \triangleright_d P$ behaves like $\langle F(\dot{s}, s) = 0 \& B \rangle$, if the evolution terminates before d time units. Otherwise, after d time units of evolution according to F , it moves on to execute P :

$$\llbracket \langle F(\dot{s}, s) = 0 \& B \rangle \triangleright_d P \rrbracket_{\text{HUTP}} \hat{=} \text{exit} \sqcap (\text{ode}_{<d}^0 ; \text{exit}) \sqcap (\text{ode}_d^0 ; \llbracket P \rrbracket_{\text{HUTP}})$$

It can either terminate at the beginning, i.e., exit , evolve less than d time units before terminating, i.e., $\text{ode}_{<d}^0 ; \text{exit}$, or evolve for d time units and then continue to execute P , i.e., $\text{ode}_d^0 ; \llbracket P \rrbracket_{\text{HUTP}}$.

- $\langle F(\dot{s}, s) = 0 \& B \rangle \triangleright \llbracket \bigwedge_{i \in I} (io_i \rightarrow P_i) \rrbracket$ behaves like $\langle F(\dot{s}, s) = 0 \& B \rangle$, except that the continuous evolution is preempted as soon as one of the communications io_i takes place, which is followed by the respective P_i , where io_i stands for a communication event, i.e., either $ch?x$ or $ch!e$. Notice that if the continuous part terminates before a communication among $\{io_i \mid i \in I\}$ occurs, then the process terminates without communicating. Concretely,

$$\llbracket \langle F(\dot{s}, s) = 0 \& B \rangle \triangleright \llbracket \bigwedge_{i \in I} (io_i \rightarrow P_i) \rrbracket \rrbracket_{\text{HUTP}} \hat{=} \text{exit} \sqcap \text{comm} \sqcap (\text{ode}^{RS} ; (\text{exit} \sqcap \text{comm})) \quad (4)$$

Without loss of generality, we let $I = I_? \cup I_!$ with $I_? \cap I_! = \emptyset$ such that $io_i = ch_i?x_i$ if $i \in I_?$ and $ch_i!e_i$ otherwise ($i \in I_!$), where e_i is an arithmetic expression. Note that x_i and x_j can denote the same variable even if $i \neq j$. Let $RS = \{ch_i?, ch_j! \mid i \in I_?, j \in I_!\}$ be the set of waiting channel operations. Then, the preemption by communication can be described by

$$\text{comm} \hat{=} \text{input} \sqcap \text{output}$$

where

$$\begin{aligned} \text{input} &\triangleq \prod_{i \in I_i} (\vdash [ch_i?, x_i] \ ; \ \llbracket P_i \rrbracket_{\text{HUTP}}) \\ \text{output} &\triangleq \prod_{j \in I_o} (\text{E}(e_j(s)) \vdash [ch_j!, e_j(s)] \ ; \ \llbracket P_j \rrbracket_{\text{HUTP}}) \end{aligned}$$

It states that the process can either terminate or communicate at the beginning: $\text{exit} \sqcap \text{comm}$, evolve for a finite period before terminating or communicating: $\text{ode}^{RS} \ ; \ (\text{exit} \sqcap \text{comm})$.

- $ch?x$ receives a value along channel RS and assigns it to x , and as the dual, $ch!e$ sends the value of e along channel RS :

$$\begin{aligned} \llbracket ch?x \rrbracket_{\text{HUTP}} &\triangleq \llbracket \langle \dot{s} = 0 \& \text{true} \rangle \sqsupseteq (ch?x \rightarrow \text{skip}) \rrbracket_{\text{HUTP}} \\ \llbracket ch!e \rrbracket_{\text{HUTP}} &\triangleq \llbracket \langle \dot{s} = 0 \& \text{true} \rangle \sqsupseteq (ch!e \rightarrow \text{skip}) \rrbracket_{\text{HUTP}} \end{aligned}$$

A communication takes place when both the sending and the receiving parties are ready, and may cause one side to wait if the other side is not ready.

- $P_0 \parallel P_1$ behaves as if sequential processes P_0 and P_1 run independently except that all communications along the common channels connecting P_0 and P_1 are to be synchronised. The processes P_0 and P_1 in parallel can neither share variables, nor input or output channels, i.e., a channel operation $ch*$ appears in P_0 iff it does not appear in P_1 . The parallel composition of P_0 and P_1 can be translated as the parallel composition defined by Definition 5.4 with the merge predicate M_I defined by Definition 4.5:

$$\llbracket P_0 \parallel P_1 \rrbracket_{\text{HUTP}} \triangleq \llbracket P_0 \rrbracket_{\text{HUTP}} \parallel_{M_I}^{\text{HP}} \llbracket P_1 \rrbracket_{\text{HUTP}} \quad (5)$$

where I is the set of common channels between P_0 and P_1 .

In what follows we prove the consistency between the HUTP semantics and the structural operational semantics (SOS) of HCSP given in [64]. The latter consists of a collection of transition rules of the following form¹:

$$\frac{\text{Pre-condition}}{(P, (ti, s)) \xrightarrow{\text{tb}} (P', (ti', s'), \underline{s})}$$

where P and P' are HCSP processes, (ti, s) and (ti', s') are process states, described by the state variables with timestamps, tb denote a trace block defined in Section 3, and $\underline{s}$ is a flow depicting the evolution from s to s' . Note that the flow $\underline{s}$ can be removed if the above transition describes a discrete action. For example, the SOS of an assignment is

$$\frac{e \text{ can be evaluated on } s}{(x := e, (ti, s)) \xrightarrow{\tau} (\epsilon, (ti, s[x \mapsto e(s)]))}$$

where ϵ denotes termination. A more comprehensive understanding of the operational semantics can be found in [64].

We say the HUTP semantics and the SOS of an HCSP process are consistent iff they keep consistent on time, state and trace. Concretely, the changes of time and state and the generated trace of the SOS of an HCSP process should be reflected by its HUTP semantics, and vice versa. For example, the SOS for the ODE with communication interruption

$$\text{ODE}_{\sqsupseteq} \triangleq \langle F(\dot{s}, s) = 0 \& B \rangle \sqsupseteq \prod_{i \in I} (ch_i * i \rightarrow P_i)$$

¹We here revise the operational semantics in [64] a little bit.

is described by the following rules:

$$\begin{array}{c}
\frac{\neg B(\mathbf{s})}{(\text{ODE}_{\geq}, (ti, \mathbf{s})) \xrightarrow{\tau} (\varepsilon, (ti, \mathbf{s}))} [\text{Int0}] \quad \frac{\begin{array}{c} \mathbf{s} \text{ is a solution of } F(\mathbf{s}, \mathbf{s}) = 0 \\ \mathbf{s}(ti) = \mathbf{s} \quad \mathbf{s}' = \mathbf{s}(ti') \quad \forall t \in [ti, ti'] \cdot B(\mathbf{s}(t)) \end{array}}{(\text{ODE}_{\geq}, (ti, \mathbf{s})) \xrightarrow{\langle ti' - ti, \{ch_i?, ch_j! \mid i \in I_?, j \in I_!\} \rangle} (\text{ODE}_{\geq}, (ti', \mathbf{s}'), \mathbf{s})} [\text{Int1}] \\
\\
\frac{i \in I_? \quad ch_i * i = ch_i? x_i}{(\text{ODE}_{\geq}, (ti, \mathbf{s})) \xrightarrow{\langle ch_i?, d \rangle} (P_i, (ti, \mathbf{s}[x_i \mapsto d]))} [\text{Int2}] \quad \frac{j \in I_! \quad ch_j * j = ch_j! e_j \quad e_j \text{ can be evaluated on } \mathbf{s}}{(\text{ODE}_{\geq}, (ti, \mathbf{s})) \xrightarrow{\langle ch_j!, e_j(\mathbf{s}) \rangle} (P_j, (ti, \mathbf{s}))} [\text{Int3}]
\end{array}$$

The rule [Int0] is reflected by exit and the rule [Int1] is reflected by $\text{ode}^{RS} \circ \llbracket \text{ODE}_{\geq} \rrbracket_{\text{HUTP}}$, where $RS = \{ch_i?, ch_j! \mid i \in I_?, j \in I_!\}$. The rule [Int2], equivalent to $\vdash [ch_i?, x_i]$, depicts a receiving event that interrupts the ODE, followed by the process P_i . The HUTP semantics of P_i is denoted by $\llbracket P_i \rrbracket_{\text{HUTP}}$, and therefore, the HUTP semantics for the receiving event can be described by

$$\vdash [ch_i?, x_i] \quad ; \quad \llbracket P_i \rrbracket_{\text{HUTP}}$$

Similarly, by [Int3], the HUTP semantics of the sending event is denoted by

$$E(e_j) \vdash [ch_j?, e_j] \quad ; \quad \llbracket P_j \rrbracket_{\text{HUTP}}$$

where $E(e_j)$ means e_j can be evaluated successfully. The process comm summarises the HUTP semantics of the above communications. Note that $\text{ODE}_{\geq}$ terminates after applying these rules except [Int1]. In summary, we can get

$$\text{ODE}_{\geq} = \text{exit} \sqcap \text{comm} \sqcap (\text{ode}^{RS} \circ \text{ODE}_{\geq})$$

In other words, the SOS of $\text{ODE}_{\geq}$ is equivalent to the equation $X = F(X)$, where

$$F(X) \triangleq \text{exit} \sqcap \text{comm} \sqcap (\text{ode}^{RS} \circ X)$$

Any solution of $X = F(X)$ can be treated as the HUTP semantics of $\text{ODE}_{\geq}$. In this paper, we use the least fixed point:

$$\mu X. F(X) = \text{exit} \sqcap \text{comm} \sqcap (\text{ode}^{RS} \circ (\text{exit} \sqcap \text{comm}))$$

as shown in (4). It can be checked that the SOS of $\text{ODE}_{\geq}$, denoted by the above four transition rules, is reflected by (4), and vice versa.

REMARK 1. Another solution of the equation $X = F(X)$ above is

$$\text{ode}_{\infty}^{RS} \sqcap \text{exit} \sqcap \text{comm} \sqcap (\text{ode}^{RS} \circ (\text{exit} \sqcap \text{comm}))$$

where $\text{ode}_{\infty}^{RS} \triangleq \vdash \llbracket F(\mathbf{s}, \mathbf{s}) = 0 \wedge B[\mathbf{s}/\mathbf{s}], RS \rrbracket_{\infty}$ denotes $\text{ODE}_{\geq}$ will evolve forever, never terminate. This solution is reasonable, although it is neither the least nor the greatest fixed point.

THEOREM 6.1 (SEMANTIC CONSISTENCY). The HUTP semantics of HCSP is consistent with the structural operational semantics of HCSP.

6.3 From Simulink to HUTP

Matlab/Simulink [39] is an industrial de-facto standard for modelling embedded systems. A Simulink diagram consists of a set of blocks. Each block has a set of input and output signals. A *discrete* block computes the output signal from the input signals periodically or aperiodically. A *continuous* block computes the output signal continuously by following a differential equation. Blocks can be organised into hierarchical subsystems, such as normal, triggered and enabled subsystems.

In this subsection, we introduce a formal semantics of Simulink by encoding it into HUTP. Overall, a Simulink diagram is split into discrete and continuous sub-diagrams which consist of discrete and continuous blocks, respectively. We define the respective semantics for the discrete and continuous sub-diagrams and then compose them together. The following Simulink diagrams serve as running examples. The left of Fig. 2 is the original diagram and the right is the diagram where the ports between some blocks are added.

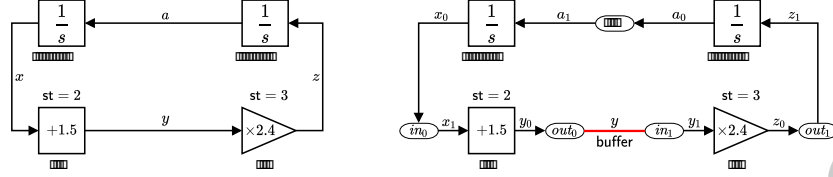


Fig. 2. Examples of plant-control loop

6.3.1 Discrete Block. A discrete block is executed during a finite time interval. The left of Fig. 3 is the typical periodic discrete block of a math operation, such as Bias and Gain in Fig. 2. This kind of blocks are stateless. The block on the right is a periodic Unit Delay block, which is stateful.

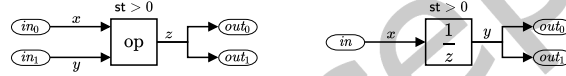


Fig. 3. Discrete blocks

For each discrete block, we use ports to serve as channels for the input and output lines. For a periodic Math Operation block, periodically, one first gets data from the input ports (get), then performs a computation based on the inputs (comp), next puts the computed result to its output ports (put), and finally keeps silent for a period of time (period), as follows:

$$\begin{aligned}
 \text{get} &\triangleq \vdash [in_0?, x] \wedge G_0(x') \ ; \ \vdash [in_1?, y] \wedge G_1(y') \\
 \text{comp} &\triangleq A(x, y) \vdash [z := \text{op}(x, y)] \\
 \text{put} &\triangleq \vdash [out_0!, z] \ ; \ \vdash [out_1!, z] \\
 \text{period} &\triangleq \vdash [\ddot{z} = 0, \emptyset]_{st}
 \end{aligned}$$

In the above, get acquires data x and y from channels (ports) in_0 and in_1 , respectively. Note that $[in_0?, x]$ and $[in_1?, y]$ are instant communications (i.e., fetch data whenever they require). The constraints $G_0(x')$ and $G_1(y')$ mean that only the desired data is received. Receiving does not consume time, so the order of channel operations $in_0?$ and $in_1?$ does not matter. comp denotes a computation, where $A(x, y)$ means that the operands x and y , provided from get, must satisfy some assumption, such as $y \neq 0$. Therefore, the relation $G_0(x) \wedge G_1(y) \Rightarrow A(x, y)$ must hold. If $A(x, y)$ is satisfied, the computation $[z := \text{op}(x, y)]$ executes. By contrast with get, put just puts the computed result to channels (ports) out_0 and out_1 . Note that get, comp and put do not consume time. Then, the process keeps quiescent for some period specified by the sample time st , denoted by period, where $\emptyset$ indicates the $in_0?$, $in_1?$, $out_0!$ and $out_1!$ are not ready to communicate with the context during the period. Finally, it can terminate at any time before the next period arrives (tail). In summary, a periodic Math Operation block can be translated as:

$$\text{MOP} \triangleq \text{get} \ ; \ \text{comp} \ ; \ \text{put} \ ; \ (\text{period} \ ; \ \text{get} \ ; \ \text{comp} \ ; \ \text{put})^* \ ; \ \text{tail} \quad (6)$$

where

$$\text{tail} \triangleq \vdash \llbracket \dot{z} = 0, \emptyset \rrbracket_{<st} \sqcap \vdash \llbracket \cdot \rrbracket$$

Unlike a Math Operation block, a Unit Delay block is stateful. The right of Fig. 3 is a typical periodic Unit Delay block. At the beginning, its state is initialised (init). Then, periodically, it puts the current state to the output channels (ports) out_0 and out_1 , then gets data from the input channel (port) in and updates the state (get), and finally holds the state for some period (period). In summary, a Unit Delay block can be translated as follows:

$$\text{UnitDelay} \triangleq \text{init} \circ \text{put} \circ \text{get} \circ (\text{period} \circ \text{put} \circ \text{get})^* \circ \text{tail}$$

where

$$\begin{aligned} \text{init} &\triangleq \vdash [y := \text{init}_y] \\ \text{put} &\triangleq \vdash [out_0!, y] \circ \vdash [out_1!, y] \\ \text{get} &\triangleq \vdash [in?, x] \circ \vdash [y := x] \\ \text{period} &\triangleq \vdash \llbracket \dot{y} = 0, \emptyset \rrbracket_{st} \\ \text{tail} &\triangleq \vdash \llbracket \dot{y} = 0, \emptyset \rrbracket_{<st} \sqcap \vdash \llbracket \cdot \rrbracket \end{aligned}$$

6.3.2 Continuous Block. Blocks containing continuous states are called continuous blocks. Computing a continuous state requires one to know the derivative of the state variables. The left of Fig. 4 is an Integrator block and the right is a Math Operation block of which sample time is 0. They are both typical continuous blocks.



Fig. 4. Continuous blocks

For an Integrator block, the state is first initialised by an initial condition, as follows:

$$\text{init} \triangleq \vdash [y := \text{init}_y]$$

Then, its semantics depends on the context it interacts with:

- If the source block is continuous (such as Integrator0 in Fig. 2), then the block will evolve continuously following the ODE $\dot{y} = x$, instead of receiving x periodically or aperiodically from the input port. Meanwhile, it will send the current value of y to the output port out_1 whenever the non-continuous target block requires it (put). In that case, the corresponding HUTP semantics will be given as follows:

$$\text{Integrator} \triangleq \text{init} \circ (\text{put} \sqcap \text{ode}_0)^* \quad (7)$$

where

$$\begin{aligned} \text{put} &\triangleq \vdash [out_1!, y] \\ \text{ode}_0 &\triangleq \vdash \llbracket \dot{y} = x, \{out_1!\} \rrbracket \end{aligned}$$

- On the other hand, if the source block is non-continuous (such as Integrator1 in Fig. 2), then, at each iteration, the block will either get data x from the input channel (get), evolve according to the latest received data (ode₁), or put the current value of y to the output channel (put), as follows:

$$\text{Integrator} \triangleq \text{init} \circ (\text{get} \sqcap \text{put} \sqcap \text{ode}_1)^* \quad (8)$$

where

$$\begin{aligned} \text{get} &\triangleq \vdash [in?, x] \\ \text{ode}_1 &\triangleq \vdash \llbracket \dot{y} = x \wedge \dot{x} = 0, \{in?, out_1!\} \rrbracket \end{aligned}$$

The semantics of a Math Operation block with sample time 0 in the right of Fig. 4 is a bit more complicated. It requires the conversions between discrete and continuous signals, as illustrated by Fig. 5. Concretely, the conversion of the discrete input signal y_0 to the continuous one (D/A converter) can be specified as follows:

$$\text{converter0} \hat{=} (\vdash \lceil in_1?, y_0 \rceil \wedge G(y'_0) \ ; \ \vdash \llbracket \tilde{y}_0 = 0, \{in_1?\} \rrbracket) ^*$$

At each iteration, converter0 gets the desired data (constrained by $G(y'_0)$) from the input channel in_1 and then keeps the data unchanged for some period. Thus, the input y_0 can be treated as a continuous signal from the view of the continuous block.

On the other hand, the conversion of the continuous output signal z_1 to the discrete one (A/D converter) can be specified as follows:

$$\text{converter1} \hat{=} (\vdash \lceil out_1!, z_1 \rceil \ ; \ \vdash \exists \tilde{d} \cdot \llbracket \tilde{z}_1 = \tilde{d}, \{out_1!\} \rrbracket) ^*$$

At each iteration, converter1 puts the current value of z_1 to the output channel out_1 and then z_1 continues evolving for some period. Thus, from the view of out_1 , the signal z_1 is discrete.

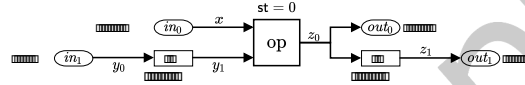


Fig. 5. Continuous Math Operation Block with D/A and A/D converters

Given that all the inputs and outputs are continuous, the computation of the continuous Math Operation block can be translated as follows:

$$\text{comp} \hat{=} A(\tilde{x}, \tilde{y}_1) \vdash \llbracket \tilde{z}_0 = \text{op}(\tilde{x}, \tilde{y}_1) \rrbracket \quad \sqcap \quad \vdash \lceil \rceil$$

It means that if the continuous inputs $\tilde{x}$ and $\tilde{y}_1$ satisfy the expected assumptions during the whole course of action then, the value of $\tilde{z}_0$ is, at all times, equal to the result of the operation of $\tilde{x}$ and $\tilde{y}_1$ in time. $\vdash \lceil \rceil$ denotes the case that simulation is not started ($ti = ti'$). In summary, the semantics of the continuous Math Operation block of Fig. 5 can be described by the following parallel composition:

$$\text{CMOP} \hat{=} \mathcal{H}_{\text{share}} (\text{converter0} \parallel_{\emptyset} \text{comp} \parallel_{\emptyset} \text{converter1})$$

where (1) $\parallel_{\emptyset}$ denotes $\parallel_{\text{SYN}_0}^{\text{HP}}$ with the merge predicate $\text{SYN}_0 \hat{=} ti_X = ti_Y \wedge M_0$. According to Definition 4.5, M_0 allows $ti_X \neq ti_Y$. However, blocks in Simulink are synchronous on time, i.e., $ti_X = ti_Y$; (2) $\mathcal{H}_{\text{share}}$ is the healthiness condition that enforces the consistency between $\tilde{y}_0$ and $\tilde{y}_1$ and between $\tilde{z}_0$ and $\tilde{z}_1$ when the diagram evolves, specified as follows:

$$\mathcal{H}_{\text{share}}(P \vdash Q) \hat{=} P \vdash Q \wedge \forall t \in [ti, ti') \cdot \tilde{y}_0(t) = \tilde{y}_1(t) \wedge \tilde{z}_0(t) = \tilde{z}_1(t)$$

As the parallel-by-merge in this paper (Section 4.2.5) does not support shared state variables, the healthiness condition above is used to enforce the parallel by shared state variables, i.e., it states that the evolutions of $\tilde{y}_0$ and $\tilde{z}_0$ should keep consistent with $\tilde{y}_1$ and $\tilde{z}_1$, respectively, because $\tilde{y}_1$ and $\tilde{z}_1$ are respective aliases of $\tilde{y}_0$ and $\tilde{z}_0$.

6.3.3 Lines between discrete Blocks. In order to synchronise communications between discrete blocks, lines between blocks serve as buffers to receive data whenever source blocks provide and target blocks require, as illustrated by the buffer in Fig. 2. A typical buffer is shown below:

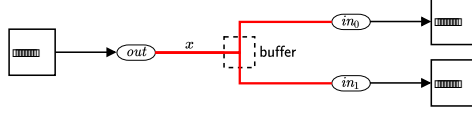


Fig. 6. A two-branch line between discrete blocks

Notice that the buffer must render the causality relation of ports (if the source block provides data and, meanwhile, a target block requires). It should guarantee that the target block can only get the latest data from the source block. In other words, if the channel operations $out?$ and $in_0!$ of the buffer occur simultaneously, $out?$ should be earlier than $in_0!$. Therefore, the buffer should be translated as follows:

$$\text{buffer} \triangleq (\text{get} \circ (\text{put}_0 \sqcap \text{put}_1 \sqcap \text{ready})^* \circ \text{ready})^* \quad (9)$$

where

$$\begin{aligned} \text{get} &\triangleq \vdash [out?, x] \\ \text{put}_0 &\triangleq \vdash [in_0!, x] \\ \text{put}_1 &\triangleq \vdash [in_1!, x] \\ \text{ready} &\triangleq \vdash [\dot{x} = 0, \{out?, in_0!, in_1!\}] \end{aligned}$$

This means that, at each iteration, the buffer first gets data from the output port of the source block (get), then puts the received data to the input ports of the target blocks whenever they require it (put_0 and put_1), and finally waits for a period of time, during which channel operations $out?$, $in_0!$ and $in_1!$ are made ready for communication (ready).

6.3.4 Composition. According to the above analysis, before translating the Simulink diagram on the left of Fig. 2, we need to add the necessary ports between some blocks, resulting in the diagram shown on the right of Fig. 2. Before execution, the system must be initialised by setting time to 0 and trace to ϵ , as follows:

$$\text{INIT} \triangleq \vdash \mathcal{H}_{\text{HP}}(ti' = 0 \wedge tr' = \epsilon)$$

Then, the composite system, the Diagram on the right of Fig. 2, can be defined in two parts. The Plant part consists of the continuous blocks Integrator0 and Integrator1. The Control part consists of the discrete blocks Bias and Gain, and a buffer between them. The Diagram can be modelled by the parallel composition of Plant and Control, as follows:

$$\text{Plant} \triangleq \mathcal{H}_{\text{share}}(\text{Integrator0} \parallel_0 \text{Integrator1}) \quad (10)$$

$$\text{Control} \triangleq \text{Bias} \parallel_{out_0} \text{buffer} \parallel_{in_1} \text{Gain} \quad (11)$$

$$\llbracket \text{Diagram} \rrbracket_{\text{HUTP}} \triangleq \text{INIT} \circ (\text{Plant} \parallel_{\{in_0, out_1\}} \text{Control}) \quad (12)$$

where $\parallel_I$ represents $\parallel_{\text{SYN}_I}^{\text{HP}}$, with $\text{SYN}_I \triangleq ti_X = ti_Y \wedge M_I$, for simplicity, and we use $\parallel_{ch}$ if $I = \{ch\}$. It can be proved that SYN_I is well-defined from Property 6. In the above case,

$$\mathcal{H}_{\text{share}}(P \vdash Q) \triangleq P \vdash Q \wedge \forall t \in [ti, ti'] \cdot \underline{q}_0(t) = \underline{q}_1(t) = \underline{q}(t) \quad (13)$$

because the continuous blocks Integrator0 and Integrator1 share a which has two aliases a_0 and a_1 .

Example 6.2 (Algebraic Loop). Consider the loop of Bias and Gain in Fig. 7. It forms an algebraic loop. According to (6), the respective prefixes of the communication traces of Bias and Gain are $\langle in_0?, d_0 \rangle \frown \tau \frown \langle out_0!, d_1 \rangle$ and $\langle in_1?, d_2 \rangle \frown \tau \frown \langle out_1!, d_3 \rangle$, where τ denotes timeless computations. According to (9), the prefixes of the communication traces of two buffers between Bias and Gain are $\langle out_0?, d_4 \rangle \frown \tau \frown \langle in_1!, d_5 \rangle$ and $\langle out_1?, d_6 \rangle \frown \tau \frown \langle in_0!, d_7 \rangle$. It can be verified that these four traces cannot be composed by $\parallel_I$, as defined in Section 3.3, which indicates that this unsound system will reduce to $\top_{\text{HD}}$, i.e., it can do nothing.

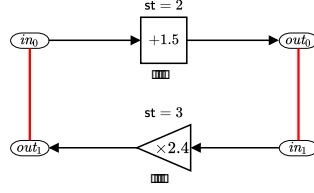


Fig. 7. A loop of discrete blocks

In addition to the above, the semantics of hierarchical diagrams, such as normal, triggered and enabled subsystems, can also be defined using HUTP. Since describing their HUTP semantics clearly needs pages of space, they are not introduced in this paper. Note that compared with the HUTP semantics of Simulink given in [10, 64], the HUTP semantics defined above is much simpler with more solid foundations as HUTP theory is well developed in this paper.

6.4 Applying HUTP to Justify the Translation from Simulink to HCSP

In the work [69], we introduced how to translate Simulink (as well as Stateflow) diagrams to the corresponding HCSP models for the purpose of verifying them using a Hybrid Hoare Logic prover in Isabelle/HOL [65, 68]. The translation is automatic and is implemented as a module `ss2hcsp` in the tool chain MARS² [63, 64] which implements the flow from modelling, simulation, verification to code generation for hybrid systems. In this subsection, we justify the correctness of the translation by `ss2hcsp` from the Simulink diagram of Fig. 2 to the corresponding HCSP model as an application of the HUTP proposed in this paper.

Definition 6.3 (Conditional Equivalence). Let $T \triangleq \{t_n\} \subseteq \mathbb{R}_{\geq 0}$ be a chain of increasing time points ($t_n < t_{n+1}$) with $\lim_{n \rightarrow +\infty} t_n = +\infty$ and S the set of state variables. Let P and Q be differential relations. Then, we say P is equivalent to Q w.r.t. T and S iff $P \wedge ti' \in T$ is equivalent to $Q \wedge ti' \in T$ on the state variables in S , denoted by $P \equiv_{T,S} Q$.

Intuitively, $P \equiv_{T,S} Q$ means that if P and Q terminate at time $ti' \in T$, then their state variables in S are consistent. In this subsection, we wish to justify $\llbracket \text{Diagram} \rrbracket_{\text{HUTP}} \equiv_{T,S} \llbracket \llbracket \text{Diagram} \rrbracket_{\text{HCSP}} \rrbracket_{\text{HUTP}}$ with some T and S , where *Diagram* is the Simulink diagram of Fig. 2.

For the Simulink Diagram of Fig. 2, `ss2hcsp` generates the following HCSP process:

$$\llbracket \text{Diagram} \rrbracket_{\text{HCSP}} \triangleq \text{Plant}_{\text{HCSP}} \parallel \text{Control}_{\text{HCSP}}$$

where

$$\begin{aligned} \text{Plant}_{\text{HCSP}} &\triangleq a := 0; x_0 := 0; \left(\langle \dot{x}_0 = a, \dot{a} = z_1, \dot{z}_1 = 0 \&\text{true} \rangle \triangleright \left[\begin{array}{l} ch_x!x_0 \rightarrow \text{skip}, \\ ch_z?z_1 \rightarrow \text{skip} \end{array} \right] \right)^* \\ \text{Control}_{\text{HCSP}} &\triangleq ti := 0; ch_x?x_1; y := x_1 + 1.5; z_0 := y \times 2.4; ch_z!z_0; \\ &\quad \left(\begin{array}{l} \text{wait}(1); \quad ti\%2 = 0 \rightarrow (ch_x?x_1; y := x_1 + 1.5); \\ \quad \quad \quad ti\%3 = 0 \rightarrow (z_0 := y \times 2.4; ch_z!z_0) \end{array} \right)^* \end{aligned}$$

The period of $\text{Control}_{\text{HCSP}}$ is the greatest common divisor (GCD) of periods of the blocks (Bias and Gain) it contains, i.e., $\text{GCD}(2, 3) = 1$, as denoted by **wait**(1). Since no shared variable is allowed between different HCSP processes in parallel, we let x_0 and x_1 be two aliases of x , and z_0 and z_1 the aliases of z , as shown by the right diagram in Fig. 2.

²<https://gitee.com/bhzhan/mars.git>

We first study the conditions under which $\text{Plant}_{\text{HCSP}}$ is consistent with Plant of (10). According to (4), $\text{Plant}_{\text{HCSP}}$ can be translated as follows:

$$\llbracket \text{Plant}_{\text{HCSP}} \rrbracket_{\text{HUTP}} \hat{=} \text{init}_a \circ \text{init}_{x_0} \circ (\text{put}_x \sqcap \text{get}_z \sqcap (\text{ode} \circ (\text{put}_x \sqcap \text{get}_z)))^*$$

where $\text{init}_a \hat{=} \vdash [a := 0]$, $\text{init}_{x_0} \hat{=} \vdash [x_0 := 0]$, $\text{put}_x \hat{=} \vdash [ch_x!, x_0]$, $\text{get}_z \hat{=} \vdash [ch_z?, z_1]$ and

$$\text{ode} \hat{=} \vdash [\ddot{x}_0 = \underline{q} \wedge \dot{q} = \underline{z}_1 \wedge \dot{z}_1 = 0, \{ch_x!, ch_z?\}] \quad (14)$$

According to (7) and (8), Plant of (10) can be unfolded as

$$\text{Plant} \hat{=} \mathcal{H}_{\text{share}} ((\text{init}_{x_0} \circ (\text{put}_x \sqcap \text{ode}_0)^*) \parallel_{\emptyset} (\text{init}_{a_0} \circ (\text{get}_z \sqcap \text{ode}_1)^*)) \quad (15)$$

where $\text{init}_{a_0} \hat{=} \vdash [a_0 := 0]$, $\text{put}_x \hat{=} \vdash [in_0!, x_0]$, $\text{get}_z \hat{=} \vdash [out_1?, z_1]$ and

$$\begin{aligned} \text{ode}_0 &\hat{=} \vdash [\ddot{x}_0 = \underline{q}_1, \{in_0!\}] \\ \text{ode}_1 &\hat{=} \vdash [\ddot{q}_0 = \underline{z}_1 \wedge \dot{z}_1 = 0, \{out_1?\}] \end{aligned}$$

Note that in_0 and out_1 are the aliases of ch_x and ch_z , respectively, so we do not distinguish put_x and get_z in $\llbracket \text{Plant}_{\text{HCSP}} \rrbracket_{\text{HUTP}}$ and Plant. Now we demonstrate Plant of (15) is equivalent to

$$\text{init}_a \circ \text{init}_{x_0} \circ (\text{get}_z \sqcap \text{put}_x \sqcap \text{ode})^* \quad (16)$$

if τ s are removed from traces, where ode is defined in (14). The traces of $\text{init}_{x_0} \circ (\text{put}_x \sqcap \text{ode}_0)^*$ and $\text{init}_{a_0} \circ (\text{get}_z \sqcap \text{ode}_1)^*$ can be modelled by the finite state machines (FSMs) in Fig. 8, where τ_x and τ_a denote the actions of init_{x_0} and init_{a_0} , respectively.



Fig. 8. FSMs of traces of $\text{init}_{x_0} \circ (\text{put}_x \sqcap \text{ode}_0)^*$ (left) and $\text{init}_{a_0} \circ (\text{get}_z \sqcap \text{ode}_1)^*$ (right)

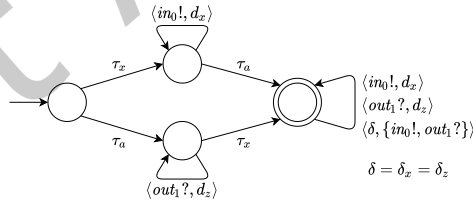


Fig. 9. The composition of FSMs in Fig. 8 by $\parallel_{\emptyset}$

According to the definition of $\parallel_{\emptyset}$, the two FSMs in Fig. 8 only synchronise on wait blocks $\langle \delta_x, \{in_0!\} \rangle$ and $\langle \delta_z, \{out_1?\} \rangle$, which indicates $\delta_x = \delta_z$. Therefore, their composition is illustrated in Fig. 9, which reflects the following result:

$$((\text{init}_{x_0} \circ \text{put}_x^* \circ \text{init}_{a_0}) \sqcap (\text{init}_{a_0} \circ \text{get}_z^* \circ \text{init}_{x_0})) \circ (\text{put}_x \sqcap \text{get}_z \sqcap \text{ode}_{01})^* \quad (17)$$

where

$$\text{ode}_{01} \hat{=} \vdash [\ddot{x}_0 = \underline{q}_1 \wedge \dot{q}_0 = \underline{z}_1 \wedge \dot{z}_1 = 0, \{in_0!, out_1?\}]$$

If we ignore τs in traces, then $(\text{init}_{x_0} \circ \text{put}_x^* \circ \text{init}_{a_0})$ and $(\text{init}_{a_0} \circ \text{get}_z^* \circ \text{init}_{x_0})$ are equivalent to $(\text{init}_{a_0} \circ \text{init}_{x_0} \circ \text{put}_x^*)$ and $(\text{init}_{a_0} \circ \text{init}_{x_0} \circ \text{get}_z^*)$, respectively. Then, (17) can be simplified:

$$\begin{aligned} & ((\text{init}_{a_0} \circ \text{init}_{x_0} \circ \text{put}_x^*) \sqcap (\text{init}_{a_0} \circ \text{init}_{x_0} \circ \text{get}_z^*)) \circ (\text{put}_x \sqcap \text{get}_z \sqcap \text{ode}_{01})^* \\ &= \text{init}_{a_0} \circ \text{init}_{x_0} \circ (\text{put}_x^* \sqcap \text{get}_z^*) \circ (\text{put}_x \sqcap \text{get}_z \sqcap \text{ode}_{01})^* \\ &= \text{init}_{a_0} \circ \text{init}_{x_0} \circ (\text{put}_x \sqcap \text{get}_z \sqcap \text{ode}_{01})^* \end{aligned} \quad (18)$$

Note that the above result should be constrained by the healthiness condition of (13) which states a_0 and a_1 are aliases of a . Therefore, the above result is equivalent to (16), which is equivalent to Plant of (15) (if τs are removed from traces).

However, (16) is not equivalent to $\llbracket \text{Plant}_{\text{HCSP}} \rrbracket_{\text{HUTP}}$, because the latter always terminates with put_x or get_z . Thus, the premise of the equivalence should be (1) ϕ_0 : ignoring τs in traces and (2) ϕ_1 : Plant terminates with put_x or get_z . In summary,

$$\text{Plant} \equiv \phi_0 \wedge \phi_1 \quad \llbracket \text{Plant}_{\text{HCSP}} \rrbracket_{\text{HUTP}} \quad (19)$$

On the other hand, we study the conditions under which $\text{Control}_{\text{HCSP}}$ is consistent with Control of (11) under some conditions. $\text{Control}_{\text{HCSP}}$ can be translated as follows:

$$\begin{aligned} \llbracket \text{Control}_{\text{HCSP}} \rrbracket_{\text{HUTP}} &\hat{=} \vdash [ti := 0] \circ \text{get}_x \circ \text{comp}_y \circ \text{comp}_z \circ \text{put}_z \circ \\ &\quad \left(\vdash [\dot{x}_1 = \dot{y} = \dot{z}_0 = 0, \emptyset]_1 \circ \right. \\ &\quad \left. \left(\text{get}_x \circ \text{comp}_y \right) \triangleleft ti\%2 = 0 \triangleright \llbracket \text{skip} \rrbracket_{\text{HUTP}} \circ \right. \\ &\quad \left. \left(\text{comp}_z \circ \text{put}_z \right) \triangleleft ti\%3 = 0 \triangleright \llbracket \text{skip} \rrbracket_{\text{HUTP}} \right)^* \end{aligned}$$

as illustrated by Fig. 10, where $\text{get}_x \hat{=} \vdash [ch_x?, x_1]$, $\text{put}_z \hat{=} \vdash [ch_z!, z_0]$, $\text{comp}_y \hat{=} \vdash [y := x_1 + 1.5]$ and $\text{comp}_z \hat{=} \vdash [z_0 := y \times 2.4]$. During the time intervals, the values of x_1 , y and z_0 keep unchanged, denoted by the side condition on the left of the time line.

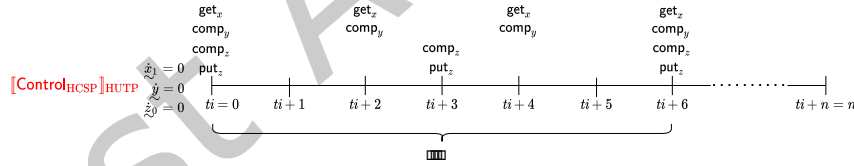


Fig. 10. Visual representation of HUTP semantics of $\text{Control}_{\text{HCSP}}$

Similarly, the behavior of Control in (11) can be illustrated by Fig. 11, where $\text{put}_y^0 \hat{=} \vdash [out_0!, y_0]$, $\text{get}_y^0 \hat{=} \vdash [out_0?, y]$, $\text{put}_y^1 \hat{=} \vdash [in_1!, y]$ and $\text{get}_y^1 \hat{=} \vdash [in_1?, y_1]$. Compared to Fig. 10, the behaviour of Control is consistent with $\llbracket \text{Control}_{\text{HCSP}} \rrbracket_{\text{HUTP}}$, except that the latter does not contain syn_y^0 (synchronous communication between put_y^0 and get_y^0) and syn_y^1 (synchronous communication between put_y^1 and get_y^1), because $\llbracket \text{Control}_{\text{HCSP}} \rrbracket_{\text{HUTP}}$ is a sequential process (and does not involve synchronous communication). This will lead to the inconsistency of the traces of $\llbracket \text{Control}_{\text{HCSP}} \rrbracket_{\text{HUTP}}$ and Control. In addition, the end time ti' of Control is not necessarily equal to the end time $ti + n = n$ of $\llbracket \text{Control}_{\text{HCSP}} \rrbracket_{\text{HUTP}}$. Thus, the premise of the consistency should be (1) ϕ_0 : ignoring trace variables and (2) ϕ_1 : $ti'\%1 = 0$. In summary,

$$\text{Control} \equiv \phi_0 \wedge \phi_1 \quad \llbracket \text{Control}_{\text{HCSP}} \rrbracket_{\text{HUTP}} \quad (20)$$

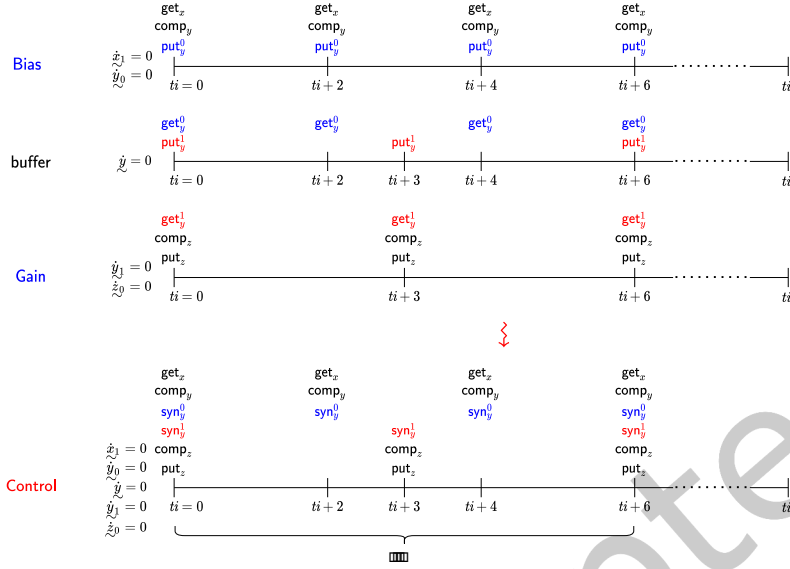


Fig. 11. Visual representation HUTP semantics of Control

Finally, according to (5), the HUTP semantics of $\llbracket \text{Diagram} \rrbracket_{\text{HCSP}}$ is

$$\begin{aligned} \llbracket \llbracket \text{Diagram} \rrbracket_{\text{HCSP}} \rrbracket_{\text{HUTP}} &= \llbracket \text{Plant}_{\text{HCSP}} \parallel \text{Control}_{\text{HCSP}} \rrbracket_{\text{HUTP}} \\ &= \llbracket \text{Plant}_{\text{HCSP}} \rrbracket_{\text{HUTP}} \parallel_{M_{\{ch_x, ch_z\}}^{\text{HP}}} \llbracket \text{Control}_{\text{HCSP}} \rrbracket_{\text{HUTP}} \end{aligned}$$

Note that the merge predicate M_I (Definition 4.5) allows the different termination time of parallel processes and it will delay the shorter one for the final time synchronisation. For example, $\llbracket \text{Plant}_{\text{HCSP}} \rrbracket_{\text{HUTP}}$ always terminate with put_x or get_z , while $\llbracket \text{Control}_{\text{HCSP}} \rrbracket_{\text{HUTP}}$ can terminate with put_z or get_x . For time synchronisation, $M_{\{ch_x, ch_z\}}$ will delay $\llbracket \text{Plant}_{\text{HCSP}} \rrbracket_{\text{HUTP}}$ for 1 time unit. In fact, the termination time difference raises when $\llbracket \text{Control}_{\text{HCSP}} \rrbracket_{\text{HUTP}}$ terminates with $ti' \% 2 \neq 0 \wedge ti' \% 3 \neq 0$. Thus, if $ti' \% 2 = 0 \vee ti' \% 3 = 0$, $\parallel_{M_{\{ch_x, ch_z\}}^{\text{HP}}}$ will degrade into $\parallel_{\{ch_x, ch_z\}}$. Consider the condition $\gamma : ti' \% 2 = 0 \vee ti' \% 3 = 0$, it implies ϕ_1 , and furthermore, it implies ϕ_1 because Control terminates with get_x or put_z under γ , which indicates Plant will terminate with put_x or get_z . In a word, $\gamma \Rightarrow \phi_1 \wedge \phi_1$. Additionally, it is known $\phi_0 \Rightarrow \phi_0$.

Combining (19), (20) and the monotonicity of $\parallel_{\{ch_x, ch_y\}}$ (Property 11),

$$\text{Plant} \parallel_{\{in_0, out_1\}} \text{Control} \equiv_{\phi_0 \wedge \gamma} \llbracket \text{Plant}_{\text{HCSP}} \rrbracket_{\text{HUTP}} \parallel_{\{ch_x, ch_z\}} \llbracket \text{Control}_{\text{HCSP}} \rrbracket_{\text{HUTP}}$$

given that in_0 and out_1 are aliases of ch_x and ch_z , respectively. In summary, we can conclude

$$\llbracket \text{Diagram} \rrbracket_{\text{HUTP}} \equiv_{T, S} \llbracket \llbracket \text{Diagram} \rrbracket_{\text{HCSP}} \rrbracket_{\text{HUTP}}$$

where $T = \{t \geq 0 \mid t \% 2 = 0 \vee t \% 3 = 0\}$ and $S = \{x_0, x_1, y, z_0, z_1, a_0, a_1\}$.

7 RELATED WORK

Hoare and He's unifying theories of programming (UTP) [28] is the first proposal towards unifying different programming paradigms and models under a common relational calculus suitable for design and verification. Another notable contribution in the same spirit is Milner's theory of bigraphs [40].

The concept of higher-order UTP dates back to [28], where higher-order variables are introduced to represent predicates of methods and procedures. Based on [28], Zeyda et al. propose a concept of higher-order UTP to set up a UTP theory for object-orientation [61, 62]. [61] builds a UTP theory for object-orientation by addressing four problems: consistency of the program model, redefinition of methods in subclasses, recursion and mutual recursion, and simplicity. [62] supports recursion, dynamic binding, and compositional method definitions all at the same time. It shows how higher-order programming can be used to reason about object-oriented programs in a compositional manner and exemplifies its use by creating an object-oriented variant of a refinement language for real-time systems. In addition, it introduces the higher-order quantification over predicates. Unlike these higher-order extensions to UTP [28, 61, 62], that presented in this paper focuses on higher-order quantification and the semantics of hybrid systems in the UTP theory.

Besides the de facto standards Simulink and Modelica, many more programming paradigms and domain-specific formalisms have been explored in the aim of modeling hybrid systems.

- Functional programming variants of data-flow concepts comprise Yampa [29], an extension of Haskell's functional reactive programming model (FRP, [46]), or Zelus [8], a functional synchronous data-flow language to model ordinary differential equations in a so-called non-standard time model. Like Modelica and Simulink, both lines of formalisms are devoted to simulation supporting type-based analysis of basic safety properties, such as causality.
- Another line of work regards earlier attempts to extend rich process algebras like the π -calculus with continuous real variables [7, 11, 30]. These extensions lead to the corresponding formulations of bi-simulations as a foundation to verification.

Our approach is to define such an algebraically expressive formalism, yet devoted to the primary purpose of verifying hybrid system models, by embedding its definition, verification and proof capabilities in higher-order logic (Isabelle/HOL).

Hybrid automata [3] are a popular formalism to model hybrid systems, but composition/decomposition of automata results in an exponential blow up which is intractable to analyze in practice. *I/O hybrid automata* [35] is an extension of hybrid automata with explicit inputs and outputs. Assume-guarantee reasoning [26] on such automata tackles composability to prevent state-space explosion. However, their applicability is in practice restricted to linear hybrid automata.

Alternatively, proof-theoretic approaches to hybrid systems verification, such as differential dynamic logic (dL, [47, 50]), action systems [4], hybrid Hoare logic together with hybrid communicating sequential processes (HHL/HCSP, [22, 31, 64, 67]) and hybrid Event-B [2, 12], have gained much attention by offering powerful abstraction, refinement and proof mechanisms to scale hybrid system verification to practical use cases. Also note an early model of CPSs in the Coq proof assistant [37], which introduces a library VERIDRONE, a foundational framework for reasoning about CPSs at all levels from high-level models to C code that implements the system. Empowered by the extension of SAT modulo-theory of real variables, implementing the seminal works on δ -decidability of [19, 20] in tools like dReal and dReach, provers allow for the tactics-guided mechanized verification of logic models of hybrid systems. [41] introduces a verification-driven engineering toolset *S ϕ nx*, which uses KeYmaera as hybrid verification tool, for modelling hybrid systems, exchanging and comparing models and proofs, and managing verification tasks.

Differential dynamic logic (dL) [47, 48] extends dynamic logic [51] to hybrid systems by allowing modalities over hybrid programs that extend classical sequential programs with ordinary differential equations to model the continuous evolution of physical artifacts. In order to deal with more complex behaviour of hybrid systems, some variants of dL were established, e.g., stochastic differential dynamic logic [48] and differential game logic [49].

In [32] Loos and Platzer propose a differential refinement logic to cope with refinement relations among different levels of abstraction for a given simple hybrid system. It remains an open problem whether the approach

can apply to more complex hybrid system. In [52] the authors investigate how to apply dL to define architecture of CPSs. However, parallelism and communication are not considered in dL and its variants. Recent component-based verification methodologies developed in dL [44, 45, 55] propose composition operators in dL to split verification of systems into more manageable pieces. [33, 34] extend them with parallel composition (true concurrency) using design patterns defined using the primitive constructs of dL, which requires the corresponding mechanization effort as tactics in KeYmaeraX.

HCSP is a hybrid extension of Hoare’s Communicating Sequential Processes [27]. It features a native parallel composition operator and communicating primitives in addition to standard constructs for hybrid systems (sequences, loops, ODEs) and a proof calculus called *hybrid Hoare logic* (HHL) [31], as well as several extensions for dealing with more complex behaviour, e.g., fault-tolerance [58] and noise [59]. Also, the notion of *approximate bisimulation* was proposed in [60], with which a refinement relation between continuous models and between continuous models and discrete models can be well coped with. However, it is difficult to use HHL/HCSP to play a semantics foundation for the design of hybrid systems using MBD and DbC, as it is not easy to unify different models and views at different level of abstraction in HHL/HCSP.

Rönnkö and Ravn, and Back *et al.* extended action system [5] to hybrid systems, called *hybrid action system* [53] and *continuous action system* [4], respectively. As conservative extensions of action systems, theoretically speaking, both hybrid action system and continuous action system could support MBD and DbC. But both of them are very weak on dealing with continuous behaviours. In particular, they lack refinement relations, different continuous models and/or discrete models, which are essential for the MBD and DbC of hybrid systems.

8 CONCLUSIONS AND FUTURE WORKS

This paper defines a conservative extension to Hoare and He’s UTP theory with higher-order quantification and provides a formal semantics for modeling and verifying hybrid systems, mixing discrete real-time processes and continuous dynamics. To this end, HUTP introduces real-time variables, denoted as functions over time, and allows the derivative of a variable to occur in a predicate, to define differential relations. This introduction allows to draw the concepts of hybrid processes and (normal) hybrid designs.

Our goal is to use the HUTP to define a formal semantics for engineering and formally verifying models such as Matlab’s Simulink/Stateflow and/or the SAE’s AADL, but also HCSP, and support the workflow of these environment with formal verification and certification functionalities inherited from its implementation in higher-order logic (Isabelle/HOL). We intend to additionally use HUTP to justify the correctness of the translation between these models and HCSP by translation validation as an application of HUTP.

Future Works. Starting from the definition of normal hybrid design contracts presented in this paper, we are seeking principled abstractions to support modular analysis and compositional verification of hybrid system models. Our initial intuition was to start from the definition of abstract hybrid processes in Section 4.5 and define a theory of contracts using Abadi and Lamport’s “composing specifications” [1] or Benveniste’s meta-theory of contracts [6]. Indeed, consider a normal hybrid design $HC \vdash HP$ and $\mathcal{H}_{HP}^A$ -healthy abstract hybrid processes C and P such that $HC \sqsubseteq C$ and $P \sqsubseteq HP$. The abstract design $C \vdash P$ naturally forms a contractual property of $HC \vdash HP$, in the sense of Abadi and Lamport or Benveniste.

However, related works in the UTP [15] show that the algebraic elaboration of such a contract theory requires non-constructive definitions such as weakest liberal preconditions and weakest rely, which prevent the structural decomposition and abstraction of parallel composition and communications from the abstracted hybrid designs (unless one uses, e.g., Lamport’s encoding of hand-shake communications using shared variables).

As a result, our aim will instead be to elaborate a dependently-typed session calculus [36] in the UTP and equip it with abstract model checking capabilities [56].

ACKNOWLEDGMENTS

This research is partly supported by NSFC under grant No. 61625206, 62192732, 62032024, 61972385 and 61732001, and is also partly funded by Inria's joint research project CONVEX. The authors would like to thank the editors and anonymous reviewers, whose criticisms and suggestions did improve the presentation of our work very much.

REFERENCES

- [1] M. Abadi and L. Lamport. 1993. Composing Specifications. *ACM Transactions on Programming Languages and Systems (TOPLAS)* 15, 1 (January 1993), 73–132.
- [2] J.-R. Abrial, W. Su, and H. Zhu. 2012. Formalizing Hybrid Systems with Event-B. In *3rd International Conference on Abstract State Machines, Alloy, B, VDM, and Z (ABZ) (Lecture Notes in Computer Science, Vol. 7316)*. Springer, Pisa, Italy, 178–193.
- [3] R. Alur, C. Courcoubetis, T. A. Henzinger, and P.-H. Ho. 1993. Hybrid Automata: An Algorithmic Approach to the Specification and Verification of Hybrid Systems. In *Hybrid Systems*. Springer, 209–229.
- [4] R.-J. Back, L. Petre, and I. Porres. 2000. Generalizing Action Systems to Hybrid Systems. In *6th International Symposium on Formal Techniques in Real-Time and Fault-Tolerant Systems (FTRIFT) (Lecture Notes in Computer Science, Vol. 1926)*. Pune, India, 202–213.
- [5] R.-J. Back and J. v. Wright. 1998. *Refinement Calculus—A Systematic Introduction*. Springer.
- [6] A. Benveniste, B. Caillaud, D. Nickovic, R. Passerone, J.-B. Raclet, P. Reinkemeier, A.-L. Sangiovanni-Vincentelli, W. Damm, T.-A. Henzinger, and K. G. Larsen. 2018. Contracts for System Design. *Foundations and Trends in Electronic Design Automation* 12, 2–3 (2018), 124–400.
- [7] J. A. Bergstra and C. A. Middelburg. 2005. Process Algebra for Hybrid Systems. *Theoretical Computer Science* 335, 2–3 (May 2005), 215–280.
- [8] T. Bourke and M. Pouzet. 2013. Zélus: A Synchronous Language with ODEs. In *Proceedings of the 16th International Conference on Hybrid Systems: Computation and Control (HSCC)*. ACM, Philadelphia, PA, USA, 113–118.
- [9] A. Butterfield, P. Gancarski, and J. Woodcock. 2009. State Visibility and Communication in Unifying Theories of Programming. In *3rd IEEE International Symposium on Theoretical Aspects of Software Engineering*. IEEE Computer Society, Tianjin, China, 47–54.
- [10] M. Chen, A. P. Ravn, S. Wang, M. Yang, and N. Zhan. 2016. A Two-Way Path between Formal and Informal Design of Embedded Systems. In *6th International Symposium on Unifying Theories of Programming (UTP) (Lecture Notes in Computer Science, Vol. 10134)*. Springer, Reykjavik, Iceland, 65–92.
- [11] P. J. L. Cuijpers and M. A. Reniers. 2005. Hybrid Process Algebra. *Journal of Logic and Algebraic Programming* 62, 2 (February 2005), 191–245.
- [12] G. Dupont, Y. Ait-Ameur, N. K. Singh, and M. Pantel. 2021. Event-B Hybridization: A Proof and Refinement-based Framework for Modelling Hybrid Systems. *ACM Transactions on Embedded Computing Systems* 20, 4 (June 2021), 35:1–35:37.
- [13] S. Foster. 2019. Hybrid Relations in Isabelle/UTP. In *7th International Symposium on Unifying Theories of Programming (UTP), Dedicated to Tony Hoare on the Occasion of His 85th Birthday (Lecture Notes in Computer Science, Vol. 11885)*. Springer, Porto, Portugal, 130–153.
- [14] S. Foster and J. Baxter. 2020. Automated Algebraic Reasoning for Collections and Local Variables with Lenses. In *18th International Conference on Relational and Algebraic Methods in Computer Science (RAMICS) (Lecture Notes in Computer Science, Vol. 12062)*. Springer, Palaiseau, France, 100–116.
- [15] S. Foster, A. Cavalcanti, S. Canham, J. Woodcock, and F. Zeyda. 2020. Unifying Theories of Reactive Design Contracts. *Theoretical Computer Science* 802 (January 2020), 105–140.
- [16] S. Foster, A. Cavalcanti, J. Woodcock, and F. Zeyda. 2018. Unifying Theories of Time with Generalised Reactive Processes. *Inform. Process. Lett.* 135 (July 2018), 47–52.
- [17] S. Foster, B. Thiele, A. Cavalcanti, and J. Woodcock. 2016. Towards a UTP Semantics for Modelica. In *6th International Symposium on Unifying Theories of Programming (UTP) (Lecture Notes in Computer Science, Vol. 10134)*. Springer, Reykjavik, Iceland, 44–64.
- [18] D. D. Gajski, S. Abdi, A. Gerstlauer, and G. Schirner. 2009. *Embedded System Design: Modeling, Synthesis, Verification*. Springer-Verlag.
- [19] S. Gao, J. Avigad, and E. M. Clarke. 2012. Delta-Decidability over the Reals. In *Proceedings of the 27th Annual IEEE Symposium on Logic in Computer Science (LICS)*. IEEE Computer Society, Dubrovnik, Croatia, 305–314.
- [20] S. Gao, S. Kong, and E. M. Clarke. 2013. dReal: An SMT Solver for Nonlinear Theories over the Reals. In *24th International Conference on Automated Deduction on Automated Deduction (CADE) (Lecture Notes in Computer Science, Vol. 7898)*. Springer, Lake Placid, NY, USA, 208–214.
- [21] W. Guttman and B. Möller. 2010. Normal Design Algebra. *Journal of Logic and Algebraic Programming* 79, 2 (February 2010), 144–173.
- [22] J. He. 1994. From CSP to Hybrid Systems. In *A Classical Mind: Essays in Honour of C. A. R. Hoare*. Prentice Hall International (UK) Ltd., 171–189.

- [23] J. He and Q. Li. 2017. A Hybrid Relational Modelling Language. In *Concurrency, Security, and Puzzles - Essays Dedicated to Andrew William Roscoe on the Occasion of His 60th Birthday*. Number 10160 in Lecture Notes in Computer Science. Springer, 124–143.
- [24] J. He, X. Li, and Z. Liu. 2006. A Theory of Reactive Components. *Electronic Notes in Theoretical Computer Science* 160 (August 2006), 173–195.
- [25] E. C. R. Hehner. 1993. *A Practical Theory of Programming*. Springer.
- [26] T. A. Henzinger, M. Minea, and V. Prabhu. 2001. Assume-Guarantee Reasoning for Hierarchical Hybrid Systems. In *4th International Workshop on Hybrid Systems: Computation and Control (HSCC) (Lecture Notes in Computer Science, Vol. 2034)*. Springer, Rome, Italy, 275–290.
- [27] C. A. R. Hoare. 1978. Communicating Sequential Processes. *Communications of the ACM (CACM)* 21, 8 (August 1978), 666–677.
- [28] C. A. R. Hoare and J. He. 1998. *Unifying Theories of Programming*. Prentice Hall, Englewood Cliffs.
- [29] P. Hudak, A. Courtney, H. Nilsson, and J. Peterson. 2002. Arrows, Robots, and Functional Reactive Programming. In *4th International School on Advanced Functional Programming (AFP) (Lecture Notes in Computer Science, Vol. 2638)*. Springer, Oxford, UK, 159–187.
- [30] R. Lanotte and M. Merro. 2017. A Calculus of Cyber-Physical Systems. In *11th International Conference on Language and Automata Theory and Applications (LATA) (Lecture Notes in Computer Science, Vol. 10168)*. Springer International Publishing, Umeå, Sweden, 115–127.
- [31] J. Liu, J. Lv, Z. Quan, N. Zhan, H. Zhao, C. Zhou, and L. Zou. 2010. A Calculus for Hybrid CSP. In *8th Asian Symposium on Programming Languages and Systems (APLAS) (Lecture Notes in Computer Science, Vol. 6461)*. Springer, Shanghai, China, 1–15.
- [32] S. M. Loos and A. Platzer. 2016. Differential Refinement Logic. In *Proceedings of the 31st Annual ACM/IEEE Symposium on Logic in Computer Science (LICS)*. ACM, New York, NY, USA, 505–514.
- [33] S. Lunel, B. Boyer, and J.-P. Talpin. 2017. Compositional Proofs in Differential Dynamic Logic dL. In *17th International Conference on Application of Concurrency to System Design (ACSD)*. IEEE Computer Society, Zaragoza, Spain, 19–28.
- [34] S. Lunel, S. Mitsch, B. Boyer, and J.-P. Talpin. 2019. Parallel Composition and Modular Verification of Computer Controlled Systems in Differential Dynamic Logic. In *3rd World Congress on Formal Methods (FM) (Lecture Notes in Computer Science, Vol. 11800)*. Springer, Porto, Portugal, 354–370.
- [35] N. A. Lynch, R. Segala, and F. W. Vaandrager. 2003. Hybrid I/O Automata. *Information and Computation* 185, 1 (August 2003), 105–157.
- [36] R. Majumdar, N. Yoshida, and D. Zufferey. 2020. Multiparty Motion Coordination: From Choreographies to Robotics Programs. *Proceedings of the ACM on Programming Languages* 4, OOPSLA (November 2020), 134:1–134:30.
- [37] G. Malecha, D. Ricketts, M. M. Alvarez, and S. Lerner. 2016. Towards Foundational Verification of Cyber-Physical Systems. In *Science of Security for Cyber-Physical Systems Workshop*. IEEE Computer Society, Vienna, Austria, 1–5.
- [38] O. Maler, Z. Manna, and A. Pnueli. 1991. From Timed to Hybrid Systems. In *REX Workshop on Real-Time: Theory in Practice (Lecture Notes in Computer Science, Vol. 600)*. Springer, Mook, The Netherlands, 447–484.
- [39] MathWorks. 2013. *Simulink® User's Guide*. http://www.mathworks.com/help/pdf_doc/simulink/sl_using.pdf.
- [40] R. Milner. 2006. Pure Bigraphs: Structure and Dynamics. *Information and Computation* 204, 1 (January 2006), 60–122.
- [41] S. Mitsch. 2013. *Modeling and Analyzing Hybrid Systems with Sphinx*. Carnegie Mellon University / Johannes Kepler University.
- [42] Modelica. 2014. *Modelica® – A Unified Object-Oriented Language for Systems Modeling (Language Specification)* (version 3.3 revision 1 ed.). Modelica Association.
- [43] B. Möller. 2006. The Linear Algebra of UTP. In *Proceedings of the 8th International Conference on Mathematics of Program Construction (MPC) (Lecture Notes in Computer Science, Vol. 4014)*. Springer, Kuressaare, Estonia, 338–358.
- [44] A. Müller, S. Mitsch, W. Retschitzegger, W. Schwinger, and A. Platzer. 2016. A Component-Based Approach to Hybrid Systems Safety Verification. In *12th International Conference on Integrated Formal Methods (IFM) (Lecture Notes in Computer Science, Vol. 9681)*. Springer, Reykjavik, Iceland, 441–456.
- [45] A. Müller, S. Mitsch, W. Retschitzegger, W. Schwinger, and A. Platzer. 2018. Tactical Contract Composition for Hybrid System Component Verification. *International Journal on Software Tools for Technology Transfer (STTT)* 20, 6 (November 2018), 615–643.
- [46] J. Peterson, Z. Wan, P. Hudak, and H. Nilsson. 2001. *Yale FRP User's Manual*. Department of Computer Science, Yale University. <http://www.haskell.org/frp/manual.html>.
- [47] A. Platzer. 2008. Differential Dynamic Logic for Hybrid Systems. *Journal of Automated Reasoning* 41, 2 (August 2008), 143–189.
- [48] A. Platzer. 2010. *Logical Analysis of Hybrid Systems*. Springer.
- [49] A. Platzer. 2015. Differential Game Logic. *ACM Transactions on Computational Logic* 17, 1 (December 2015), 1:1–1:51.
- [50] A. Platzer. 2018. *Logical Foundations of Cyber-Physical Systems*. Springer.
- [51] V. R. Pratt. 1976. Semantical Considerations on Floyd-Hoare Logic. In *17th Annual Symposium on Foundations of Computer Science (FOCS)*. IEEE Computer Society, Houston, Texas, USA, 109–121.
- [52] A. Rajhans, A. Bhawe, I. Ruchkin, B. H. Krogh, D. Garlan, A. Platzer, and B. R. Schmerl. 2014. Supporting Heterogeneity in Cyber-Physical Systems Architectures. *IEEE Trans. Automat. Control* 59, 12 (December 2014), 3178–3193.
- [53] M. Rönkkö and A. P. Ravn. 1997. Action Systems with Continuous Behaviour. In *Proceedings of the 5th International Workshop on Hybrid Systems (Hybrid Systems V) (Lecture Notes in Computer Science, Vol. 1567)*. Springer, Notre Dame, IN, USA, 304–323.
- [54] A. W. Roscoe. 1997. *The Theory and Practice of Concurrency*. Prentice Hall.

- [55] I. Ruchkin, B. Schmerl, and D. Garlan. 2015. Architecture Abstractions for Hybrid Programs. In *18th International ACM SIGSOFT Symposium on Component-Based Software Engineering (CBSE)*. ACM, Montreal, QC, Canada, 65–74.
- [56] A. Scalas, N. Yoshida, and E. Benussi. 2019. Verifying Message-Passing Programs with Dependent Behavioural Types. In *Proceedings of the 40th ACM SIGPLAN Conference on Programming Language Design and Implementation (PLDI)*. Phoenix, AZ, USA, 502–516.
- [57] A. Tarski. 1955. A Lattice-Theoretical Fixpoint Theorem and Its Applications. *Pacific J. Math.* 5, 2 (1955), 285–309.
- [58] S. Wang, F. Nielson, H. R. Nielson, and N. Zhan. 2017. Modelling and Verifying Communication Failure of Hybrid Systems in HCSP. *Comput. J.* 60, 8 (August 2017), 1111–1130.
- [59] S. Wang, N. Zhan, and L. Zhang. 2017. A Compositional Modelling and Verification Framework for Stochastic Hybrid Systems. *Formal Aspects of Computing* 29, 4 (July 2017), 751–775.
- [60] G. Yan, L. Jiao, S. Wang, L. Wang, and N. Zhan. 2020. Automatically Generating SystemC Code from HCSP Formal Models. *ACM Transactions on Software Engineering and Methodology* 29, 1 (January 2020), 4:1–4:39.
- [61] F. Zeyda and A. Cavalcanti. 2012. Higher-Order UTP for a Theory of Methods. In *4th International Symposium on Unifying Theories of Programming (UTP) (Lecture Notes in Computer Science, Vol. 7681)*. Springer, Paris, France, 204–223.
- [62] F. Zeyda, T. Santos, A. Cavalcanti, and A. Sampaio. 2014. A Modular Theory of Object Orientation in Higher-Order UTP. In *19th International Symposium on Formal Methods (FM) (Lecture Notes in Computer Science, Vol. 8442)*. Springer, Singapore, 627–642.
- [63] Bohua Zhan, Bin Gu, Xiong Xu, Xiangyu Jin, Shuling Wang, Bai Xue, Xiaofeng Li, Yao Chen, Mengfei Yang, and Naijun Zhan. 2021. Brief Industry Paper: Modeling and Verification of Descent Guidance Control of Mars Lander. In *RTAS 2021*. 457–460.
- [64] N. Zhan, S. Wang, and H. Zhao. 2017. *Formal Verification of Simulink/Stateflow Diagrams—A Deductive Approach*. Springer.
- [65] H. Zhao, M. Yang, N. Zhan, B. Gu, L. Zou, and Y. Chen. 2014. Formal Verification of a Descent Guidance Control Program of a Lunar Lander. In *19th International Symposium on Formal Methods (FM) (Lecture Notes in Computer Science, Vol. 8442)*. Springer, Singapore, 733–748.
- [66] C. Zhou, C. A. R. Hoare, and A. P. Ravn. 1991. A Calculus of Durations. *Inform. Process. Lett.* 40, 5 (December 13 1991), 269–276.
- [67] C. Zhou, J. Wang, and A. P. Ravn. 1995. A Formal Description of Hybrid Systems. In *Proceedings of the DIMACS/SYCON Workshop on Verification and Control of Hybrid Systems, Hybrid Systems III: Verification and Control (Lecture Notes in Computer Science, Vol. 1066)*. Springer, Rutgers University, New Brunswick, NJ, USA, 511–530.
- [68] L. Zou, N. Zhan, S. Wang, and M. Fränzle. 2015. Formal Verification of Simulink/Stateflow Diagrams. In *13th International Symposium on Automated Technology for Verification and Analysis (ATVA) (Lecture Notes in Computer Science, Vol. 9364)*. Springer, Shanghai, China, 464–481.
- [69] L. Zou, N. Zhan, S. Wang, M. Fränzle, and S. Qin. 2013. Verifying Simulink Diagrams Via A Hybrid Hoare Logic Prover. In *International Conference on Embedded Software (EMSOFT)*. IEEE, Montreal, QC, Canada, 9:1–9:10.

A PROOFS

A.1 Proofs for Timed Trace Model

PROPERTY 1 (ASSOCIATIVITY). *Let I_0, I_1 and I_2 be the respective common channel sets between timed traces tt_0 and tt_1 , tt_1 and tt_2 , and tt_2 and tt_0 , then $(tt_0 \parallel_{I_0} tt_1) \parallel_{I_1 \uplus I_2} tt_2 = tt_0 \parallel_{I_0 \uplus I_2} (tt_1 \parallel_{I_1} tt_2)$.*

PROOF. We prove by induction on the length of traces. Since the traces we consider could be infinite, the proof is coinductive. We start from the cases that $|tt_0|, |tt_1|, |tt_2| \leq 1$, such as $tt_0 = tt_1 = tt_2 = \epsilon$, then by the fact that I_0, I_1 and I_2 are disjoint, we can get

$$(tt_0 \parallel_{I_0} tt_1) \parallel_{I_1 \uplus I_2} tt_2 = tt_0 \parallel_{I_0 \uplus I_2} (tt_1 \parallel_{I_1} tt_2)$$

which indicates the associativity of the parallel composition of timed traces. $\square$

A.2 Proofs for Hybrid Processes

THEOREM 4.2 (IDEMPOTENCE AND MONOTONICITY). $\mathcal{H}_{HP}$ is idempotent and monotonic.

PROOF. First, $\mathcal{H}_{HP}$ is monotonic because it is constructed by monotonic operators. Then, we prove it is also idempotent. It can be checked that $\mathcal{H}_0, \mathcal{H}_1, \mathcal{H}_2, \mathcal{H}_3$ and $\mathcal{H}_4$ themselves are idempotent and $\mathcal{H}_0$ and $\mathcal{H}_4$ are

commutative with their partners. Therefore,

$$\begin{aligned}
 \mathcal{H}_{\text{HP}} \circ \mathcal{H}_{\text{HP}} &= (\mathcal{H}_0 \circ \mathcal{H}_1 \circ \mathcal{H}_2 \circ \mathcal{H}_3 \circ \mathcal{H}_4) \circ (\mathcal{H}_0 \circ \mathcal{H}_1 \circ \mathcal{H}_2 \circ \mathcal{H}_3 \circ \mathcal{H}_4) \\
 &= (\mathcal{H}_1 \circ \mathcal{H}_2 \circ \mathcal{H}_3) \circ (\mathcal{H}_0 \circ \mathcal{H}_0 \circ \mathcal{H}_1 \circ \mathcal{H}_2 \circ \mathcal{H}_3 \circ \mathcal{H}_4 \circ \mathcal{H}_4) \\
 &= (\mathcal{H}_1 \circ \mathcal{H}_2 \circ \mathcal{H}_3) \circ (\mathcal{H}_0 \circ \mathcal{H}_1 \circ \mathcal{H}_2 \circ \mathcal{H}_3 \circ \mathcal{H}_4)
 \end{aligned} \tag{21}$$

It is not difficult to check that $\mathcal{H}_3$ is commutative with $\mathcal{H}_0$ and $\mathcal{H}_1$, and we prove it is also commutative with $\mathcal{H}_2$. For short, let $\text{inf} \hat{=} (ti = +\infty \vee |tr| = +\infty)$ and $\text{inf}' \hat{=} (ti' = +\infty \vee |tr'| = +\infty)$ correspondingly. Let $\mathfrak{h} \hat{=} (ti = ti' \wedge tr = tr')$. Then,

$$\mathcal{H}_2 \circ \mathcal{H}_3(X) = \mathfrak{h} \triangleleft \text{inf} \triangleright ((\exists s' \cdot X) \triangleleft \text{inf}' \triangleright X)$$

and

$$\begin{aligned}
 \mathcal{H}_3 \circ \mathcal{H}_2(X) &= (\exists s' \cdot (\mathfrak{h} \triangleleft \text{inf} \triangleright X)) \triangleleft \text{inf}' \triangleright (\mathfrak{h} \triangleleft \text{inf} \triangleright X) \\
 &= (\mathfrak{h} \triangleleft \text{inf} \triangleright \exists s' \cdot X) \triangleleft \text{inf}' \triangleright (\mathfrak{h} \triangleleft \text{inf} \triangleright X) \\
 &= \mathfrak{h} \triangleleft \text{inf} \triangleright (\exists s' \cdot X \triangleleft \text{inf}' \triangleright X)
 \end{aligned}$$

Therefore, $\mathcal{H}_2 \circ \mathcal{H}_3 = \mathcal{H}_3 \circ \mathcal{H}_2$, and we can continue the equation (21):

$$\begin{aligned}
 \mathcal{H}_{\text{HP}} \circ \mathcal{H}_{\text{HP}} &= (\mathcal{H}_1 \circ \mathcal{H}_2 \circ \mathcal{H}_3) \circ (\mathcal{H}_0 \circ \mathcal{H}_1 \circ \mathcal{H}_2 \circ \mathcal{H}_3 \circ \mathcal{H}_4) \\
 &= \mathcal{H}_0 \circ \mathcal{H}_1 \circ \mathcal{H}_2 \circ \mathcal{H}_1 \circ \mathcal{H}_2 \circ \mathcal{H}_3 \circ \mathcal{H}_3 \circ \mathcal{H}_4 \\
 &= \mathcal{H}_0 \circ \mathcal{H}_1 \circ \mathcal{H}_2 \circ \mathcal{H}_1 \circ \mathcal{H}_2 \circ \mathcal{H}_3 \circ \mathcal{H}_4
 \end{aligned}$$

Lastly, we prove the composition $\mathcal{H}_1 \circ \mathcal{H}_2$ is idempotent. Concretely,

$$\mathcal{H}_1 \circ \mathcal{H}_2(X) = \mathcal{H}_1(\mathfrak{h} \triangleleft \text{inf} \triangleright \mathcal{H}_1(X))$$

Then,

$$\begin{aligned}
 \mathcal{H}_1 \circ \mathcal{H}_2 \circ \mathcal{H}_1 \circ \mathcal{H}_2(X) &= \mathcal{H}_1(\mathfrak{h} \triangleleft \text{inf} \triangleright \mathcal{H}_1(\mathcal{H}_1(\mathfrak{h} \triangleleft \text{inf} \triangleright \mathcal{H}_1(X))) \\
 &= \mathcal{H}_1(\mathfrak{h} \triangleleft \text{inf} \triangleright (\mathcal{H}_1(\mathfrak{h} \triangleleft \text{inf} \triangleright \mathcal{H}_1(X))) \\
 &= \mathcal{H}_1(\mathfrak{h} \triangleleft \text{inf} \triangleright \mathcal{H}_1(X)) = \mathcal{H}_1 \circ \mathcal{H}_2(X)
 \end{aligned}$$

In summary, $\mathcal{H}_{\text{HP}} \circ \mathcal{H}_{\text{HP}} = \mathcal{H}_0 \circ \mathcal{H}_1 \circ \mathcal{H}_2 \circ \mathcal{H}_3 \circ \mathcal{H}_4 = \mathcal{H}_{\text{HP}}$, i.e., it is idempotent. $\square$

PROPERTY 2. If HP_0 and HP_1 are $\mathcal{H}_{\text{HP}}$ -healthy, $\text{HP}_0 \sqcap \text{HP}_1$ and $\text{HP}_0 \sqcup \text{HP}_1$ are $\mathcal{H}_{\text{HP}}$ -healthy.

PROOF. It can be checked that $\mathcal{H}_4$ is commutative with its partners, thus

$$\begin{aligned}
 \mathcal{H}_{\text{HP}}(X) &= \mathcal{H}_0 \circ \mathcal{H}_1 \circ \mathcal{H}_2 \circ \mathcal{H}_3 \circ \mathcal{H}_4(X) \\
 &= \mathcal{H}_0 \circ \mathcal{H}_1 \circ \mathcal{H}_4 \circ \mathcal{H}_2 \circ \mathcal{H}_3(X) \\
 &= \mathcal{H}_{014}(\mathfrak{h} \triangleleft \text{inf} \triangleright ((\exists s' \cdot X) \triangleleft \text{inf}' \triangleright X))
 \end{aligned} \tag{22}$$

where $\mathcal{H}_{014} \hat{=} \mathcal{H}_0 \circ \mathcal{H}_1 \circ \mathcal{H}_4$. Then, $\mathcal{H}_{\text{HP}}(X) \sqcap \mathcal{H}_{\text{HP}}(Y)$ and $\mathcal{H}_{\text{HP}}(X) \sqcup \mathcal{H}_{\text{HP}}(Y)$ are

$$\begin{aligned}
 \mathcal{H}_{014}(\mathfrak{h} \triangleleft \text{inf} \triangleright ((\exists s' \cdot X \vee Y) \triangleleft \text{inf}' \triangleright (X \vee Y))) &= \mathcal{H}_{\text{HP}}(X \vee Y) \\
 \mathcal{H}_{014}(\mathfrak{h} \triangleleft \text{inf} \triangleright ((\exists s' \cdot X \wedge Y) \triangleleft \text{inf}' \triangleright (X \wedge Y))) &= \mathcal{H}_{\text{HP}}(X \wedge Y)
 \end{aligned}$$

Therefore, $\sqcap$ and $\sqcup$ are $\mathcal{H}_{\text{HP}}$ -preserving. $\square$

LEMMA A.1. $\mathcal{H}_{014}(X) \circ \mathcal{H}_{014}(Y) = \mathcal{H}_{014}(X \circ Y)$, where $\mathcal{H}_{014} \hat{=} \mathcal{H}_0 \circ \mathcal{H}_1 \circ \mathcal{H}_4$.

PROOF. The computation is performed as follows:

$$\begin{aligned}
\mathcal{H}_{014}(X) \circ \mathcal{H}_{014}(Y) &= X \wedge \text{lim}(tr) = ti \leq ti' = \text{lim}(tr') \wedge tr \leq tr' \wedge RC \wedge SD \circ \\
&\quad Y \wedge \text{lim}(tr) = ti \leq ti' = \text{lim}(tr') \wedge tr \leq tr' \wedge RC \wedge SD \\
&= \exists ti_*, s_*, tr_* \cdot X[ti_*, s_*, tr_*/ti', s', tr'] \wedge Y[ti_*, s_*, tr_*/ti, s, tr] \\
&\quad \wedge \text{lim}(tr) = ti \leq ti_* = \text{lim}(tr_*) \leq ti' = \text{lim}(tr') \wedge tr \leq tr_* \leq tr' \\
&\quad \wedge RC[ti_*/ti'] \wedge RC[ti_*/ti] \wedge SD[ti_*/ti'] \wedge SD[ti_*/ti] \\
&= (X \circ Y) \wedge \text{lim}(tr) = ti \leq ti' = \text{lim}(tr') \wedge tr \leq tr' \wedge RC \wedge SD \\
&= \mathcal{H}_{014}(X \circ Y)
\end{aligned}$$

□

PROPERTY 3. If HP_0 and HP_1 are $\mathcal{H}_{\text{HP}}$ -healthy, then so is $\text{HP}_0 \circ \text{HP}_1$.

PROOF. As stated in the proof of Property 2 (see (22)), a hybrid process $\mathcal{H}_{\text{HP}}(X)$ can be represented by $\mathcal{H}_{014}(\text{h} \triangleleft \text{inf} \triangleright (\exists s' \cdot X \triangleleft \text{inf}' \triangleright X))$. This form can be transformed into a matrix:

$$\begin{pmatrix} \overset{\neg \text{inf}'}{\mathcal{H}_{014}(X \wedge \neg \text{inf} \wedge \neg \text{inf}')} & \overset{\text{inf}'}{\mathcal{H}_{014}(\exists s' \cdot X \wedge \neg \text{inf} \wedge \text{inf}')} \\ \text{false} & \mathcal{H}_{014}(\text{h} \wedge \text{inf} \wedge \text{inf}') \end{pmatrix} \begin{matrix} \neg \text{inf} \\ \text{inf} \end{matrix}$$

Imagine $\circ$ and $\vee$ as matrix multiplication and addition, respectively, then $\mathcal{H}_{\text{HP}}(X) \circ \mathcal{H}_{\text{HP}}(Y)$ is

$$\begin{pmatrix} \mathcal{H}_{014}(X \wedge \neg \text{inf} \wedge \neg \text{inf}') \circ & \mathcal{H}_{014}(X \wedge \neg \text{inf} \wedge \neg \text{inf}') \circ \mathcal{H}_{014}(\exists s' \cdot Y \wedge \neg \text{inf} \wedge \text{inf}') \\ \mathcal{H}_{014}(Y \wedge \neg \text{inf} \wedge \neg \text{inf}') & \vee \mathcal{H}_{014}(\exists s' \cdot X \wedge \neg \text{inf} \wedge \text{inf}') \circ \mathcal{H}_{014}(\text{h} \wedge \text{inf} \wedge \text{inf}') \\ \text{false} & \mathcal{H}_{014}(\text{h} \wedge \text{inf} \wedge \text{inf}') \end{pmatrix}$$

By Lemma A.1, it can be simplified to

$$\begin{pmatrix} \mathcal{H}_{014}((X \circ Y) \wedge \neg \text{inf} \wedge \neg \text{inf}') & \mathcal{H}_{014}(\exists s' \cdot (X \circ Y \vee X) \wedge \neg \text{inf} \wedge \text{inf}') \\ \text{false} & \mathcal{H}_{014}(\text{h} \wedge \text{inf} \wedge \text{inf}') \end{pmatrix}$$

which is equivalent to

$$\mathcal{H}_{014}(\text{h} \triangleleft \text{inf} \triangleright (\exists s' \cdot (X \circ Y \vee X) \triangleleft \text{inf}' \triangleright (X \circ Y))) \quad (23)$$

where $\circ$ takes precedence over $\vee$. The predicate $\text{h} \triangleleft \text{inf} \triangleright (\exists s' \cdot (X \circ Y \vee X) \triangleleft \text{inf}' \triangleright (X \circ Y))$ is $\mathcal{H}_2$ -healthy, and we prove it is also $\mathcal{H}_3$ -healthy.

$$\begin{aligned}
\mathcal{H}_3(\text{h} \triangleleft \text{inf} \triangleright (\exists s' \cdot (X \circ Y \vee X) \triangleleft \text{inf}' \triangleright (X \circ Y))) &= \\
&(\text{h} \triangleleft \text{inf} \triangleright (\exists s' \cdot (X \circ Y \vee X) \triangleleft \text{inf}' \triangleright \exists s' \cdot (X \circ Y))) \\
&\triangleleft \text{inf}' \triangleright (\text{h} \triangleleft \text{inf} \triangleright (\exists s' \cdot (X \circ Y \vee X) \triangleleft \text{inf}' \triangleright (X \circ Y)))
\end{aligned}$$

which can be simplified to $(\text{h} \triangleleft \text{inf} \triangleright (\exists s' \cdot (X \circ Y \vee X))) \triangleleft \text{inf}' \triangleright (\text{h} \triangleleft \text{inf} \triangleright (X \circ Y))$, equivalent to $\text{h} \triangleleft \text{inf} \triangleright (\exists s' \cdot (X \circ Y \vee X) \triangleleft \text{inf}' \triangleright (X \circ Y))$ itself. Therefore, (23) is $\mathcal{H}_0 \circ \mathcal{H}_1 \circ \mathcal{H}_4 \circ \mathcal{H}_2 \circ \mathcal{H}_3$ -healthy, i.e., $\mathcal{H}_{\text{HP}}(X) \circ \mathcal{H}_{\text{HP}}(Y)$ is $\mathcal{H}_{\text{HP}}$ -healthy. □

PROPERTY 4. The healthiness condition $\mathcal{H}_3$ is equivalent to

$$X = X \circ \text{skip}$$

where $\text{skip} \triangleq \mathcal{H}_2(ti = ti' \wedge tr = tr' \wedge s = s')$.

PROOF. With the aid of matrix representation, skip can be transformed into a matrix:

$$\begin{pmatrix} \neg \text{inf}' & \text{inf}' \\ \text{false} & \text{false} \end{pmatrix} \begin{pmatrix} \neg \text{inf} & \text{false} \\ \text{inf} & \text{false} \end{pmatrix} \begin{pmatrix} \neg \text{inf} & \text{false} \\ \text{inf} & \text{false} \end{pmatrix}$$

Then, $X \circ \text{skip}$ can be computed as

$$\begin{pmatrix} X \wedge \neg \text{inf} \wedge \neg \text{inf}' & X \wedge \neg \text{inf} \wedge \text{inf}' \\ X \wedge \text{inf} \wedge \neg \text{inf}' & X \wedge \text{inf} \wedge \text{inf}' \end{pmatrix} \begin{pmatrix} \neg \text{inf} & \text{false} \\ \text{inf} & \text{false} \end{pmatrix}$$

which can be simplified to

$$\begin{pmatrix} X \wedge \neg \text{inf} \wedge \neg \text{inf}' & \exists s' \cdot X \wedge \neg \text{inf} \wedge \text{inf}' \\ X \wedge \neg \text{inf} \wedge \neg \text{inf}' & \exists s' \cdot X \wedge \text{inf} \wedge \text{inf}' \end{pmatrix}$$

The above is equivalent to $(\exists s' \cdot X) \triangleleft \text{inf}' \triangleright X$, which denotes $\mathcal{H}_3$. □

PROPERTY 5 (QUANTIFICATION).

$$\begin{aligned} \exists \tilde{x} \cdot (\text{HP}_0 \circ \text{HP}_1) &= (\exists \tilde{x} \cdot \text{HP}_0) \circ (\exists \tilde{x} \cdot \text{HP}_1) \\ \forall \tilde{x} \cdot (\text{HP}_0 \circ \text{HP}_1) &\supseteq (\forall \tilde{x} \cdot \text{HP}_0) \circ (\forall \tilde{x} \cdot \text{HP}_1) \\ \exists \tilde{x} \cdot (\text{HP}_0 \sqcap \text{HP}_1) &= (\exists \tilde{x} \cdot \text{HP}_0) \sqcap (\exists \tilde{x} \cdot \text{HP}_1) \\ \forall \tilde{x} \cdot (\text{HP}_0 \sqcap \text{HP}_1) &\supseteq (\forall \tilde{x} \cdot \text{HP}_0) \sqcap (\forall \tilde{x} \cdot \text{HP}_1) \\ \exists \tilde{x} \cdot (\text{HP}_0 \sqcup \text{HP}_1) &\sqsubseteq (\exists \tilde{x} \cdot \text{HP}_0) \sqcup (\exists \tilde{x} \cdot \text{HP}_1) \\ \forall \tilde{x} \cdot (\text{HP}_0 \sqcup \text{HP}_1) &= (\forall \tilde{x} \cdot \text{HP}_0) \sqcup (\forall \tilde{x} \cdot \text{HP}_1) \\ \neg_i \exists \tilde{x} \cdot \text{HP} &= \forall \tilde{x} \cdot \neg_i \text{HP} \\ \neg_i \forall \tilde{x} \cdot \text{HP} &= \exists \tilde{x} \cdot \neg_i \text{HP} \end{aligned}$$

PROOF. We only prove the second item, and others can be proved similarly.

$$\begin{aligned} &(\forall \tilde{x} \cdot \text{HP}_0) \circ (\forall \tilde{x} \cdot \text{HP}_1) \\ &= \wedge \{ \text{HP}_0[f_0, g_0/\tilde{x}, \tilde{x}] \mid f_0 : [ti, ti'] \rightarrow \mathbb{D} \wedge g_0 : (ti, ti') \rightarrow \mathbb{D} \wedge \text{CON}(f_0, g_0, ti, ti') \} \circ \\ &\quad \wedge \{ \text{HP}_1[f_1, g_1/\tilde{x}, \tilde{x}] \mid f_1 : [ti, ti'] \rightarrow \mathbb{D} \wedge g_1 : (ti, ti') \rightarrow \mathbb{D} \wedge \text{CON}(f_1, g_1, ti, ti') \} \\ &= \exists ti_*, s_*, tr_* \cdot \wedge \left\{ \text{HP}_0[ti_*, s_*, tr_*, f_0, g_0/ti', s', tr', \tilde{x}, \tilde{x}] \left| \begin{array}{l} f_0 : [ti, ti'] \rightarrow \mathbb{D} \wedge g_0 : (ti, ti') \rightarrow \mathbb{D} \\ \wedge \text{CON}(f_0, g_0, ti, ti') \end{array} \right. \right\} \\ &\quad \wedge \wedge \left\{ \text{HP}_1[ti_*, s_*, tr_*, f_1, g_1/ti', s', tr', \tilde{x}, \tilde{x}] \left| \begin{array}{l} f_1 : [ti, ti'] \rightarrow \mathbb{D} \wedge g_1 : (ti, ti') \rightarrow \mathbb{D} \\ \wedge \text{CON}(f_1, g_1, ti, ti') \end{array} \right. \right\} \\ &= \exists ti_*, s_*, tr_* \cdot \wedge \left\{ \text{HP}_0[ti_*, s_*, tr_*, f_0, g_0/ti', s', tr', \tilde{x}, \tilde{x}] \left| \begin{array}{l} f_0 : [ti, ti_*] \rightarrow \mathbb{D} \wedge g_0 : (ti, ti_*) \rightarrow \mathbb{D} \wedge \\ \wedge \text{HP}_1[ti_*, s_*, tr_*, f_1, g_1/ti, s, tr, \tilde{x}, \tilde{x}] \left| \begin{array}{l} f_1 : [ti_*, ti'] \rightarrow \mathbb{D} \wedge g_1 : (ti, ti_*) \rightarrow \mathbb{D} \wedge \\ \text{CON}(f_0, g_0, ti, ti_*) \wedge \text{CON}(f_1, g_1, ti_*, ti') \end{array} \right. \end{array} \right. \right\} \\ &= \exists ti_*, s_*, tr_* \cdot \wedge \left\{ \left(\text{HP}_0[ti_*, s_*, tr_*, f_0, g_0/ti', s', tr', \tilde{x}, \tilde{x}] \right) [f, g/\tilde{x}, \tilde{x}] \left| \begin{array}{l} f : [ti, ti'] \rightarrow \mathbb{D} \wedge \\ \text{CON}(f, g, ti, ti') \end{array} \right. \right\} \\ &\sqsubseteq \wedge \left\{ \exists ti_*, s_*, tr_* \cdot \left(\text{HP}_0[ti_*, s_*, tr_*, f_0, g_0/ti', s', tr', \tilde{x}, \tilde{x}] \right) [f, g/\tilde{x}, \tilde{x}] \left| \begin{array}{l} f : [ti, ti'] \rightarrow \mathbb{D} \wedge \\ \text{CON}(f, g, ti, ti') \end{array} \right. \right\} \\ &= \wedge \{ (\text{HP}_0 \circ \text{HP}_1)[f, g/\tilde{x}, \tilde{x}] \mid f : [ti, ti'] \rightarrow \mathbb{D} \wedge g : (ti, ti') \rightarrow \mathbb{D} \wedge \text{CON}(f, g, ti, ti') \} \\ &= \forall \tilde{x} \cdot (\text{HP}_0 \circ \text{HP}_1) \end{aligned}$$

where f is the concatenation of f_0 and f_1 , g is the concatenation of g_0 and g_1 , and

$$\text{CON}(f, g, ti, ti') \hat{=} \forall t \in (ti, ti') \cdot \dot{f}(t^-) = g(t^-) \wedge \dot{f}(t^+) = g(t^+)$$

□

PROPERTY 6 (WELL-DEFINEDNESS). M_I is well-defined.

PROOF. The conditions (1) and (2) of well-definedness of M_I hold by its definition. We prove the condition (3) also holds for M_I . First, if $|tr_X| < +\infty$ and $|tr_Y| < +\infty$, by the definition of $\|_I$, $|tr'| < +\infty$. Then, if $|tr_X| = +\infty$ or $|tr_Y| = +\infty$, say $|tr_X| = +\infty$, there are two cases: (a) $|tr| = +\infty$, since $tr \leq tr'$ by $\mathcal{H}_0$, $|tr'| = +\infty$; (b) $|tr| < +\infty$, meaning $|tr_X - tr| = +\infty$. According to the definition of $\|_I$, the composed trace is not shorter than the individual traces, i.e., $|tr' - tr| \geq |tr_X - tr| = +\infty$, implying $|tr'| = +\infty$. Therefore, the condition (3) holds, indicating M_I is well-defined. □

PROPERTY 7 (MONOTONICITY). The operators $\circ, \sqcup, \sqcap, \triangleleft, \exists$ and $\|_M$ are monotonic and $\neg_i$ is anti-monotone, with respect to the refinement order $\sqsubseteq$.

PROOF. These operators except $\neg_i$ are constructed by monotonic atom operators. □

THEOREM 4.6 (COMPLETE LATTICE). Let $\mathbb{HP}$ be the image of $\mathcal{H}_{HP}$, i.e., it is the set of all hybrid processes, then it forms a complete lattice with top and bottom:

$$\begin{aligned} \top_{HP} &\triangleq \sqcup \mathbb{HP} = \mathcal{H}_{HP}(\text{false}) \\ \perp_{HP} &\triangleq \sqcap \mathbb{HP} = \mathcal{H}_{HP}(\text{true}) \end{aligned}$$

PROOF. Since $\mathcal{H}_{HP}$ is idempotent and monotonic by Theorem 4.2, the complete lattice can be proved directly by the properties of $\sqcap$ and $\sqcup$. □

THEOREM 4.11 (MIRACLE). $\top_{HP}$ is the only miracle of hybrid processes.

PROOF. As mentioned in the proof of Property 2 (see (22)), a hybrid process $\mathcal{H}_{HP}(X)$ can be unfolded as $\mathcal{H}_{014}(\mathfrak{h} \triangleleft \text{inf} \triangleright ((\exists s' \cdot X) \triangleleft \text{inf}' \triangleright X))$, where $\mathcal{H}_{014} \triangleq \mathcal{H}_0 \circ \mathcal{H}_1 \circ \mathcal{H}_4$. Then,

$$\begin{aligned} \mathcal{H}_{HP} \circ \text{MIR}(X) &= \mathcal{H}_{014}(\mathfrak{h} \triangleleft \text{inf} \triangleright ((\exists s' \cdot X \wedge \text{inf}) \triangleleft \text{inf}' \triangleright (X \wedge \text{inf}))) \\ &= \mathcal{H}_{014}(\mathfrak{h} \triangleleft \text{inf} \triangleright (\text{false} \triangleleft \text{inf}' \triangleright \text{false})) \\ &= \mathcal{H}_{014} \circ \mathcal{H}_2 \circ \mathcal{H}_3(\text{false}) = \mathcal{H}_{HP}(\text{false}) \end{aligned}$$

which is equivalent to $\top_{HP}$. □

PROPERTY 8 (LEFT ZERO OF $\circ$). $\top_{HP} \circ HP = \top_{HP}$ for any hybrid process HP .

PROOF. As mentioned in the proof (see (23)) of Property 3, $\top_{HP} \circ \mathcal{H}_{HP}(X)$ is equivalent to

$$\mathcal{H}_{HP}(\text{false}) \circ \mathcal{H}_{HP}(X) = \mathcal{H}_{014}(\mathfrak{h} \triangleleft \text{inf} \triangleright (\exists s' \cdot (\text{false} \circ X \vee \text{false}) \triangleleft \text{inf}' \triangleright (\text{false} \circ X)))$$

which can be simplified to $\mathcal{H}_{HP}(\text{false}) = \top_{HP}$. □

PROPERTY 9 (ZERO OF $\|_M$). $HP \|_M \top_{HP} = \top_{HP} \|_M HP = \top_{HP}$ for any hybrid process HP .

PROOF. As mentioned in the proof (see (22)) of Property 2, $\top_{HP}$ can be unfolded as

$$\mathcal{H}_{014}(\mathfrak{h} \triangleleft \text{inf} \triangleright \text{false}) = \text{inf} \wedge \mathcal{H}_{014}(\mathfrak{h})$$

Then, $\top_{HP,X} = \text{inf} \wedge \mathcal{H}_{014}(\mathfrak{h})_X$ (note that inf has no primed variables). Therefore,

$$\begin{aligned} \top_{HP} \|_M HP &= \mathcal{H}_{HP}(\top_{HP,X} \wedge HP_Y \wedge \mathbb{I} \circ M) \\ &= \mathcal{H}_{HP}(\text{inf} \wedge (\mathcal{H}_{014}(\mathfrak{h})_X \wedge HP_Y \wedge \mathbb{I} \circ M)) \\ &= \mathcal{H}_0 \circ \mathcal{H}_1 \circ \mathcal{H}_4 \circ \mathcal{H}_3 \circ \mathcal{H}_2(\text{inf} \wedge (\mathcal{H}_{014}(\mathfrak{h})_X \wedge HP_Y \wedge \mathbb{I} \circ M)) \end{aligned}$$

where $\mathbb{I} \hat{=} ti = ti' \wedge s = s' \wedge tr = tr'$. Notice that $\mathcal{H}_2(\inf \wedge (\mathcal{H}_{014}(\mathfrak{h})_X \wedge \text{HP}_Y \wedge \mathbb{I} \mathbin{\text{\textcircled{;}}} M))$ is

$$\mathfrak{h} \triangleleft \inf \triangleright (\inf \wedge (\mathcal{H}_{014}(\mathfrak{h})_X \wedge \text{HP}_Y \wedge \mathbb{I} \mathbin{\text{\textcircled{;}}} M)) = \mathfrak{h} \triangleleft \inf \triangleright \text{false} = \mathcal{H}_2(\text{false})$$

which means

$$\top_{\text{HP}} \parallel_M \text{HP} = \mathcal{H}_0 \circ \mathcal{H}_1 \circ \mathcal{H}_4 \circ \mathcal{H}_3 \circ \mathcal{H}_2(\text{false}) = \top_{\text{HP}}.$$

Similarly, we can prove $\text{HP} \parallel_M \top_{\text{HP}} = \top_{\text{HP}}$. $\square$

THEOREM 4.13 (IDEMPOTENCE AND MONOTONICITY). $\mathcal{H}_{\text{HP}}^A$ is idempotent and monotonic.

PROOF. First, $\mathcal{H}_{\text{HP}}^A$ is monotonic as it is constructed by monotonic operators. Then, it can be proved that $\mathcal{H}_0^A, \mathcal{H}_2^A, \mathcal{H}_3^A$ and $\mathcal{H}_4$ are idempotent and commutative with each other, thus $\mathcal{H}_{\text{HP}}^A = \mathcal{H}_0^A \circ \mathcal{H}_2^A \circ \mathcal{H}_3^A \circ \mathcal{H}_4$ is idempotent. $\square$

A.3 Proofs for Hybrid Designs

THEOREM 5.2 (OPERATIONS).

- (1) $(\text{HC}_0 \vdash \text{HP}_0) \sqcap (\text{HC}_1 \vdash \text{HP}_1) = (\text{HC}_0 \wedge \text{HC}_1) \vdash (\text{HP}_0 \vee \text{HP}_1)$
- (2) $(\text{HC}_0 \vdash \text{HP}_0) \sqcup (\text{HC}_1 \vdash \text{HP}_1) = (\text{HC}_0 \vee \text{HC}_1) \vdash ((\text{HC}_0 \Rightarrow_i \text{HP}_0) \wedge (\text{HC}_1 \Rightarrow_i \text{HP}_1))$
- (3) $(\text{HC}_0 \vdash \text{HP}_0) \mathbin{\text{\textcircled{;}}} (\text{HC}_1 \vdash \text{HP}_1) = (\text{HC}_0 \wedge \neg_i(\text{HP}_0 \mathbin{\text{\textcircled{;}}} \neg_i \text{HC}_1)) \vdash (\text{HP}_0 \mathbin{\text{\textcircled{;}}} \text{HP}_1)$

PROOF. The matrix representations of $\text{HC}_0 \vdash \text{HP}_0$ and $\text{HC}_1 \vdash \text{HP}_1$ are respectively

$$\begin{pmatrix} \perp_{\text{HP}} & \perp_{\text{HP}} \\ \neg_i \text{HC}_0 & \text{HC}_0 \Rightarrow_i \text{HP}_0 \end{pmatrix} \quad \text{and} \quad \begin{pmatrix} \perp_{\text{HP}} & \perp_{\text{HP}} \\ \neg_i \text{HC}_1 & \text{HC}_1 \Rightarrow_i \text{HP}_1 \end{pmatrix}$$

Their meet ($\sqcap$, $\vee$) and join ($\sqcup$, $\wedge$) can be computed component-wisely as

$$\begin{pmatrix} \perp_{\text{HP}} \vee \perp_{\text{HP}} & \perp_{\text{HP}} \vee \perp_{\text{HP}} \\ \neg_i \text{HC}_0 \vee \neg_i \text{HC}_1 & (\text{HC}_0 \Rightarrow_i \text{HP}_0) \vee (\text{HC}_1 \Rightarrow_i \text{HP}_1) \end{pmatrix} = \begin{pmatrix} \perp_{\text{HP}} & \perp_{\text{HP}} \\ \neg_i(\text{HC}_0 \wedge \text{HC}_1) & (\text{HC}_0 \wedge \text{HC}_1) \Rightarrow_i (\text{HP}_0 \vee \text{HP}_1) \end{pmatrix}$$

$$\begin{pmatrix} \perp_{\text{HP}} \wedge \perp_{\text{HP}} & \perp_{\text{HP}} \wedge \perp_{\text{HP}} \\ \neg_i \text{HC}_0 \wedge \neg_i \text{HC}_1 & (\text{HC}_0 \Rightarrow_i \text{HP}_0) \wedge (\text{HC}_1 \Rightarrow_i \text{HP}_1) \end{pmatrix} = \begin{pmatrix} \perp_{\text{HP}} & \perp_{\text{HP}} \\ \neg_i(\text{HC}_0 \vee \text{HC}_1) & (\text{HC}_0 \Rightarrow_i \text{HP}_0) \wedge (\text{HC}_1 \Rightarrow_i \text{HP}_1) \end{pmatrix}$$

respectively, which are respectively equivalent to

$$(\text{HC}_0 \wedge \text{HC}_1) \vdash (\text{HP}_0 \vee \text{HP}_1) \text{ and } (\text{HC}_0 \vee \text{HC}_1) \vdash ((\text{HC}_0 \Rightarrow_i \text{HP}_0) \wedge (\text{HC}_1 \Rightarrow_i \text{HP}_1))$$

according to (2). Imagine $\mathbin{\text{\textcircled{;}}}$ and $\vee$ as matrix multiplication and addition, respectively, then

$$\begin{pmatrix} \perp_{\text{HP}} & \perp_{\text{HP}} \\ \neg_i \text{HC}_0 & \text{HC}_0 \Rightarrow_i \text{HP}_0 \end{pmatrix} \mathbin{\text{\textcircled{;}}} \begin{pmatrix} \perp_{\text{HP}} & \perp_{\text{HP}} \\ \neg_i \text{HC}_1 & \text{HC}_1 \Rightarrow_i \text{HP}_1 \end{pmatrix}$$

$$= \begin{pmatrix} (\perp_{\text{HP}} \mathbin{\text{\textcircled{;}}} \perp_{\text{HP}}) \vee (\perp_{\text{HP}} \mathbin{\text{\textcircled{;}}} \neg_i \text{HC}_1) & (\perp_{\text{HP}} \mathbin{\text{\textcircled{;}}} \perp_{\text{HP}}) \vee (\perp_{\text{HP}} \mathbin{\text{\textcircled{;}}} (\text{HC}_1 \Rightarrow_i \text{HP}_1)) \\ (\neg_i \text{HC}_0 \mathbin{\text{\textcircled{;}}} \perp_{\text{HP}}) \vee ((\text{HC}_0 \Rightarrow_i \text{HP}_0) \mathbin{\text{\textcircled{;}}} \neg_i \text{HC}_1) & (\neg_i \text{HC}_0 \mathbin{\text{\textcircled{;}}} \perp_{\text{HP}}) \vee ((\text{HC}_0 \Rightarrow_i \text{HP}_0) \mathbin{\text{\textcircled{;}}} (\text{HC}_1 \Rightarrow_i \text{HP}_1)) \end{pmatrix}$$

$$= \begin{pmatrix} \perp_{\text{HP}} & \perp_{\text{HP}} \\ \neg_i \text{HC}_0 \vee (\text{HP}_0 \mathbin{\text{\textcircled{;}}} \neg_i \text{HC}_1) & \neg_i \text{HC}_0 \vee (\text{HP}_0 \mathbin{\text{\textcircled{;}}} \neg_i \text{HC}_1) \vee (\text{HP}_0 \mathbin{\text{\textcircled{;}}} \text{HP}_1) \end{pmatrix}$$

$$= \begin{pmatrix} \perp_{\text{HP}} & \perp_{\text{HP}} \\ \neg_i(\text{HC}_0 \wedge \neg_i(\text{HP}_0 \mathbin{\text{\textcircled{;}}} \neg_i \text{HC}_1)) & (\text{HC}_0 \wedge \neg_i(\text{HP}_0 \mathbin{\text{\textcircled{;}}} \neg_i \text{HC}_1)) \Rightarrow_i (\text{HP}_0 \mathbin{\text{\textcircled{;}}} \text{HP}_1) \end{pmatrix}$$

which is equivalent to $(\text{HC}_0 \wedge \neg_i(\text{HP}_0 \mathbin{\text{\textcircled{;}}} \neg_i \text{HC}_1)) \vdash (\text{HP}_0 \mathbin{\text{\textcircled{;}}} \text{HP}_1)$ by (2). $\square$

THEOREM 5.3 (PARALLEL COMPOSITION).

$$(\text{HC}_0 \vdash \text{HP}_0) \parallel_{\text{NHD}(M)} (\text{HC}_1 \vdash \text{HP}_1) = \left(\begin{array}{l} \neg_l(\neg_l \text{HC}_0 \parallel_{\perp_{\text{HP}}} \neg_l \text{HC}_1) \\ \wedge \neg_l(\neg_l \text{HC}_0 \parallel_{\perp_{\text{HP}}} \text{HP}_1) \\ \wedge \neg_l(\neg_l \text{HC}_1 \parallel_{\perp_{\text{HP}}} \text{HP}_0) \end{array} \right) \vdash \text{HP}_0 \parallel_M \text{HP}_1$$

PROOF. According to Section 4.2.5, $(\text{HC}_0 \vdash \text{HP}_0) \parallel_{\text{NHD}(M)} (\text{HC}_1 \vdash \text{HP}_1)$ can be unfolded as

$$\mathcal{H}_{\text{HP}}((\text{HC}_0 \vdash \text{HP}_0)_X \wedge (\text{HC}_1 \vdash \text{HP}_1)_Y \wedge \Pi \ ; \ \text{NHD}(M)) \quad (24)$$

where $\Pi \triangleq (ti = ti' \wedge s = s' \wedge tr = tr')$. With the aid of the matrix representation, the predicate $(\text{HC}_0 \vdash \text{HP}_0)_X \wedge (\text{HC}_1 \vdash \text{HP}_1)_Y \wedge \Pi$ can be rewritten as

$$\left(\begin{array}{cccc} \neg ok'_X \wedge \neg ok'_Y & ok'_X \wedge \neg ok'_Y & \neg ok'_X \wedge ok'_Y & ok'_X \wedge ok'_Y \\ \perp_{\text{HP}}^X \wedge \perp_{\text{HP}}^Y \wedge \Pi & \perp_{\text{HP}}^X \wedge \perp_{\text{HP}}^Y \wedge \Pi & \perp_{\text{HP}}^X \wedge \perp_{\text{HP}}^Y \wedge \Pi & \perp_{\text{HP}}^X \wedge \perp_{\text{HP}}^Y \wedge \Pi \\ (\neg_l \text{HC}_0)_X \wedge & (\neg_l \text{HC}_0)_X \wedge \Pi \wedge & (\neg_l \text{HC}_1)_Y \wedge \Pi \wedge & (\text{HC}_0 \Rightarrow_l \text{HP}_0)_X \wedge \Pi \\ (\neg_l \text{HC}_1)_Y \wedge \Pi & (\text{HC}_1 \Rightarrow_l \text{HP}_1)_Y & (\text{HC}_0 \Rightarrow_l \text{HP}_0)_X & \wedge (\text{HC}_1 \Rightarrow_l \text{HP}_1)_Y \end{array} \right) \begin{array}{l} \neg ok \\ ok \end{array}$$

Similarly, $\text{NHD}(M)$ can also be represented by the transpose of the following matrix:

$$\text{NHD}(M)^T = \left(\begin{array}{cccc} \neg ok_X \wedge \neg ok_Y & ok_X \wedge \neg ok_Y & \neg ok_X \wedge ok_Y & ok_X \wedge ok_Y \\ \perp_{\text{HP}} & \perp_{\text{HP}} & \perp_{\text{HP}} & \top_{\text{HP}} \\ \perp_{\text{HP}} & \perp_{\text{HP}} & \perp_{\text{HP}} & M \end{array} \right) \begin{array}{l} \neg ok' \\ ok' \end{array}$$

Then, $(\text{HC}_0 \vdash \text{HP}_0)_X \wedge (\text{HC}_1 \vdash \text{HP}_1)_Y \wedge \Pi \ ; \ \text{NHD}(M)$, where $\wedge$ takes precedence over $\ ;$, is equivalent to the following matrix, by treating $\ ;$ and $\vee$ as matrix multiplication and addition, respectively.

$$\left(\begin{array}{cc} \neg ok' & ok' \\ \perp_{\text{HP}} & \perp_{\text{HP}} \\ (\neg_l \text{HC}_0)_X \wedge (\neg_l \text{HC}_1)_Y \wedge \Pi \ ; \ \perp_{\text{HP}} \vee & (\neg_l \text{HC}_0)_X \wedge (\neg_l \text{HC}_1)_Y \wedge \Pi \ ; \ \perp_{\text{HP}} \vee \\ (\neg_l \text{HC}_0)_X \wedge (\text{HC}_1 \Rightarrow_l \text{HP}_1)_Y \wedge \Pi \ ; \ \perp_{\text{HP}} & (\neg_l \text{HC}_0)_X \wedge (\text{HC}_1 \Rightarrow_l \text{HP}_1)_Y \wedge \Pi \ ; \ \perp_{\text{HP}} \vee \\ \vee (\text{HC}_0 \Rightarrow \text{HP}_0)_X \wedge (\neg_l \text{HC}_1)_Y \wedge \Pi \ ; \ \perp_{\text{HP}} & (\text{HC}_0 \Rightarrow \text{HP}_0)_X \wedge (\neg_l \text{HC}_1)_Y \wedge \Pi \ ; \ \perp_{\text{HP}} \vee \\ & (\text{HC}_0 \Rightarrow \text{HP}_0)_X \wedge (\neg_l \text{HC}_1 \Rightarrow_l \text{HP}_1)_Y \wedge \Pi \ ; \ M \end{array} \right) \begin{array}{l} \neg ok \\ ok \end{array}$$

This matrix can be simplified as follows:

$$\left(\begin{array}{cc} \neg ok' & ok' \\ \perp_{\text{HP}} & \perp_{\text{HP}} \\ (\neg_l \text{HC}_0)_X \wedge (\neg_l \text{HC}_1)_Y \wedge \Pi \ ; \ \perp_{\text{HP}} & (\neg_l \text{HC}_0)_X \wedge (\neg_l \text{HC}_1)_Y \wedge \Pi \ ; \ \perp_{\text{HP}} \\ \vee (\neg_l \text{HC}_0)_X \wedge (\text{HP}_1)_Y \wedge \Pi \ ; \ \perp_{\text{HP}} & \vee (\neg_l \text{HC}_0)_X \wedge (\text{HP}_1)_Y \wedge \Pi \ ; \ \perp_{\text{HP}} \\ \vee (\text{HP}_0)_X \wedge (\neg_l \text{HC}_1)_Y \wedge \Pi \ ; \ \perp_{\text{HP}} & \vee (\text{HP}_0)_X \wedge (\neg_l \text{HC}_1)_Y \wedge \Pi \ ; \ \perp_{\text{HP}} \\ & \vee (\text{HP}_0)_X \wedge (\text{HP}_1)_Y \wedge \Pi \ ; \ M \end{array} \right) \begin{array}{l} \neg ok \\ ok \end{array}$$

Therefore, the hybrid process of (24) is equivalent to the following matrix:

$$\left(\begin{array}{cc} \neg ok' & ok' \\ \perp_{HP} & \perp_{HP} \end{array} \right) \begin{array}{l} \neg ok \\ ok \end{array}$$

$$\left(\begin{array}{cc} (\neg_i HC_0 \parallel_{\perp_{HP}} HP_1) \vee (HP_0 \parallel_{\perp_{HP}} \neg_i HC_1) & (\neg_i HC_0 \parallel_{\perp_{HP}} \neg_i HC_1) \vee (\neg_i HC_0 \parallel_{\perp_{HP}} HP_1) \\ \vee (\neg_i HC_0 \parallel_{\perp_{HP}} \neg_i HC_1) & \vee (HP_0 \parallel_{\perp_{HP}} \neg_i HC_1) \vee (HP_0 \parallel_M HP_1) \end{array} \right)$$

which is equivalent to

$$\neg_i \left(\frac{(\neg_i HC_0 \parallel_{\perp_{HP}} HP_1) \vee (HP_0 \parallel_{\perp_{HP}} \neg_i HC_1)}{\vee (\neg_i HC_0 \parallel_{\perp_{HP}} \neg_i HC_1)} \right) \vdash (HP_0 \parallel_M HP_1)$$

by the matrix representation of hybrid designs (see (2)). □

LEMMA A.2. $\mathcal{H}_{HP}(X) \circ \perp_{HP} = \mathcal{H}_{HP}(X \circ \perp_{HP})$.

PROOF. As mentioned in the proof (see (23)) of Property 3, $\mathcal{H}_{HP}(X) \circ \perp_{HP}$ is equivalent to

$$\mathcal{H}_{HP}(X) \circ \mathcal{H}_{HP}(\perp_{HP}) = \mathcal{H}_{014}(\downarrow \triangleleft \text{inf} \triangleright (\exists s' \cdot (X \circ \perp_{HP} \vee X) \triangleleft \text{inf}' \triangleright X \circ \perp_{HP}))$$

which can be simplified to $\mathcal{H}_{HP}(X \circ \perp_{HP})$ according to (22). □

THEOREM 5.5 (CLOSURE). *Normal hybrid designs are closed on $\sqcap, \sqcup, \circ, \parallel_{NHD(M)}$ and $\parallel_M^{HP}$.*

PROOF. We only prove the closure of normal hybrid designs on $\parallel_{NHD(M)}$. The key is to prove

$$\text{Assumption} \triangleq \neg_i (\neg_i HC_0 \parallel_{\perp_{HP}} HP_1) \wedge \neg_i (HP_0 \parallel_{\perp_{HP}} \neg_i HC_1) \wedge \neg_i (\neg_i HC_0 \parallel_{\perp_{HP}} \neg_i HC_1)$$

is $\mathcal{H}_{HC}$ -healthy, i.e., $\neg_i \text{Assumption} \circ \perp_{HP} = \neg_i \text{Assumption}$. Concretely,

$$\neg_i \text{Assumption} \circ \perp_{HP} = ((\neg_i HC_0 \parallel_{\perp_{HP}} HP_1) \vee (HP_0 \parallel_{\perp_{HP}} \neg_i HC_1) \vee (\neg_i HC_0 \parallel_{\perp_{HP}} \neg_i HC_1)) \circ \perp_{HP}$$

By Lemma A.2, $(\neg_i HC_0 \parallel_{\perp_{HP}} HP_1) \circ \perp_{HP}$ is equivalent to

$$\begin{aligned} \mathcal{H}_{HP}(\neg_i HC_0^X \wedge HP_1^Y \wedge \perp_{HP}) \circ \perp_{HP} &= \mathcal{H}_{HP}(\neg_i HC_0^X \wedge HP_1^Y \wedge \perp_{HP} \circ \perp_{HP}) \\ &= \mathcal{H}_{HP}(\neg_i HC_0^X \wedge HP_1^Y \wedge \perp_{HP}) \\ &= \neg_i HC_0 \parallel_{\perp_{HP}} HP_1 \end{aligned}$$

Similarly, we can prove that $(HP_0 \parallel_{\perp_{HP}} \neg_i HC_1) \circ \perp_{HP}$ and $(\neg_i HC_0 \parallel_{\perp_{HP}} \neg_i HC_1) \circ \perp_{HP}$ are equivalent to $HP_0 \parallel_{\perp_{HP}} \neg_i HC_1$ and $\neg_i HC_0 \parallel_{\perp_{HP}} \neg_i HC_1$, respectively. Thus, $\neg_i \text{Assumption} \circ \perp_{HP} = \neg_i \text{Assumption}$, i.e., Assumption is $\mathcal{H}_{HC}$ -healthy. □

LEMMA A.3. $HP_0 \sqsubseteq HP_1$ iff $HP_0 \sqcap \neg_i HP_1 = \perp_{HP}$ for HP_0 and HP_1 are hybrid processes.

PROOF. $HP_0 \sqsubseteq HP_1$ iff $\neg_i HP_1 \sqsubseteq \neg_i HP_0$ iff

$$\perp_{HP} \sqsubseteq (HP_0 \sqcap \neg_i HP_1) \sqsubseteq (HP_0 \sqcap \neg_i HP_0) = \perp_{HP}$$

according to the monotonicity of $\sqcap$ (Property 7). □

THEOREM 5.6 (REFINEMENT). $(HC_0 \vdash HP_0) \sqsubseteq (HC_1 \vdash HP_1)$ iff

$$HC_1 \sqsubseteq HC_0 \quad \text{and} \quad HP_0 \sqsubseteq (HC_0 \wedge HP_1)$$

PROOF. According to the matrix representation, $(HC_0 \vdash HP_0) \sqsubseteq (HC_1 \vdash HP_1)$ iff

$$\begin{pmatrix} \perp_{HP} & \perp_{HP} \\ \neg_l HC_0 & HC_0 \Rightarrow_l HP_0 \end{pmatrix} \sqsubseteq \begin{pmatrix} \perp_{HP} & \perp_{HP} \\ \neg_l HC_1 & HC_1 \Rightarrow_l HP_1 \end{pmatrix}$$

iff $\neg_l HC_0 \sqsubseteq \neg_l HC_1$ and $(HC_0 \Rightarrow_l HP_0) \sqsubseteq (HC_1 \Rightarrow_l HP_1)$. Then,

$$\begin{aligned} \perp_{HP} &= (HC_0 \Rightarrow_l HP_0) \sqcap \neg_l (HC_1 \Rightarrow_l HP_1) && [\text{Lemma A.3}] \\ &= \neg_l HC_0 \sqcap HP_0 \sqcap (HC_1 \sqcup \neg_l HP_1) \\ &= \neg_l HC_0 \sqcap HP_0 \sqcap \neg_l HP_1 && [\neg_l HC_0 \sqsubseteq \neg_l HC_1 \text{ and Lemma A.3}] \end{aligned}$$

Applying Lemma A.3 again, the above result is equivalent to

$$HP_0 \sqsubseteq \neg_l (\neg_l HC_0 \sqcap \neg_l HP_1) \equiv (HC_0 \wedge HP_1)$$

□

PROPERTY 10 (CONTRA-VARIANCE). *If $HC_0 \sqsupseteq HC_1$ and $HP_0 \sqsubseteq HP_1$ then*

$$(HC_0 \vdash HP_0) \sqsubseteq (HC_1 \vdash HP_1)$$

PROOF. It can be proved directly from Theorem 5.6. □

PROPERTY 11 (MONOTONICITY). *Operators $\sqcup$, $\sqcap$, $\circ$, $\parallel_{NHD(M)}$ and $\parallel_M^{HP}$ for normal hybrid designs are monotonic with respect to $\sqsubseteq$.*

PROOF. These operators are constructed from monotonic atomic operators. □

THEOREM 5.7 (COMPLETE LATTICE). *Normal hybrid designs form a complete lattice with top*

$$\top_{NHD} \hat{=} \perp_{HP} \vdash \top_{HP}$$

and bottom

$$\perp_{NHD} \hat{=} \top_{HP} \vdash \perp_{HP}$$

PROOF. According to Theorem 5.2, $\sqcap$ and $\sqcup$ can be generalised to

$$\begin{aligned} \sqcap_i (HC_i \vdash HP_i) &= (\bigwedge_i HC_i) \vdash (\bigvee_i HP_i) \\ \sqcup_i (HC_i \vdash HP_i) &= (\bigvee_i HC_i) \vdash (\bigwedge_i (HC_i \Rightarrow_l HP_i)) \end{aligned}$$

Therefore, normal hybrid designs form a complete lattice under $\sqsubseteq$ with

$$\begin{aligned} \top_{NHD} &\hat{=} \perp_{HP} \vdash \top_{HP} \\ \perp_{NHD} &\hat{=} \top_{HP} \vdash \perp_{HP} \end{aligned}$$

□

PROPERTY 12 (CHAOS). $\perp_{NHD} = \top_{HP} \vdash HP$ for any hybrid process HP .

PROOF. By the matrix representation,

$$\top_{HP} \vdash HP = \begin{pmatrix} \perp_{HP} & \perp_{HP} \\ \neg_l \top_{HP} & \top_{HP} \Rightarrow_l HP \end{pmatrix} = \begin{pmatrix} \perp_{HP} & \perp_{HP} \\ \perp_{HP} & \perp_{HP} \end{pmatrix} = \perp_{NHD}$$

□

PROPERTY 13 (NON-TERMINATION). *For any normal hybrid design NHD ,*

$$\perp_{NHD} \circ NHD = \perp_{NHD} \quad \text{and} \quad \top_{NHD} \circ NHD = \top_{NHD}$$

PROOF. Let $\text{NHD} \triangleq \text{HC} \vdash \text{HP}$. By matrix representations,

$$\begin{aligned} \perp_{\text{NHD}} \circ \text{NHD} &= \begin{pmatrix} \perp_{\text{HP}} & \perp_{\text{HP}} \\ \perp_{\text{HP}} & \perp_{\text{HP}} \end{pmatrix} \circ \begin{pmatrix} \perp_{\text{HP}} & \perp_{\text{HP}} \\ \neg_{\text{HP}} \text{HC} & \text{HC} \Rightarrow_{\text{HP}} \text{HP} \end{pmatrix} = \begin{pmatrix} \perp_{\text{HP}} & \perp_{\text{HP}} \\ \perp_{\text{HP}} & \perp_{\text{HP}} \end{pmatrix} = \perp_{\text{NHD}} \\ \top_{\text{NHD}} \circ \text{NHD} &= \begin{pmatrix} \perp_{\text{HP}} & \perp_{\text{HP}} \\ \top_{\text{HP}} & \top_{\text{HP}} \end{pmatrix} \circ \begin{pmatrix} \perp_{\text{HP}} & \perp_{\text{HP}} \\ \neg_{\text{HP}} \text{HC} & \text{HC} \Rightarrow_{\text{HP}} \text{HP} \end{pmatrix} = \begin{pmatrix} \perp_{\text{HP}} & \perp_{\text{HP}} \\ \top_{\text{HP}} & \top_{\text{HP}} \end{pmatrix} = \top_{\text{NHD}} \end{aligned}$$

□

PROPERTY 14 (PARALLEL COMPOSITION). *For any normal hybrid design NHD,*

$$\text{NHD} \parallel_{\text{NHD}(M)} \top_{\text{NHD}} = \top_{\text{NHD}} \parallel_{\text{NHD}(M)} \text{NHD} = \top_{\text{NHD}}$$

PROOF. It can be proved by Property 9 and Theorem 5.3. □

PROPERTY 15 (PARALLEL COMPOSITION). *For any normal hybrid design NHD,*

$$\text{NHD} \parallel_M^{\text{HP}} \perp_{\text{NHD}} = \perp_{\text{NHD}} \parallel_M^{\text{HP}} \text{NHD} = \perp_{\text{NHD}}$$

PROOF. Let $\text{NHD} \triangleq \text{HC} \vdash \text{HP}$, then by Definition 5.4 and Property 12 we can get

$$\perp_{\text{NHD}} \parallel_M \text{NHD} = (\top_{\text{HP}} \vdash \perp_{\text{HP}}) \parallel_M (\text{HC} \vdash \text{HP}) = \top_{\text{HP}} \vdash (\perp_{\text{HP}} \parallel_M \text{HP}) = \perp_{\text{NHD}}$$

Similarly, we can prove $\text{NHD} \parallel_M \perp_{\text{NHD}} = \perp_{\text{NHD}}$. □

A.4 Proofs for Reflection of HCSP and Simulink with HUTP

PROPERTY 16 (UNIT). $\llbracket \cdot \rrbracket$ is a unit of hybrid processes w.r.t. $\circ$.

PROOF. According to the intermediate result (23) from Property 3,

$$\begin{aligned} \mathcal{H}_{\text{HP}}(X) \circ \llbracket \cdot \rrbracket &= \mathcal{H}_{\text{HP}}(X) \circ \mathcal{H}_{\text{HP}}(ti = ti' < +\infty \wedge s = s' \wedge tr = tr') \\ &= \mathcal{H}_{014}(\text{tr} < \inf \triangleright ((\exists s' \cdot X) < \inf \triangleright X)) = \mathcal{H}_{\text{HP}}(X) \end{aligned}$$

by the intermediate result (22) from Property 2. Similarly, we can prove $\llbracket \cdot \rrbracket \circ \mathcal{H}_{\text{HP}}(X) = \mathcal{H}_{\text{HP}}(X)$. □

COROLLARY 1 (UNIT). $\vdash \llbracket \cdot \rrbracket$ is a unit of hybrid processes w.r.t. $\circ$.

PROOF. According to Property 16 and Theorem 5.2(3), for any normal hybrid design $\text{HC} \vdash \text{HP}$,

$$\begin{aligned} (\vdash \llbracket \cdot \rrbracket) \circ (\text{HC} \vdash \text{HP}) &= (\perp_{\text{HP}} \wedge \neg_{\text{HP}}(\llbracket \cdot \rrbracket \circ \neg_{\text{HP}} \text{HC})) \vdash (\llbracket \cdot \rrbracket \circ \text{HP}) = \text{HC} \vdash \text{HP} \\ (\text{HC} \vdash \text{HP}) \circ (\vdash \llbracket \cdot \rrbracket) &= (\text{HC} \wedge \neg_{\text{HP}}(\text{HP} \circ \neg_{\text{HP}} \perp_{\text{HP}})) \vdash (\text{HP} \circ \llbracket \cdot \rrbracket) = \text{HC} \vdash \text{HP} \end{aligned}$$

□

THEOREM 6.1 (SEMANTICS CONSISTENCY). *The HUTP semantics of HCSP is consistent with the operational semantics of HCSP.*

PROOF. We say the HUTP semantics and the structural operational semantics (SOS) of an HCSP process are consistent iff they keep consistent on time, state and trace. The proof for the consistency is divided into two stages: first, we prove the consistency between the SOS and the HUTP semantics of the syntactic entities in (3); then, we prove the semantics consistency for the operators, such as sequential and parallel compositions, which connect the syntactic entities. For brevity, we just show the key points of the proof in the following content.

The semantics consistency for ODE with communication interruption ($\text{ODE}_{\geq}$) has been proved in the end of Section 6.2. $\text{ODE}_{\geq}$ is representative because it covers the characteristic features of HCSP, such as continuous evolution, communication and interruption. The semantics consistency for other syntactic entities of HCSP like $\langle F(\dot{s}, s) = 0 \& B \rangle$ and $ch?x$ can be proved similarly. So, we only focus on proving the semantics consistency for the connector of parallel composition as it plays an important role in composite systems.

The SOS for parallel composition of HCSP processes is described by the following rules, where I is the common channel set between parallel operands.

$$\begin{array}{c}
\frac{}{(\varepsilon \parallel \varepsilon, (ti, s_P \uplus s_Q)) \rightarrow (\varepsilon, (ti, s_P \uplus s_Q))} [\text{Par0}] \quad \frac{(P, (ti, s_P)) \xrightarrow{\tau} (P', (ti, s'_P))}{(P \parallel Q, (ti, s_P \uplus s_Q)) \xrightarrow{\tau} (P' \parallel Q, (ti, s'_P \uplus s_Q))} [\text{Par1}] \\
\\
\frac{\begin{array}{c} ch \in I \quad (P, s_P) \xrightarrow{\langle ch?, d \rangle} (P', s'_P) \\ (Q, s_Q) \xrightarrow{\langle ch!, d \rangle} (Q', s'_Q) \end{array}}{(P \parallel Q, s_P \uplus s_Q) \xrightarrow{\langle ch, d \rangle} (P' \parallel Q', s'_P \uplus s'_Q)} [\text{Par2}] \quad \frac{ch \notin I \quad (P, (ti, s_P)) \xrightarrow{\langle ch*, d \rangle} (P', (ti, s'_P))}{(P \parallel Q, s_P \uplus s_Q) \xrightarrow{\langle ch*, d \rangle} (P' \parallel Q, s'_P \uplus s_Q)} [\text{Par3}] \\
\\
\frac{\begin{array}{c} (P, (ti, s_P)) \xrightarrow{\langle ti' - ti, RS_P \rangle} (P', (ti', s'_P), \underline{s}_P) \\ (Q, (ti, s_Q)) \xrightarrow{\langle ti' - ti, RS_Q \rangle} (Q', (ti', s'_Q), \underline{s}_Q) \quad \overline{RS}_P \cap RS_Q = \emptyset \end{array}}{(P \parallel Q, (ti, s_P \uplus s_Q)) \xrightarrow{\langle ti' - ti, RS_P \uplus RS_Q \rangle} (P' \parallel Q', (ti', s'_P \uplus s'_Q), \underline{s}_P \uplus \underline{s}_Q)} [\text{Par4}]
\end{array}$$

Since parallel HCSP processes do not share variables, the HUTP semantics and the SOS of the parallel connector are consistent on state variables naturally. Then, we focus on the parallel composition of timed traces, i.e., prove the consistency on timed traces. The above five rules correspond to the respective five composition rules of timed traces in Section 3.3. Concretely, [Par0], [Par1], [Par2], [Par3] and [Par4] relate to [Empty], [τ -Act], [SynIO], [NoSynIO] and [SynWait], respectively.

Consider two HCSP processes P and Q with respective states s_P and s_Q . Their execution histories are recorded by timed traces tt_P and tt_Q , respectively, denoted by

$$(P, (ti, s_P)) \xrightarrow[\ast]{\text{tt}_P} (\varepsilon, (ti', s'_P), \underline{s}_P) \quad \text{and} \quad (Q, (ti, s_Q)) \xrightarrow[\ast]{\text{tt}_Q} (\varepsilon, (ti', s'_Q), \underline{s}_Q)$$

where $\underline{s}_P$ and $\underline{s}_Q$ can be removed if there is no flow generated. The proof is by induction because all the traces involved are finite in the semantics of HCSP. So, this proceeds by induction on the length of traces and start from cases that $|\text{tt}_P|, |\text{tt}_Q| \leq 1$, such as $\text{tt}_P = \text{tt}_Q = \varepsilon$ or tt_P and tt_Q are just trace blocks. Then, by induction on the derivation of $\text{tt}_P \parallel_I \text{tt}_Q \leadsto \text{tt}$, we can prove that the parallel composition $\parallel_I$ of timed traces in HUTP can be interpreted by the structural operational semantics of the parallel composition of HCSP. On the other side, we can prove that the SOS of the parallel composition of HCSP can also be interpreted by $\parallel_I$ of timed traces in HUTP by induction on the derivation of

$$(P \parallel Q, (ti, s_P \uplus s_Q)) \xrightarrow[\ast]{\text{tt}} (\varepsilon, (ti', s'_P \uplus s'_Q))$$

from the cases that $|\text{tt}| \leq 1$. In summary, the SOS and the HUTP semantics of the parallel composition of HCSP are consistent on time, state and trace. The semantics consistency for other operators can be proved similarly. $\square$